

Frontend Development

Complete Course Version 1.0

(Beginner to Advanced)

Prepared By: Engr.Umar Naveed

(Learn HTML, CSS, JavaScript, React, Git & GitHub from Beginner to Advanced with Real-World Projects)

Compiled & Prepared for Deep, Step-by-Step Learning

About This Course:

Welcome to the Complete Frontend Development Course.

Ye course beginners aur intermediate students ke liye design kiya gaya hai jo professional Frontend Developer banna chahte hain.

Is course mein aap seekhenge:

- HTML5
- CSS3
- Responsive Web Design
- Flexbox & CSS Grid
- JavaScript (Basic to Advanced)
- DOM Manipulation
- Fetch API & REST APIs
- Git & GitHub
- React.js (Basic to Advanced)
- React Router
- Context API
- Deployment
- Real-World Projects

Course complete karne ke baad aap modern aur responsive websites aur web applications banana seekh jayenge.

Instructor :
Engr.Umar Naveed

Introduction to Frontend Development:

What is Frontend Development?

Ek Frontend Developer ka kaam website ko attractive, responsive aur user-friendly banana hota hai taake website desktop, tablet aur mobile sab devices par sahi tarah kaam kare.

Frontend Development ki bunyad 3 core technologies par hoti hai:

- **HTML (HyperText Markup Language):** Website ka structure banata hai.
- **CSS (Cascading Style Sheets):** Website ko design aur styling deta hai. □ **JavaScript:** Website ko interactive aur dynamic banata hai.

In technologies ke ilawa modern Frontend Development mein React.js, Git, GitHub, APIs aur deployment tools bhi use kiye jate hain.

Why Learn Frontend Development?

Frontend Development seekhne ke kai faide hain:

- Professional websites aur web applications bana sakte hain.
- Freelancing aur remote jobs ke opportunities milti hain.
- Modern JavaScript frameworks jaise React.js seekh sakte hain.
- Apne ideas ko websites ki shakal mein convert kar sakte hain.
- Software House ya IT industry mein Frontend Developer ke taur par career bana sakte hain.

Course ke end tak aap modern, responsive aur professional websites aur web applications banana seekh jayenge aur real-world projects par kaam karne ke liye tayyar honge.

Index — Table of Contents

Frontend Development	1
Complete Course Version 1.0	1
About This Course:	2
Introduction to Frontend Development:	3
What is Frontend Development?	3
Why Learn Frontend Development?	3
Chapter # 1 HTML	20
Module 1: Introduction.....	20
What is HTML?	20
History of HTML	20
HTML Versions	20
HTML Editors	21
VS Code Setup	21
HTML File Structure	22
HTML Boilerplate	22
DOCTYPE	23
HTML Elements	23
HTML Attributes	23

Module 2: Text Formatting	24
Headings (h1 to h6)	24
Paragraphs (<p>)	24
Line Break ()	24
Horizontal Rule (<hr>).....	24
Comments	25
Text Formatting Tag 	25
Text Formatting Tag 	25
Text Formatting Tag <i>	25
Text Formatting Tag 	26
Text Formatting Tag <u>	26
Text Formatting Tag <mark>	26
Text Formatting Tag <small>	26
Text Formatting Tag 	26
Text Formatting Tag <ins>.....	26
Text Formatting Tag <sub>	27
Text Formatting Tag <sup>	27
Module 3: Links	27
Anchor Tag (<a>)	27
Relative Links	27
Absolute Links	27
Open Link in New Tab	27
Email Links	28
Phone Links	28
Download Links	28
Bookmark Links (Same-page anchors)	28
Module 4: Images	28
Image Tag ()	28
Image Formats.....	29
Responsive Images	29
Figure & Figcaption	29
Image Maps	29

Module 5: Lists	30
Ordered List ()	30
Unordered List ()	30
Description List (<dl>)	30
Nested Lists	30
Module 6: Tables	31
Table Structure	31
Rows & Columns.....	31
Table Header (<th>)	31
Caption	32
colspan	32
Output / Result:"full bame" heading 2 ki width rowspan tak spread hoga.	32
Rowspan.....	32
Table Styling Basics	33
Module 7: Forms	33
Form Tag	33
Input Types	33
Labels	34
Placeholder	34
Required	34
Readonly	34
Disabled	34
Select & Option	35
Textarea	35
Button	35
Fieldset	35
Legend	35
Datalist	35
Output	36
Module 8: Semantic HTML	36
Header	36
Nav	36

Main	36
Section	37
Article	37
Aside	37
Footer	37
Figure	38
Details & Summary	38
Time	38
Module 9: Multimedia	38
Audio	38
Video	39
Source	39
Track	39
iframe.....	39
YouTube Embed	39
Module 10: HTML Entities.....	40
Symbols (Special Characters)	40
Emojis	40
Character Entities	40
Module 11: Meta Tags	40
40 charset	40
viewport	40
description	41
keywords	41
author	41
favicon	41
Module 12: SEO Basics	41
Title Tag.....	41
Meta Description	41
Heading Structure.....	42
Semantic HTML for SEO	42
Image Alt Attribute for SEO.....	42

Module 13: Accessibility	42
42 alt Attribute	42
aria-label	42
aria-hidden	43
tabindex	43
Screen Readers	43
Module 14: Best Practices.....	43
Clean Code	43
Proper Indentation	44
File Naming	44
Folder Structure	44
Validation	44
Module 15: HTML Projects - Personal Portfolio Project:	45
Project Steps:	45
Complete Project Code:	45
Practice Karne Ke Liye Baaki Project Ideas:	47

Chapter # 2 CSS

48

Module 1: Introduction.....	48
What is CSS?	48
Types of CSS	48
Selectors.....	48
Comments	49
Module 2: Colors & Units	49
Colors (Named Colors)	49
HEX Colors	49
RGB Colors.....	50
RGBA Colors (with Transparency)	50
HSL Colors	50
Units (px, %, rem, em, vw, vh).....	50

Module 3: Backgrounds	
51 background-color	51
background-image	51
background-repeat	51
background-position	51
background-size	52
background-attachment	52
Module 4: Borders	52
Border	52
Border Radius	52
Outline	53
Module 5: Box Model	53
Box Model Overview	53
Margin	53
Padding	54
Content	54
Box-sizing	54
Module 6: Typography	55
Fonts (font-family)	55
Google Fonts	55
Font Weight.....	55
Line Height	55
Letter Spacing	56
Align	56
Text Transform	56
Text Decoration	56
Module 7: Display	
57 display: block	57
display: inline	57
display: inline-block	57
display: none	58
Module 8: Position	
58 position: static	58

position: relative.....	58
position: absolute	58
position: fixed	59
position: sticky.....	59
index	59
Module 9: Flexbox	60
Flex Container	60
Flex Items & flex-direction	60
justify-content	60
align-items.....	60
gap	61
flex-wrap	61
Module 10: CSS Grid	61
Grid Container	61
Grid Items.....	62
Grid Template Columns/Rows	62
Grid Area	62
Gap (Grid)	62
Module 11: Responsive Design	63
Media Queries	63
Mobile First Approach	63
Breakpoints	63
Module 12: Transforms.....	64
translate	64
rotate	64
scale	64
skew	64
Module 13: Transitions	65
CSS Transitions	65
Multiple Property Transitions	65
Module 14: Animations	65
CSS Animations (@keyframes)	65
Animation Properties	66

Module 15: Pseudo Classes	66
What are Pseudo Classes	66
:hover, :focus, :active.....	66
:nth-child().....	67
:first-child & :last-child	67
:not()	67
Module 16: Pseudo Elements	67
::before and ::after	67
::first-letter and ::first-line	68
::selection	68
Module 17: CSS Variables	68
Defining & Using CSS Variables	68
CSS Variables for Theming (Dark Mode Example)	69
Module 18: Shadows	69
box-shadow	69
text-shadow	69
Module 19: Gradients	70
Linear Gradient	70
Radial Gradient	70
Gradient as Text (Creative Use).....	70
Module 20: Best Practices	70
Consistent Naming Convention (BEM)	70
Avoid !important	71
Organize CSS with Comments & Sections	71
Use Shorthand Properties	71
Module 21: CSS Projects - Responsive Landing Page	72
Project Steps:	72
Complete Project Code — index.html:	72
Complete Project Code — style.css:	73
Practice Karne Ke Liye Baaki Project Ideas:	75
Chapter # 3 JAVASCRIPT (ES6+)	76

Module 1: Introduction & Basics	76
Introduction to JavaScript	76
Variables (var, let, const)	76
Data Types.....	76
Operators	77
Type Conversion	77
Module 2: Strings & Numbers	77
Strings & String Methods	77
Numbers & Number Methods	78
Math Object	78
Template Literals	78
Module 3: Control Flow	79
Conditionals (if, else if, else)	79
Switch Statement	79
Loops (for, while)	79
break & continue	80
Module 4: Functions & Scope	80
Functions	80
Default & Rest Parameters	80
Scope (Global, Function, Block)	81
Hoisting	81
Closures	81
'this' Keyword	82
Module 5: Arrays, Objects & Dates	82
Arrays	82
Array Methods: map, filter, reduce	83
Array Methods: forEach, find, sort	83
Objects	83
Object Methods (keys, values, entries)	84
Dates	84
Module 6: DOM & Events	84
What is the DOM?.....	84
Selecting Elements	85

Modifying Elements	85
Creating & Removing Elements	85
Events (addEventListener)	85
Event Delegation	86
Module 7: Classes, OOP & Modules	86
Classes	86
Inheritance	87
OOP Principles (Encapsulation, Abstraction)	87
Modules (import/export)	88
Module 8: ES6+ Features	88
let & const (ES6 Basics)	88
Destructuring	88
Spread Operator	89
Rest Parameter	89
Module 9: Asynchronous JavaScript	89
Callbacks	89
Promises	90
Async/Await	90
Fetch API	90
JSON	91
Module 10: Browser Storage	91
Local Storage	91
Session Storage	91
Module 11: Error Handling & Regular Expressions	92
Error Handling (try, catch, finally)	92
Regular Expressions (Regex)	92
Module 12: Advanced Concepts	93
Debounce & Throttle	93
Execution Context & Call Stack	93
Event Loop	93
Web APIs (Browser APIs)	94
Module 13: Mini Projects - Todo List App	94

Project Steps:	94
Chapter # 4 Git Version Control	98
Module 1: Getting Started with Git	98
Git Installation	98
Config	98
git init	98
Module 2: Basic Git Commands	99
git status	99
git add	99
git commit	99
git log	99
git diff	99
Module 3: Undoing Changes	100
git restore	100
git reset	100
Module 4: Branching & Merging	100
Branches	100
Merge	100
Rebase (Basic)	101
Module 5: Stash & Tags	101
Stash	101
Tags	101
Chapter # 5 : GitHub	102
Module 1: GitHub Basics	102
GitHub Introduction	102
Create Repository	102
Clone	102
Module 2: Syncing Code	102
Push	102

Pull	103
Module 3: Collaboration	103
Fork	103
Pull Request	103 Issues
.....	103
Module 4: Documentation & Hosting	104
README	104
GitHub Pages	104
 Chapter # 6 : npm & Node.js	
.....	105
Module 1: Node.js & npm Basics	105
Node.js	105
npm	105
Module 2: package.json Deep Dive	
105 package.json	
105 Dependencies	
105	
Dev Dependencies	106
Scripts	106
Module 3: Managing Packages	106
Install/Uninstall Packages	106
 Chapter # 7: Vite	
.....	107
Module 1: Getting Started with Vite	107
Installation	107
Project Structure	107
Module 2: Development & Build.....	107
Dev Server	107
Build	108
Environment Variables	108

Chapter # 8 : React.js

.....	109
Module 1: React Fundamentals	109
Introduction	109
JSX	109
Components	109
Module 2: Props & State	110
Props	110
State	110
Module 3: Handling Events & Rendering	110
Event Handling	110
Conditional Rendering	111
Lists & Keys.....	111
Module 4: Forms in React	111
Forms	111
Controlled Components	112
Module 5: React Hooks	112
112 useState	112
112 useEffect	112
113 useRef	113
113 useMemo	113
113 useCallback.....	113
113 useContext	113
114 useReducer	114
114	114
Custom Hooks	114
Module 6: Component Patterns	115
Component Lifecycle	115
Lifting State Up	115
Composition	115
Portals	116
Error Boundaries	116

Module 7: Performance & Code Splitting	116
Lazy Loading	116
Suspense	117
Code Splitting	117
Performance Optimization	117
Module 8: Advanced React Patterns & Modern Hooks	117
Fragments	117
forwardRef & useImperativeHandle.....	118
useLayoutEffect	118
useTransition & useDeferredValue	119
Higher-Order Components (HOC)	119
Render Props	119
React.StrictMode	120
PropTypes	120
TypeScript with React (Introduction)	120
 Chapter # 9 : React Router	 122
Module 1: Setting Up React Router.....	122
Installation	122
BrowserRouter	122
Routes	122
Route	122
Module 2: Navigation	123
Link	123
NavLink	123
useNavigate	123
useParams	123
Module 3: Advanced Routing.....	124
Nested Routes	124
Layout Routes	124
Protected Routes	124
404 Page.....	125

Chapter # 10 : API Integration

.....	126
Module 1: REST API	126
HTTP Methods	126
Module 2: Making Requests	126
Fetch API	126
Axios	126
Module 3: Handling Data & States	127
CRUD Operations	127
Error Handling	127
Loading State	127

Chapter 11 : Tailwind CSS

.....	129
Module 1: Getting Started with Tailwind	129
Installation	129
Utility Classes	129
Module 2: Styling Basics	129
Spacing	129
Colors	129
Typography	130
Module 3: Layout with Tailwind	130
Flexbox	130
Grid	130
Responsive Design	130
Module 4: Advanced Tailwind	131
Dark Mode	131
Custom Theme	131

Chapter # 12 : Context API

.....	132
Module 1: Understanding Context API.....	132
Context.....	132

Provider.....	132
Consumer	132

Module 2: Using Context in Practice	132
132 useContext	132
132 Global State	133

Chapter # 13: Redux Toolkit

.....	134
Module 1: Redux Toolkit Basics	134
Store	134
Slice	134
Reducers	134
Actions	135
Module 2: Connecting Redux to React	135
135 Dispatch	135
135 useSelector	135
135 useDispatch	135
135	135
Module 3: Async Data & RTK Query	136
Async Thunk	136
RTK Query (Introduction)	136

Chapter	#	14:	Authentication
.....		137	

Module 1: Authentication Basics	137
JWT	137
Login	137
Signup	137
Module 2: Securing the App.....	137
Protected Routes	137
Token Storage	138

Refresh Token (Basic)	138
Chapter # 15 : React Query (TanStack Query)	139
Module 1: React Query Basics	
139 Query Client	
139 useQuery	
139	
Query Keys	139
Module 2: Mutations & Cache	
140 useMutation	
140 Caching.....	
140	
Invalidation	140
Module 3: Advanced Data Fetching	140
Pagination	140
Infinite Queries	141
Chapter 16: Testing	142
Module 1: Testing Tools	142
Jest	142
React Testing Library	142
Module 2: Writing Tests	142
Unit Testing	142
Component Testing	143
Chapter # 17 : Deployment	144
Module 1: Preparing for Deployment	144
Build Project	144
Environment Variables	144
Module 2: Hosting Your App	144
Deploy on Vercel	144
Deploy on Netlify	144

Custom Domain (Basic)	145
-----------------------------	-----

Chapter # 18 : Advanced Projects

.....	146
Module 1: Beginner-Friendly Projects	146
Portfolio Website	146
Todo App	146
Weather App	146
Notes App	146
Module 2: Advanced Projects	147
E-commerce Store	147
Admin Dashboard	147
Chat App	147
Blog CMS	147
Movie App	147
Food Delivery App	148

Chapter # 19: Interview Preparation

.....	149
Module 1: Technical Interview Questions	149
HTML Interview Questions.....	149
CSS Interview Questions	149
JavaScript Interview Questions	149
React Interview Questions	150
Redux Interview Questions	150
Git & GitHub Questions	151
Module 2: Non-Technical Preparation	151
HR Interview Questions	151
Resume Building	151
Portfolio Building	152
Final Thoughts	153
Prepared By: Engr. Umar Naveed	153

Chapter # 1 HTML

Is section mein HTML ke tamam 15 modules deeply, beginner-level se explain kiye gaye hain — har topic ke sath code example, uska output, aur real-world use case diya gaya hai. Aakhir mein ek complete project bhi banaya gaya hai jisme ye saare concepts combine ho kar use hote hain.

Module 1: Introduction

What is HTML?

HTML (HyperText Markup Language) woh basic language hai jis se hum web pages banate hain. Ye ek 'markup' language hai, programming language nahi - iska matlab hai ye sirf content ko structure aur meaning deta hai (jaise heading, paragraph, image, link), logic ya calculations nahi karta. Har website jo aap browser mein dekhte hain, uske peeche HTML hi hoti hai jo text, images, buttons, forms waghera ko arrange karti hai. Beginner ke liye samajhna zaroori hai: HTML = Skeleton/Structure, CSS = Design/Styling, JavaScript = Behavior/Interactivity. Teenon milkar ek complete website banate hain.

Code Example:

```
<!-- Ye simplest HTML page hai -->
<h1>Hello World</h1>
<p>Ye mera pehla HTML page hai.</p>
```

Output / Result: Browser mein ek bara 'Hello World' heading aur uske neeche ek paragraph show hoga.

Real-World Example: Jab aap Google.com kholte hain, us page ka structure (search box, buttons, links) HTML se hi bana hota hai.

History of HTML

HTML ko 1991 mein Tim Berners-Lee ne invent kiya tha jab woh CERN mein kaam kar rahe thay, taake scientists aasani se documents share kar saken. Waqt ke sath HTML evolve hoti gayi: HTML 2.0 (1995) pehli official standard thi, HTML 3.2 (1997) mein tables aur applets aaye, HTML 4.01 (1999) ek stable aur widely-used version tha, aur phir XHTML aaya jo XML rules follow karta tha. 2014 mein HTML5 release hui jo aaj tak use ho rahi hai aur bohot saare naye features (audio, video, canvas, semantic tags) le kar aayi.

Code Example:

```
<!-- HTML5 ka doctype -->
<!DOCTYPE html>
```

Output / Result: Ye line browser ko batati hai ke document HTML5 standard follow karta hai.

Real-World Example: Aaj koi bhi modern website (Facebook, YouTube) HTML5 par based hai.

HTML Versions

HTML ke different versions time ke sath aaye: HTML 1.0, HTML 2.0, HTML 3.2, HTML 4.01, XHTML 1.0, aur ab HTML5. Har version mein purani cheezein improve hui aur nayi cheezein add hui. HTML5 sabse important version hai kyunke isme semantic tags (header, footer, section, article), multimedia support (audio/video bina plugin ke), aur form validation jaise features aaye. Aaj hum sirf HTML5 hi seekhte aur use karte hain, purane versions sirf history ke liye jaanna zaroori hai.

Code Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTML5 Page</title>
</head>
<body>
  <h1>Ye HTML5 hai</h1>
</body>
</html>
```

Output / Result: Ek complete HTML5 page structure browser mein render hoga.

Real-World Example: Interview mein aksar poocha jata hai 'HTML5 mein naya kya hai?' - iska jawab semantic tags aur multimedia support hota hai.

HTML Editors

HTML likhne ke liye aap kisi bhi text editor ka use kar sakte hain jaise Notepad, lekin professional development ke liye code editors use hote hain jo syntax highlighting, auto-complete, aur extensions provide karte hain. Popular editors hain: VS Code (sabse zyada used aur free), Sublime Text, Atom, aur online editors jaise CodePen, JSFiddle jo bina install kiye directly browser mein code likhne dete hain. Beginners ko VS Code recommend kiya jata hai kyunke ye lightweight, free, aur bohot saare helpful extensions rakhta hai.

Code Example:

```
<!-- Koi bhi editor mein ye file .html extension se save karein --><!DOCTYPE html>
<html>
<body>
  <h1>Editor Test</h1>
</body>
</html>
```

Output / Result: File ko save karke double-click karein, browser mein khul jayegi.

Real-World Example: Professional developers roz VS Code use karte hain apna code likhne ke liye.

VS Code Setup

VS Code (Visual Studio Code) install karne ke liye code.visualstudio.com se download karein aur install karein. Setup ke baad kuch important extensions install karein: 'Live Server' (jo file save karte hi browser auto-refresh karta hai), 'Prettier' (code ko automatically format karta hai), aur 'HTML CSS Support'. Ek folder banayein apne project ke liye, VS Code mein 'File > Open Folder' se woh folder kholein, phir naya file banayein jaise index.html. Live Server extension install karne ke baad file par right-click karke 'Open with Live Server' select karein, ye browser mein page kholega aur har save par autorefresh karega.

Code Example:

```
<!-- index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>My First Project</title> </head>
<body>
  <h1>VS Code Setup Successful!</h1>
</body>
</html>
```

Output / Result: Live Server se browser mein page open hoga aur real-time update dikhega.

Real-World Example: Har professional developer apna project isi tarah setup karta hai - folder banao, editor kholo, live preview lagao.

HTML File Structure

Har HTML file ka ek fixed basic structure hota hai jisme kuch main parts hote hain: DOCTYPE declaration (top par), html tag (root element), head tag (jisme metadata, title, links hote hain jo user ko nahi dikhte), aur body tag (jisme woh actual content hota hai jo browser mein visible hota hai). Ye structure hamesha follow karna zaroori hai warna browser page ko sahi tarah render nahi kar payega.

Code Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Page Title</title>
</head>
<body>
  <h1>Ye content visible hoga</h1>
</body>
</html>
```

Output / Result: Sirf body ke andar ka content browser window mein show hota hai, head ka content hidden hota hai.

Real-World Example: Ye structure har website ke liye same rehta hai chahe woh Amazon ho ya koi personal blog.

HTML Boilerplate

Boilerplate ka matlab hai woh standard starting code jo har HTML file mein repeat hota hai. VS Code mein sirf '!' likh kar Tab (ya Enter) dabane se ye poora boilerplate automatically generate ho jata hai (Emmet abbreviation). Isme DOCTYPE, html, head (with charset aur viewport meta tags), title, aur empty body شامل hote hain. Ye time bachata hai aur ensure karta hai ke aap koi important tag miss na karein.

Code Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
</head>
<body>
</body>
</html>
```

Output / Result: Ye empty lekin complete valid HTML page banata hai jise aap fill kar sakte hain.

Real-World Example: Har naya HTML file banate waqt developers pehle ye boilerplate type karte hain (ya Emmet se generate karte hain).

DOCTYPE

DOCTYPE (Document Type Declaration) sabse pehli line hoti hai kisi bhi HTML file ki. Ye browser ko batati hai ke ye document kis HTML version mein likha gaya hai, taake browser usi standard ke mutabiq page render kare. HTML5 mein DOCTYPE bohot simple ho gaya hai - sirf '<!DOCTYPE html>'. Agar DOCTYPE missing ho, to browser 'quirks mode' mein chala jata hai jahan purane, inconsistent rendering rules use hote hain jo layout ko kharab kar sakte hain.

Code Example:

```
<!DOCTYPE html>
<html>
<head><title>Doctype Example</title></head>
<body>
  <p>Is page mein sahi doctype hai.</p>
</body>
</html>
```

Output / Result: Browser standards mode mein page render karega, consistent aur predictable layout milega.

Real-World Example: Agar DOCTYPE galat ya missing ho to CSS ka box-model different browsers mein alag tarah behave kar sakta hai.

HTML Elements

HTML Element ek complete unit hota hai jisme opening tag, content, aur closing tag shamil hota hai - jaise '<p>Hello</p>'. Kuch elements 'void elements' hote hain jinka closing tag nahi hota, jaise '', '
', '<hr>'. Elements ek dusre ke andar 'nest' ho sakte hain, jise parent-child relationship kehte hain. HTML ka poora page dar-asal elements ka ek tree structure hota hai (DOM tree) jise browser parse karke render karta hai.

Code Example:

```
<div>
  <h2>Parent Element</h2>
  <p>Ye <strong>child element</strong> hai jo parent ke andar hai.</p></div>
```

Output / Result: Heading aur paragraph dono div ke andar nested honge, aur 'child element' words bold dikhenge.

Real-World Example: Facebook ka poora page hazaron nested HTML elements ka tree hai jo mil kar layout banate hain.

HTML Attributes

Attributes elements ko extra information dete hain aur hamesha opening tag ke andar likhe jate hain, format hota hai: name="value". Common attributes hain: id (unique identifier), class (styling ke liye group), src (image/media source), href (link destination), alt (image description), style (inline CSS). Ek element mein multiple attributes ho sakte hain, space se separate karke.

Code Example:

```
  
<a href="https://google.com" target="_blank" title="Visit Google">Google Jayein</a>
```

Output / Result: Pehli line ek image dikhayegi apni width ke sath, dusri line ek clickable link banayegi jo naye tab mein khulega.

Real-World Example: id aur class attributes CSS aur JavaScript mein specific elements ko target karne ke liye har website mein use hote hain.

Module 2: Text Formatting

Headings (h1 to h6)

HTML mein 6 heading levels hote hain: h1 (sabse bara aur important) se h6 (sabse chota) tak. Har page mein sirf ek h1 use karna best practice hai (usually page ka main title), baaki headings content ko hierarchy mein organize karne ke liye use hoti hain jaise ek article mein subheadings. Headings sirf 'bara text' nahi hain - ye SEO aur accessibility ke liye bhi bohot important hain, kyunke screen readers aur search engines inhe page ki structure samajhne ke liye use karte hain.

Code Example:

```
<h1>Main Website Title</h1>  
<h2>Section Heading</h2>  
<h3>Sub-section Heading</h3>  
<h4>Smaller Heading</h4>  
<h5>Even Smaller</h5>  
<h6>Smallest Heading</h6>
```

Output / Result: Har heading ka size h1 se h6 tak chota hota jayega, sab bold aur apne default spacing ke sath.

Real-World Example: News website (jaise BBC) mein article ka title h1 hota hai aur uske andar ke sections h2, h3 hote hain.

Paragraphs (<p>)

Paragraph tag normal text content ko wrap karne ke liye use hota hai. Browser automatically har paragraph ke pehle aur baad mein thoda space (margin) add karta hai. Multiple spaces ya line breaks jo aap HTML file mein type karte hain, browser unhe ignore karke sirf ek space dikhata hai (jab tak aap CSS 'white-space' property use na karein).

Code Example:

```
<p>Ye pehla paragraph hai. Isme jitni bhi      spaces  
      ho, browser sirf ek space dikhayega.</p>  
<p>Ye dusra paragraph hai, apne aap new line se start hoga.</p>
```

Output / Result: Do alag paragraphs dikhenge, unke darmiyan automatic spacing hogi.

Real-World Example: Blog posts aur articles ka har paragraph `<p>` tag mein hota hai.

Line Break (`<br>`)

Br tag ek void element hai (closing tag nahi hota) jo line ko force karta hai next line par jane ke liye, bina naya paragraph banaye. Ye tab useful hota hai jab aapko address, poem, ya kisi aisi jagah line break chahiye jahan naya paragraph create karna semantically sahi na ho. Overuse se bachna chahiye - agar aap spacing ke liye br use kar rahe hain to CSS margin behtar option hai.

Code Example:

```
<p>Ye pehli line hai.<br>
Ye dusri line hai, same paragraph mein.<br>Ye
teesri line hai.</p>
```

Output / Result: Teeno lines ek hi paragraph mein hongii lekin har line naye row se start hogi.

Real-World Example: Postal address likhte waqt br tag use hota hai: 'House 123
Street 5
Karachi'.

Horizontal Rule (`<hr>`)

Hr tag ek horizontal line create karta hai jo content ko visually alag sections mein divide karti hai. Ye bhi void element hai. Semantically ye 'thematic break' ko represent karta hai, matlab jahan topic change ho raha ho wahan use hota hai.

Code Example:

```
<h2>Section 1</h2>
<p>Ye section 1 ka content hai.</p>
<hr>
<h2>Section 2</h2>
<p>Ye section 2 ka content hai.</p>
```

Output / Result: Do sections ke darmiyan ek horizontal line dikhegi.

Real-World Example: Blog posts mein ek topic khatam hone aur naya start hone par hr line dikhai jaati hai.

Comments

HTML comments code mein notes likhne ke liye use hote hain jo browser mein display nahi hote, sirf source code mein dikhte hain. Ye developers ko code samajhne, kisi part ko temporarily disable karne, ya team members ke liye instructions chorne mein madad karte hain. Syntax hota hai: `<!-- comment yahan -->`.

Code Example:

```
<!-- Ye header section hai -->
<h1>Website Title</h1>

<!-- <p>Ye line temporarily hidden hai</p> -->
<p>Ye line visible hai</p>
```

Output / Result: Sirf 'Ye line visible hai' wala paragraph show hoga, comments aur commented-out code invisible rahega.

Real-World Example: Bare projects mein developers comments se batate hain ke konsa section kya kaam karta hai, taake baad mein ya doosre developer ko samajhna aasan ho.

Text Formatting Tag `<b>`

Bold text banata hai bina kisi extra semantic meaning ke - sirf visual bold styling ke liye. Jab text ko important nahi batana lekin sirf bold dikhana ho (jaise keywords highlight karna) to `<b>` use hota hai.

Code Example:

```
<p>Ye <b>bold text</b> hai bina extra importance ke.</p>
```

Output / Result: 'bold text' words bold dikhenge.

Text Formatting Tag `<strong>`

Bhi text ko bold dikhata hai lekin semantic meaning ke sath - ye batata hai ke ye text 'important' hai. Screen readers is text ko extra emphasis ke sath parhte hain. SEO aur accessibility ke liye `<strong>` ko `<b>` se better maana jata hai jab importance denote karni ho.

Code Example:

```
<p><strong>Warning:</strong> Is button ko dobara mat dabana.</p>
```

Output / Result: 'Warning:' bold aur emphasized dikhega, screen reader isay important bata kar parhega.

Text Formatting Tag `<i>`

Text ko italic (tirchi) shakal mein dikhata hai bina kisi extra meaning ke - jaise technical terms ya foreign words ko visually alag dikhane ke liye.

Code Example:

```
<p>Scientific name hai <i>Panthera leo</i> (sher).</p>
```

Output / Result: 'Panthera leo' italic dikhega.

Text Formatting Tag `<em>`

Text ko emphasize karta hai (italic dikhta hai) semantic meaning ke sath - ye batata hai ke jab bola jaye to is word par stress diya jaye. Screen readers ismein tone change karke parhte hain.

Code Example:

```
<p>Aapko <em>abhi</em> ye kaam karna hoga.</p>
```

Output / Result: 'abhi' italic dikhega aur voice mein emphasis milega.

Text Formatting Tag `<u>`

Text ke neeche underline (seedhi line) laga deta hai. Careful use karein kyunke underline usually links ke liye reserved hota hai, isse confusion ho sakta hai.

Code Example:

```
<p>Ye <u>underlined</u> word hai.</p>
```

Output / Result: 'underlined' word ke neeche line dikhegi.

Text Formatting Tag `<mark>`

Text ko highlight karta hai (default yellow background) jaise kisi ne marker se text highlight kiya ho. Search results mein matching keywords highlight karne ke liye use hota hai.

Code Example:

```
<p>Search result mein <mark>HTML</mark> keyword highlight hua hai.</p>
```

Output / Result: 'HTML' word par yellow background dikhega.

Text Formatting Tag <small>

Text ko chota size mein dikhata hai, aksar fine print, disclaimers, ya copyright notices ke liye use hota hai.

Code Example:

```
<p>Regular price. <small>Terms and conditions apply.</small></p>
```

Output / Result: 'Terms and conditions apply' chote font mein dikhega.

**Text Formatting Tag **

Text ko strikethrough (beech mein line) ke sath dikhata hai, jo batata hai ke ye content delete/remove kar diya gaya hai. Aksar price changes dikhane ke liye use hota hai.

Code Example:

```
<p>Price: <del>Rs 2000</del> <ins>Rs 1500</ins></p>
```

Output / Result: 'Rs 2000' par cut line hogi aur 'Rs 1500' underlined dikhega.

Text Formatting Tag <ins>

Text ko underline ke sath dikhata hai, jo batata hai ke ye naya content insert kiya gaya hai. Aksar ke sath milkar use hota hai jaise price update dikhane mein.

Code Example:

```
<p>Old text <del>removed</del> and <ins>new text added</ins>.</p>
```

Output / Result: 'removed' strikethrough aur 'new text added' underlined dikhega.

Text Formatting Tag <sub>

Text ko subscript mein dikhata hai (thoda neeche aur chota), jaise chemical formulas mein.

Code Example:

```
<p>Pani ka formula hai H<sub>2</sub>O.</p>
```

Output / Result: '2' number chota aur neeche dikhega, H₂O ki tarah nahi balke chemistry notation ki tarah.

Text Formatting Tag <sup>

Text ko superscript mein dikhata hai (thoda upar aur chota), jaise mathematical powers ya footnotes mein.

Code Example:

```
<p>Einstein ka formula hai E = mc<sup>2</sup>.</p>
```

Output / Result: '2' number chota aur upar dikhega.

Module 3: Links

Anchor Tag (<a>)

Anchor tag hyperlinks banane ke liye use hota hai jo user ko ek page se dusre page, ya same page ke andar kisi section par le jate hain. Iska main attribute 'href' hota hai jo destination batata hai. Anchor tag ke andar text, image, ya koi bhi HTML element ho sakta hai jo clickable ban jata hai.

Code Example:

```
<a href="https://google.com">Google Par Jayein</a>
```

Output / Result: 'Google Par Jayein' text underline aur blue color mein clickable link ke tor par dikhega.

Relative Links

Relative links same website ke andar dusre pages ko link karne ke liye use hote hain, full URL likhne ki zaroorat nahi hoti - sirf file ka path likhte hain relative to current file location. Ye local development aur same-domain navigation ke liye best practice hai.

Code Example:

```
<a href="about.html">About Page</a>
<a href="pages/contact.html">Contact Page</a>
<a href="../index.html">Home Page</a>
```

Output / Result: Teeno links same website ke andar different pages par navigate karenge.

Absolute Links

Absolute links poora URL specify karte hain including protocol (http/https) aur domain name. Ye dusri websites ko link karne ke liye use hote hain kyunke woh browser ko exact location batate hain, chahe aap kahin bhi ho.

Code Example:

```
<a href="https://www.youtube.com/watch?v=xyz">YouTube Video Dekhein</a>
```

Output / Result: Click karne par user directly YouTube ke us specific video par chala jayega.

Open Link in New Tab

target="_blank" attribute link ko current tab band kiye bina naye browser tab mein khol deta hai. Ye tab useful hota hai jab aap chahte hain user aapki website chorein bina, external site bhi dekh len. Security ke liye rel="noopener noreferrer" bhi add karna chahiye.

Code Example:

```
<a href="https://facebook.com" target="_blank" rel="noopener noreferrer">Facebook (New Tab)</a>
```

Output / Result: Link click karne par naya browser tab khulega, purana tab wahin rahega.

Email Links

mailto: protocol use karke aap aisa link bana sakte hain jo click karne par user ke default email app ko khol de, with prefilled email address (aur optionally subject/body).

Code Example:

```
<a href="mailto:info@example.com?subject=Hello&body=Aapka message yahan">Email Bhejein</a>
```

Output / Result: Click karne par default email client khulega with 'info@example.com' as recipient.

Phone Links tel: protocol phone number ko clickable banata hai, mobile devices par click karne se directly call start ho jati hai.

Code Example:

```
<a href="tel:+923001234567">Call Karein: +92 300 1234567</a>
```

Output / Result: Mobile par click karne se phone dialer khul jayega us number ke sath.

Download Links

download attribute browser ko batata hai ke link ko navigate karne ke bajaye us file ko download kiya jaye. Aap download attribute mein custom filename bhi de sakte hain.

Code Example:

```
<a href="files/resume.pdf" download="MyResume.pdf">Resume Download Karein</a>
```

Output / Result: Click karne par PDF file 'MyResume.pdf' naam se download ho jayegi, browser mein khulegi nahi.

Bookmark Links (Same-page anchors)

id attribute kisi element ko diya jata hai, phir href="#id" use karke us specific section tak seedha jump kiya ja sakta hai - ye lambe pages mein navigation ke liye bohot useful hai jaise table of contents.

Code Example:

```
<a href="#section2">Section 2 Par Jayein</a>

<!-- ...bohot sara content... -->

<h2 id="section2">Section 2</h2>
<p>Ye section 2 hai.</p>
```

Output / Result: Click karne par page automatically scroll ho kar 'Section 2' heading tak pahunch jayega.

Module 4: Images

Image Tag ()

Code Example:

Img tag images ko webpage mein embed karne ke liye use hota hai. Ye void element hai. Iska sabse important attribute 'src' hai jo image ki location batata hai (local file ya URL).

```

```

Output / Result: Specified image browser mein display hogi apne original size mein (jab tak width/height set na ho). **Image Attributes**

Common image attributes hain: src (source path), alt (alternative text jo image load na ho to dikhta hai, accessibility ke liye zaroori), width aur height (dimensions in pixels), title (hover par tooltip).

Code Example:

```

```

Output / Result: Image 300x200 pixels size mein dikhegi, hover karne par 'My Cat' tooltip aayega.

Image Formats

Web ke liye common image formats hain: JPG/JPEG (photos ke liye, smaller size lekin quality loss), PNG (transparency support karta hai, logos ke liye best), GIF (simple animations ke liye), WEBP (modern format, best compression aur quality), SVG (vector graphics, icons aur logos ke liye, kabhi blurry nahi hota).

Code Example:

```
  
  

```

Output / Result: Har format apni jagah use hota hai based on requirement - photo ke liye jpg, logo ke liye png/svg.

Responsive Images

Responsive images alag-alag screen sizes par sahi size ki image load karte hain, taake mobile par bari image download hokar bandwidth waste na ho. srcset attribute se multiple image sizes provide kar sakte hain, browser khud sahi size choose karega.

Code Example:

```

```

Output / Result: Mobile par chota image load hoga, desktop par bara - performance behtar hogi.

Figure & Figcaption

Figure tag image (ya diagram, code snippet) ko uske caption ke sath semantically group karta hai. Figcaption uske andar caption text ke liye use hota hai. Ye SEO aur accessibility ke liye better structure deta hai simple img+p combination se.

Code Example:

```
<figure>
```

Code Example:

```

<figcaption>K2 - Pakistan ki sabse ooncha pahaar</figcaption> </figure>
```

Output / Result: Image ke neeche uska caption dikhega, dono semantically connected honge.

Image Maps

Image map ek image ke different clickable areas define karne deta hai jahan har area alag link par le ja sakta hai. usemap attribute image ko map se link karta hai, aur <map> ke andar <area> tags clickable regions define karte hain.

```

<map name="worldmap">
  <area shape="rect" coords="0,0,100,100" href="asia.html" alt="Asia">
  <area shape="circle" coords="200,200,50" href="europe.html" alt="Europe"> </map>
```

Output / Result: Image ke different portions par click karne se alag pages khulenge.

Module 5: Lists

Ordered List ()

Ordered list numbered items dikhata hai (1, 2, 3...). Har item tag mein hota hai. type attribute se numbering style (numbers, letters, roman) change kar sakte hain, aur 'start' attribute se shuru hone wala number set kar sakte hain.

Code Example:

```
<ol>
  <li>Pehla step</li>
  <li>Dusra step</li>
  <li>Teesra step</li>
</ol>
```

Output / Result: 1. Pehla step 2. Dusra step 3. Teesra step - is tarah numbered list dikhegi.

Unordered List ()

Unordered list bullet points ke sath items dikhata hai, jab items ki koi specific order/sequence important na ho.

Code Example:

```
<ul>
  <li>Doodh</li>
  <li>Anday</li>
  <li>Roti</li>
</ul>
```

Output / Result: Bullet points ke sath teen items dikhenge (grocery list style).

Description List (<dl>)

Description list terms aur unki definitions dikhane ke liye use hoti hai. dt (description term) aur dd (description details) tags use hote hain.

Code Example:

Code Example:

```
<dl>
  <dt>HTML</dt>
  <dd>Web pages ka structure banane wali markup language</dd>
  <dt>CSS</dt>
  <dd>Web pages ko style karne wali language</dd> </dl>
```

Output / Result: Har term bold dikhega aur uski definition indent hoke neeche dikhegi - glossary jaisa format.

Nested Lists

Ek list ke andar dusri list ko nest karna, jaise sub-categories ya multi-level menus dikhane ke liye. Koi bhi list (ol ya ul) kisi bhi list item (li) ke andar rakhi ja sakti hai.

Code Example:

```
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Banana</li>
    </ul>
  </li>
  <li>Vegetables
    <ul>
      <li>Carrot</li>
      <li>Potato</li>
    </ul>
  </li> </ul>
```

Output / Result: Main categories dikhengi (Fruits, Vegetables) aur unke andar indented sub-items honge.

Module 6: Tables

Table Structure

HTML table rows aur columns mein data organize karta hai. Main tag <table> hai jiske andar rows (<tr>) hoti hain aur har row ke andar cells (<td> ya <th>) hoti hain.

Code Example:

```
<table>
  <tr>
    <td>Cell 1</td>
    <td>Cell 2</td>
  </tr>
</table>
```

Output / Result: Ek simple 1 row, 2 column table dikhega, without borders (jab tak CSS na lagayen).

Rows & Columns

Table Row (<tr>) ek horizontal row banata hai. Har row ke andar jitne bhi <td> (Table Data) tags honge utne columns ban jayenge. Rows ki tarteeb hi table ka structure decide karti hai.

Code Example:

```
<table border="1">
  <tr><td>Ali</td><td>25</td></tr>
  <tr><td>Sara</td><td>22</td></tr>
</table>
```

Output / Result: 2 rows aur 2 columns ka table dikhega, names aur ages ke sath.

Table Header (<th>)

Th tag table headings ke liye use hota hai, jo default mein bold aur center-aligned dikhta hai. Ye screen readers ko batata hai ke ye column/row ka header hai, jo table ko accessible aur organized banata hai.

Code Example:

```
<table border="1">
  <tr>
    <th>Naam</th>
    <th>Umar</th>
  </tr>
  <tr>
    <td>Ahmed</td>
    <td>30</td>
  </tr>
</table>
```

Output / Result: 'Naam' aur 'Umar' bold headings ke tor par dikhenge, neeche actual data hoga.

Caption

Caption tag table ka title/description deta hai, table tag ke fauran baad likha jata hai aur default mein table ke upar center mein dikhta hai.

Code Example:

```
<table border="1">
  <caption>Student Records</caption>
  <tr><th>Naam</th><th>Grade</th></tr>
  <tr><td>Bilal</td><td>A</td></tr>
</table>
```

Output / Result: 'Student Records' title table ke upar dikhega.

colspan colspan attribute ek cell ko multiple columns tak fail-a deta hai (horizontally merge karta hai).

Code Example:

```
<table border="1">
  <tr>
    <th colspan="2">Full Name</th>
  </tr>
  <tr>
    <td>Ahmed</td>
    <td>Khan</td>
  </tr>
</table>
```

Output / Result: "full name" heading 2 ki width rowspan tak spread hoga.

Rowspan

Rowspan attribute ek cell ko multiple rows tak fail-a deta hai (vertically merge karta hai).

Code Example:

```
<table border="1">
```

```

<tr>
  <td rowspan="2">Karachi</td>
  <td>Area 1</td>
</tr>
<tr>
  <td>Area 2</td>
</tr>
</table>

```

Output / Result: 'Karachi' cell 2 rows tak vertically merge hoga.

Table Styling Basics

CSS ke through tables ko attractive banaya jata hai: border property lines add karti hai, border-collapse: collapse se double borders single ban jate hain, padding cells mein space deta hai, aur background-color se rows ko alternate colors diye ja sakte hain.

Code Example:

```

<style>  table { border-collapse: collapse; width:
100%; }  th, td { border: 1px solid #333; padding:
8px; }  tr:nth-child(even) { background-color:
#f2f2f2; }
</style>
<table>
  <tr><th>Naam</th><th>Shehar</th></tr>
  <tr><td>Ali</td><td>Lahore</td></tr>
  <tr><td>Zara</td><td>Karachi</td></tr> </table>

```

Output / Result: Table clean borders ke sath dikhega aur alternate rows ka background halka gray hoga (zebra striping).

Module 7: Forms

Form Tag

Form tag user input collect karne ke liye container hota hai. action attribute batata hai data kahan submit hoga, method batata hai kaise (GET ya POST) data bheja jayega.

Code Example:

```

<form action="/submit" method="POST">
  <!-- form fields yahan honge -->
</form>

```

Output / Result: Ye ek invisible container hai jo apne andar ke inputs ko group karta hai.

Input Types

Input tag sabse versatile form element hai, uska 'type' attribute decide karta hai kaisa input field dikhega: text, email, password, number, date, checkbox, radio, file, submit, waghera.

Code Example:

```

<input type="text" placeholder="Naam">
<input type="email" placeholder="Email">
<input type="password" placeholder="Password">
<input type="number" placeholder="Umar">
<input type="date">
<input type="checkbox"> Mujhe yaad rakhein

```

```
<input type="radio" name="gender"> Male  
<input type="radio" name="gender"> Female  
<input type="file">  
<input type="submit" value="Submit Karein">
```

Output / Result: Har input apni type ke mutabiq alag UI element dikhayega - text box, checkbox, radio button, date picker waghera.

Labels

Label tag input field ko naam/description deta hai, 'for' attribute se label us input ke id se link hoti hai. Isse accessibility behtar hoti hai aur label par click karne se bhi input select ho jata hai.

Code Example:

```
<label for="username">Username:</label>  
<input type="text" id="username" name="username">
```

Output / Result: 'Username:' text dikhega input field ke sath connected, label click karne se input focus ho jayega.

Placeholder

Placeholder attribute input field ke andar halka gray hint text dikhata hai jo batata hai kya type karna hai. Ye actual value nahi hai, jaise hi user type karna shuru kare ye ghayab ho jata hai.

Code Example:

```
<input type="text" placeholder="Apna poora naam likhein">
```

Output / Result: Empty field mein halke gray color mein 'Apna poora naam likhein' dikhega.

Required

Required attribute field ko mandatory banata hai - form submit hone se pehle browser automatically check karega aur agar field khali ho to error message dikhayega.

Code Example:

```
<input type="email" required>
```

Output / Result: Agar user email field khali chore aur submit kare, browser 'Please fill this field' jaisa error dikhayega.

Readonly

Readonly attribute field ko dikhata hai aur value select/copy karne deta hai lekin user usay edit nahi kar sakta. Form submit hone par uski value bhi jaati hai.

Code Example:

```
<input type="text" value="Ye edit nahi ho sakta" readonly>
```

Output / Result: Field mein value dikhegi lekin user usay change nahi kar payega.

Disabled

Disabled attribute field ko completely non-interactive bana deta hai - halka gray dikhta hai, click ya select nahi ho sakta, aur form submit hone par uski value bhi nahi jaati.

Code Example:

```
<input type="text" value="Disabled field" disabled>
```

Output / Result: Field grayed out dikhega aur completely unusable hoga.

Select & Option

Select tag dropdown menu banata hai, uske andar multiple option tags hote hain jo choices represent karte hain.

Code Example:

```
<select name="city">
  <option value="karachi">Karachi</option>
  <option value="lahore">Lahore</option>
  <option value="islamabad">Islamabad</option> </select>
```

Output / Result: Ek dropdown menu dikhega jismein click karne par teen cities show hongy.

Textarea

Textarea multi-line text input ke liye use hota hai, jaise messages ya comments. rows aur cols attributes se size set kar sakte hain.

Code Example:

```
<textarea rows="4" cols="40" placeholder="Apna message likhein"></textarea>
```

Output / Result: Ek resizable multi-line text box dikhega.

Button

Button tag clickable button banata hai. type="submit" form submit karta hai, type="reset" form clear karta hai, type="button" custom JavaScript action ke liye use hota hai.

Code Example:

```
<button type="submit">Submit Karein</button>
<button type="reset">Reset Karein</button>
```

Output / Result: Do buttons dikhenge - ek form submit karega, dusra saare fields clear kar dega.

Fieldset

Fieldset related form fields ko ek box mein group karta hai, visually border ke sath.

Code Example:

```
<fieldset>
  <legend>Personal Info</legend>
  <input type="text" placeholder="Naam">
  <input type="email" placeholder="Email">
</fieldset>
```

Output / Result: Fields ek border wale box mein group hoke dikhenge.

Legend

Legend fieldset ka title/caption hota hai, hamesha fieldset ke andar sabse pehle likha jata hai.

Code Example:

```
<fieldset>
  <legend>Account Details</legend>
  <input type="text" placeholder="Username"> </fieldset>
```

Output / Result: 'Account Details' title fieldset ke border ke upar dikhega.

Datalist

Datalist input field ke liye predefined suggestions provide karta hai - user type kare to autocomplete jaisi list dikhti hai, lekin custom value bhi likh sakta hai.

Code Example:

```
<input list="browsers" name="browser">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari"> </datalist>
```

Output / Result: Type karte hi suggestions dropdown mein dikhengi jinme se user choose kar sakta hai.

Output

Output tag calculation ka result dikhane ke liye use hota hai, aksar JavaScript ke sath milkar (jaise slider value ya form calculation ka result).

Code Example:

```
<form oninput="result.value = parseInt(a.value) + parseInt(b.value)">
  <input type="number" id="a" value="0"> +
  <input type="number" id="b" value="0"> =
  <output name="result" for="a b">0</output> </form>
```

Output / Result: Jab bhi a ya b field change hoga, output automatically unka sum dikhayega.

Module 8: Semantic HTML

Header

Header tag page ya section ke introductory content ko represent karta hai - usually logo, navigation, aur main heading yahan hoti hai. Ek page mein multiple headers ho sakte hain (jaise ek article ka apna header).

Code Example:

```
<header>
  <h1>My Website</h1>
  <p>Best blog for developers</p>
</header>
```

Output / Result: Page ke top par title aur tagline dikhega, semantically ye 'header' section hai.

Nav

Nav tag navigation links ko group karta hai - jaise main menu ya sidebar links. Screen readers aur search engines isay navigation area ke tor par pehchante hain.

Code Example:

```
<nav>
  <a href="/">Home</a>
  <a href="/about">About</a>
  <a href="/contact">Contact</a>
</nav>
```

Output / Result: Navigation links horizontally ya vertically group hoke dikhenge.

Main

Main tag page ke primary/unique content ko represent karta hai - header, footer, sidebar ke ilawa jo asal content hai. Ek page mein sirf ek main tag hona chahiye.

Code Example:

```
<main>
  <h2>Blog Post Title</h2>
  <p>Ye article ka main content hai...</p> </main>
```

Output / Result: Ye page ka central, main content area hoga.

Section

Section tag page ko thematic groups mein divide karta hai, jaise 'Services', 'About Us', 'Testimonials' - har ek apna heading rakhta hai aur ek related content ka group hota hai.

Code Example:

```
<section>
  <h2>Our Services</h2>
  <p>Hum web development provide karte hain.</p> </section>
```

Output / Result: Ye ek distinct content section hoga apne heading ke sath.

Article

Article tag independent, self-contained content ke liye use hota hai jo apne aap mein complete ho aur standalone distribute kiya ja sake - jaise blog post, news article, ya forum comment.

Code Example:

```
<article>
  <h2>Breaking News</h2>
  <p>Aaj ek important event hua...</p>
</article>
```

Output / Result: Ye ek complete news story ki tarah dikhega jo apne aap mein independent hai.

Aside

Aside tag main content se related lekin separate content ke liye use hota hai, jaise sidebar, related links, ya advertisements.

Code Example:

```
<aside>
  <h3>Related Posts</h3>
  <ul><li>Post 1</li><li>Post 2</li></ul> </aside>
```

Output / Result: Ye ek sidebar-jaisa section dikhega jo main content ke sath complementary info deta hai.

Footer

Footer tag page ya section ke ending content ko represent karta hai - usually copyright info, contact details, ya social links yahan hoti hain.

Code Example:

```
<footer>
  <p>&copy; 2026 My Website. All rights reserved.</p> </footer>
```

Output / Result: Page ke bilkul neeche copyright text dikhega.

Figure

Figure tag self-contained content (images, diagrams, code) ko uske caption ke sath group karta hai, jo main flow se reference kiya jata hai lekin usse move bhi kiya ja sakta hai bina meaning kharab kiye.

Code Example:

```
<figure>
  
  <figcaption>2026 ka sales data</figcaption> </figure>
```

Output / Result: Image apni caption ke sath ek unit ke tor par dikhegi.

Details & Summary

Details tag ek collapsible/expandable section banata hai. Summary uska visible heading hota hai jis par click karne se content show/hide hota hai - bina kisi JavaScript ke!

Code Example:

```
<details>
  <summary>FAQ: HTML kya hai?</summary>
  <p>HTML ek markup language hai jo webpages banane ke liye use hoti hai.</p> </details>
```

Output / Result: Sirf 'FAQ: HTML kya hai?' dikhega, click karne par neech ka answer expand ho jayega.

Time

Time tag dates aur times ko machine-readable format mein mark karta hai, jo browsers aur search engines ko exact date samajhne mein madad karta hai.

Code Example:

```
<p>Ye article <time datetime="2026-07-10">10 July 2026</time> ko publish hua.</p>
```

Output / Result: '10 July 2026' normal text ki tarah dikhega lekin machine ke liye structured date data ke sath.

Module 9: Multimedia

Audio

Audio tag bina kisi external plugin (jaise Flash) ke audio files play karne deta hai. controls attribute play/pause/volume buttons dikhata hai.

Code Example:

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  Aapka browser audio support nahi karta. </audio>
```

Output / Result: Ek audio player controls (play, pause, volume) ke sath dikhega.

Video

Video tag videos embed karta hai. controls attribute playback buttons deta hai, width/height size set karte hain, aur poster attribute se thumbnail image set hoti hai video load hone se pehle.

Code Example:

```
<video controls width="400" poster="thumbnail.jpg">
<source src="movie.mp4" type="video/mp4"> Aapka
browser video support nahi karta. </video>
```

Output / Result: Ek video player thumbnail aur controls ke sath dikhega, 400px width mein.

Source

Source tag audio/video ke andar multiple file formats provide karne ke liye use hota hai, taake agar browser ek format support na kare to dusra try kare.

Code Example:

```
<video controls>
  <source src="movie.webm" type="video/webm">
  <source src="movie.mp4" type="video/mp4"> </video>
```

Output / Result: Browser pehle webm try karega, agar support nahi to mp4 use karega.

Track

Track tag video ke liye subtitles ya captions add karta hai, accessibility ke liye zaroori hai deaf/hard-of-hearing users ke liye.

Code Example:

```
<video controls>
  <source src="movie.mp4" type="video/mp4">
  <track src="subtitles-ur.vtt" kind="subtitles" srclang="ur" label="Urdu"> </video>
```

Output / Result: Video mein subtitle option available hoga jo Urdu subtitles show karega.

iframe

iframe tag dusre webpage/content ko current page ke andar embed karta hai - jaise Google Maps, YouTube videos, ya external websites.

Code Example:

```
<iframe src="https://www.google.com/maps/embed?... " width="600" height="450"></iframe>
```

Output / Result: Ek embedded window dikhegi jismein external content (map) show hoga.

YouTube Embed

YouTube videos ko embed karne ke liye YouTube 'Share > Embed' se milne wala iframe code use hota hai.

Code Example:

```
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/dQw4w9WgXcQ"
  title="YouTube video" allowfullscreen> </iframe>
```

Output / Result: Ek YouTube video player directly aapke page mein embed ho jayega, play/pause aur fullscreen ke sath.

Module 10: HTML Entities

Symbols (Special Characters)

HTML mein kuch characters jaise < > & special meaning rakhte hain (tags ke liye), isliye unhe direct display karne ke liye 'entities' use hoti hain jo &code; format mein hoti hain.

Code Example:

```
<p>5 &lt; 10 aur 10 &gt; 5</p>
<p>Ye &amp; symbol hai</p>
```

Output / Result: Browser mein dikhega: '5 < 10 aur 10 > 5' aur 'Ye & symbol hai'.

Emojis

Emojis ko HTML mein direct type kar sakte hain (UTF-8 encoding ke sath) ya unicode entity code use kar sakte hain.

Code Example:

```
<p>Mujhe coding pasand hai &#128512;</p>
<p>Direct emoji: 🚀 ✔️ 📄</p>
```

Output / Result: Emoji symbols (smiley, rocket, checkmark) directly text mein dikhenge.

Character Entities

Common entities hain: © (©), ® (®), ™ (™), (non-breaking space), " ("), ' (').

Code Example:

```
<p>&copy; 2026 My Company&trade;</p>
<p>Price:&nbsp;Rs 500</p>
```

Output / Result: '© 2026 My Company™' dikhega, aur 'Price:' ke baad space rahega jo line-break nahi hoga.

Module 11: Meta Tags

charset

charset meta tag document ki character encoding define karta hai. UTF-8 sabse common hai kyunke ye duniya ki tamam languages ke characters support karta hai.

Code Example:

```
<meta charset="UTF-8">
```

Output / Result: Ye ensure karta hai ke Urdu, Arabic, Chinese jaisi languages ke characters bhi sahi dikhein.

viewport

Viewport meta tag mobile responsiveness ke liye essential hai - ye browser ko batata hai page ko device ki width ke mutabiq scale kare.

Code Example:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Output / Result: Bina is tag ke, mobile par website chota aur zoomed-out dikhegi. Is tag ke sath properly fit hogi.

description

Meta description search engine results mein page ke URL ke neeche dikhne wala short summary hota hai. Ye SEO ke liye important hai.

Code Example:

```
<meta name="description" content="Ye website frontend development seekhne ke liye best resource hai.">
```

Output / Result: Google search results mein ye description snippet ke tor par dikh sakta hai.

keywords

Meta keywords purane zamane mein SEO ke liye use hote thay, lekin modern search engines (Google) inhe ab ignore karte hain. Phir bhi kabhi kabhi use hota hai.

Code Example:

```
<meta name="keywords" content="html, css, javascript, frontend">
```

Output / Result: Ye keywords search engines ko batane ki koshish karte hain page kis baare mein hai (though modern SEO mein iska impact minimal hai).

author

Author meta tag page ke creator ka naam batata hai.

Code Example:

```
<meta name="author" content="Engr. Umar Naveed">
```

Output / Result: Ye information page source mein hoti hai, visible nahi hoti.

favicon

Favicon woh chota icon hota hai jo browser tab mein page title ke sath dikhta hai. link tag se add hota hai.

Code Example:

```
<link rel="icon" type="image/png" href="favicon.png">
```

Output / Result: Browser tab mein page ke title ke bagal mein chota icon show hoga.

Module 12: SEO Basics

Title Tag

Title tag search engine results ka clickable headline hota hai aur browser tab mein bhi dikhta hai. SEO ke liye sabse important elements mein se ek hai, ideally 50-60 characters ka hona chahiye.

Code Example:

```
<title>Best Frontend Development Course - Learn HTML CSS JS</title>
```

Output / Result: Ye text Google search results mein bara clickable heading ki tarah aur browser tab mein dikhega.

Meta Description

Achi meta description search results mein click-through-rate barhati hai. Ye compelling aur page content ka accurate summary honi chahiye, 150-160 characters ke around.

Code Example:

```
<meta name="description" content="Sikho HTML, CSS, aur JavaScript step by step, beginner se advanced tak, with real code examples.">
```

Output / Result: Search results mein page ke title ke neeche ye summary dikhegi.

Heading Structure

Sahi heading hierarchy (h1 > h2 > h3...) search engines ko page ka structure samajhne mein madad karti hai. Sirf ek h1 use karein, aur levels skip na karein (h1 se seedha h3 par mat jayen).

Code Example:

```
<h1>Frontend Development Guide</h1>
<h2>HTML Basics</h2>
<h3>What is a Tag?</h3> <h2>CSS Basics</h2>
```

Output / Result: Proper hierarchy Google ko page ka outline samajhne mein madad karti hai, jo ranking improve kar sakti hai.

Semantic HTML for SEO

Semantic tags (article, section, nav, header) search engines ko content ka context samajhne mein madad karte hain, generic divs ki nisbat better SEO deta hai.

Code Example:

```
<article>
  <h2>How to Learn HTML</h2>
  <p>Content yahan...</p> </article>
```

Output / Result: Search engine is content ko ek complete, independent article ke tor par samjhenga.

Image Alt Attribute for SEO

Alt text search engines ko batata hai image mein kya hai, kyunke search engines images ko directly 'dekh' nahi sakte. Ye Google Image Search mein ranking ke liye zaroori hai.

Code Example:

```

```

Output / Result: Ye description image search results mein aur accessibility ke liye use hoti hai.

Module 13: Accessibility

alt Attribute

Alt attribute screen readers ke liye image ka description parhta hai taake visually-impaired users bhi samajh sakein image mein kya hai. Decorative images ke liye alt="" khali chorna chahiye.

Code Example:

```

```

Output / Result: Screen reader is text ko bolega jab user is image tak pahunchega.

aria-label aria-label kisi element ko accessible naam deta hai jab visible text kaafi na ho, jaise icon buttons.

Code Example:

```
<button aria-label="Search karein">
  
</button>
```

Output / Result: Screen reader button ko 'Search karein' bolega, chahe visually sirf icon dikhe.

aria-hidden

aria-hidden="true" kisi element ko screen readers se chupa deta hai, jabke woh visually sabko dikhta rahega. Purely decorative elements (jaise decorative icons) ke liye use hota hai.

Code Example:

```
<span aria-hidden="true">★</span> Rating: 5/5
```

Output / Result: Star symbol visually dikhega lekin screen reader isay skip kar dega, sirf 'Rating: 5/5' boleگا.

tabindex

tabindex control karta hai keyboard 'Tab' key se elements kis order mein focus hon. 0 = normal order mein shamil, -1 = tab se focus na ho lekin JS se focus kiya ja sake, positive number = custom order.

Code Example:

```
<input type="text" tabindex="1">  
<input type="text" tabindex="2">  
<button tabindex="-1">Skip in Tab order</button>
```

Output / Result: Keyboard se Tab dabane par pehle field 1, phir field 2 focus hoga, button skip ho jayega.

Screen Readers

Screen readers (jaise NVDA, JAWS, VoiceOver) visually-impaired users ke liye page content ko awaaz mein parhte hain. Semantic HTML, alt text, aur ARIA attributes use karke hum apni website inke liye accessible bana sakte hain.

Code Example:

```
<nav aria-label="Main Navigation">  
  <a href="/">Home</a>  
  <a href="/about">About</a> </nav>
```

Output / Result: Screen reader 'Main Navigation' region announce karega phir uske links parhega, jo user experience behtar banata hai.

Module 14: Best Practices

Clean Code

Clean code readable aur maintainable hota hai - meaningful names use karein (id="main-nav" na ke id="div1"), consistent naming convention follow karein, aur unnecessary nested elements avoid karein.

Code Example:

```
<!-- Bad -->  
<div class="d1"><div class="d2">Content</div></div>  
  
<!-- Good -->  
<section class="article-content">Content</section>
```

Output / Result: Clean code samajhna aasan hota hai aur team collaboration mein madadgar hota hai.

Proper Indentation

Consistent indentation (usually 2 ya 4 spaces) nested elements ki hierarchy ko visually clear karta hai, code parhne mein bohot aasani hoti hai.

Code Example:

```
<div class="card">
  <h2>Title</h2>
  <p>Description text yahan hoga.</p> </div>
```

Output / Result: Properly indented code turant samajh aata hai ke kaunsa element kiske andar hai.

File Naming

File names lowercase, hyphens ke sath (spaces nahi), aur descriptive hone chahiye jaise 'about-us.html' na ke 'AboutUs.HTML' ya 'page2.html'. Ye cross-platform compatibility aur URLs ke liye zaroori hai.

Code Example:

```
index.html about-us.html
contact-page.html
styles/main.css
```

Output / Result: Consistent naming se project professional aur organized dikhta hai.

Folder Structure

Ek organized project structure development ko asaan banata hai: root mein index.html, phir alag folders images, css, js, aur pages ke liye.

Code Example:

```
project/
index.html
/css
style.css  /js
script.js
/images
logo.png
/pages
about.html
contact.html
```

Output / Result: Ye structure large projects mein files dhoondna aasan banata hai.

Validation

HTML validation W3C Validator (validator.w3.org) se check karte hain ke code mein errors ya warnings to nahi. Valid HTML cross-browser consistency aur behtar SEO deta hai.

Code Example:

```
<!-- Common mistakes jo validator catch karta hai -->
 <!-- alt missing -->
<p>Unclosed paragraph
```

Output / Result: Validator errors list karega jaise missing alt attributes ya unclosed tags, jinhe fix karna chahiye.

Module 15: HTML Projects - Personal Portfolio

Project:

Ab tak humne HTML ke saare individual topics seekhe. Is module mein hum unhe combine karke ek real project banayenge - ek complete 'Personal Portfolio + Registration Form' page - taake aap

dekh saken real project mein ye sab tags kaise mil kar kaam karte hain. Iske baad baaki project ideas (Resume, Login Form, Restaurant Website, School Website, Blog Layout) bhi list ki gayi hain jo aap khud practice ke tor par try kar sakte hain isi pattern ko follow karke.

Project Steps:

- Step 1: Sabse pehle boilerplate likhein (DOCTYPE, html, head with meta tags aur title).
- Step 2: header section banayein jisme naam, tagline, aur nav links hon.
- Step 3: main tag ke andar 'about' section banayein with figure aur description.
- Step 4: 'skills' section mein unordered list se skills list karein.
- Step 5: 'projects' section mein table se projects list karein with links.
- Step 6: 'contact' section mein form banayein with proper labels, inputs, aur submit button.
- Step 7: footer mein copyright add karein.
- Step 8: Browser mein khol kar check karein har section sahi dikh raha hai ya nahi (validation bhi karein).

Complete Project Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Ali Raza - Frontend Developer ka personal portfolio page">
</head>
<title>Ali Raza | Frontend Developer Portfolio</title>
<body>

  <header>
    <h1>Ali Raza</h1>
    <p>Frontend Web Developer</p>
    <nav aria-label="Main Navigation">
      <a href="#about">About</a>
      <a href="#skills">Skills</a>
      <a href="#projects">Projects</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>

  <main>
    <section id="about">
      <h2>Mere Baare Mein</h2>
      <figure>
        
        <figcaption>Ali Raza - Karachi, Pakistan</figcaption>
      </figure>
      <p>Mein ek <strong>Frontend Developer</strong> hun jo <em>HTML, CSS aur JavaScript</em> mein
expertise rakhta hun. Mujhe khoobsurat aur responsive websites banana pasand hai.</p>
    </section>
```

```

<hr>

<section id="skills">
  <h2>Meri Skills</h2>
  <ul>
    <li>HTML5 & Semantic Markup</li>
    <li>CSS3 & Responsive Design</li>
    <li>JavaScript (ES6+)</li>
    <li>Git & GitHub</li>
  </ul>
</section>

<hr>

<section id="projects">
  <h2>Mere Projects</h2>
  <table border="1">
    <caption>Recent Work</caption>
    <tr>
      <th>Project</th>
      <th>Technology</th>
      <th>Link</th>
    </tr>
    <tr>
      <td>Restaurant Website</td>
      <td>HTML, CSS</td>
      <td><a href="#" target="_blank" rel="noopener">Dekhein</a></td>
    </tr>
    <tr>
      <td>Todo App</td>
      <td>HTML, CSS, JS</td>
      <td><a href="#" target="_blank" rel="noopener">Dekhein</a></td>
    </tr>
  </table>
</section>

<hr>

<section id="contact">
  <h2>Mujhse Rabta Karein</h2>
  <form action="/submit-contact" method="POST">
    <fieldset>
      <legend>Contact Form</legend>

      <label for="name">Naam:</label>
      <input type="text" id="name" name="name" placeholder="Apna naam likhein" required><br><br>

      <label for="email">Email:</label>
      <input type="email" id="email" name="email" placeholder="you@example.com" required><br><br>

      <label for="subject">Subject:</label>
      <select id="subject" name="subject">
        <option value="job">Job Offer</option>
        <option value="project">Project Inquiry</option>
        <option value="other">Other</option>
      </select><br><br>

      <label for="message">Message:</label><br>
      <textarea id="message" name="message" rows="4" cols="40"
placeholder="Apna message likhein"></textarea><br><br>

      <button type="submit">Message Bhejein</button>
    </fieldset>
  </form>

```

```
        <button type="reset">Clear Karein</button>  
      </fieldset>  
</form>
```

```
ya      <p>Ya seedha <a href="mailto:ali@example.com">Email</a>
      <a href="tel:+923001234567">Call</a> karein.</p>

</section>
</main>

    <footer>
      <p>&copy; 2026 Ali Raza. All rights reserved.</p>
    </footer>

</body>
</html>
```

Practice Karne Ke Liye Baaki Project Ideas:

- Resume Page: Apna resume HTML mein banayein using headings, lists (experience/education), aur tables (skills matrix).
- Registration Form: Mukammal signup form banayein with text, email, password, date-of-birth, gender radio buttons, aur terms checkbox.
- Login Form: Simple login form banayein with username/password fields aur 'Forgot Password' link.
- Restaurant Website: Multi-section page banayein - Menu (table ke sath), Gallery (images), aur Reservation form.
- School Website: Homepage banayein with nav, admission form, aur faculty details table.
- Blog Layout: Article list banayein using article tags, har article ka apna heading, image, aur 'Read More' link.

— End of Part 1: HTML —

Chapter # 2 CSS

Is section mein CSS ke tamam 21 modules deeply, beginner-level se explain kiye gaye hain — har topic ke sath code example, uska output, aur real-world use case diya gaya hai. Aakhir mein ek complete Responsive Landing Page project bhi banaya gaya hai jisme ye saare concepts combine ho kar use hote hain.

Module 1: Introduction

What is CSS?

CSS (Cascading Style Sheets) woh language hai jo HTML elements ko style aur design deti hai - colors, fonts, spacing, layout, animations waghera. Agar HTML skeleton hai to CSS uska makeup/clothes hai. 'Cascading' ka matlab hai styles upar se neeche 'cascade' (flow) hoti hain aur agar multiple rules ek element par apply hon to specific rules (priority ke mutabiq) jeet jati hain.

Code Example:

```
<style> h1 {  
  color: blue;  
  font-size: 32px;  
}  
</style>  
<h1>Styled Heading</h1>
```

Output / Result: Heading blue color mein aur 32px size mein dikhega.

Real-World Example: Har professional website (Instagram, Netflix) apna unique look CSS se hi banati hai.

Types of CSS

CSS teen tariqon se likhi ja sakti hai: Inline CSS (element ke andar style attribute mein, sirf ek element ke liye), Internal CSS (head mein <style> tag ke andar, ek page ke liye), aur External CSS (alag .css file mein, पूरी website ke liye - best practice kyunke reusable aur maintainable hoti hai).

Code Example:

```
<!-- Inline -->  
<p style="color: red;">Red text</p>  
  
<!-- Internal -->  
<style> p { color: red; } </style>  
  
<!-- External (style.css file link karke) -->  
<link rel="stylesheet" href="style.css">
```

Output / Result: Teeno methods same visual result de sakte hain, lekin External CSS large projects ke liye best hai.

Real-World Example: Professional projects mein hamesha External CSS use hoti hai taake ek hi file se पूरी site ka design control ho sake.

Selectors

Selectors batate hain CSS rule kis HTML element par apply hogi. Common selectors: element selector (p {}), class selector (.classname {}), id selector (#idname {}), universal selector (* {}), aur group selector (h1, h2 {}).

Code Example:

```
p { color: gray; }
.highlight { background: yellow; }
#main-title { font-size: 40px; } * { margin: 0; padding: 0; }
```

Output / Result: Har selector apne matching elements par style apply karega - class sirf usi class wale elements par, id sirf us specific element par.

Real-World Example: class ka use hota hai jab multiple elements ko same style deni ho, id sirf ek unique element ke liye.

Comments

CSS comments `/* */` ke darmiyan likhe jate hain, browser inhe ignore karta hai. Ye code explain karne ya temporarily disable karne ke liye use hote hain.

Code Example:

```
/* Ye main heading ka style hai */ h1
{
  color: navy;
  /* font-size: 50px; ye line disabled hai */
}
```

Output / Result: Sirf color apply hoga, commented-out font-size ignore ho jayegi.

Real-World Example: Team projects mein comments se pata chalta hai konsa CSS block kis purpose ke liye hai.

Module 2: Colors & Units

Colors (Named Colors)

CSS mein 140+ predefined named colors hain jaise red, blue, green, coral, tomato jo direct naam se use ho sakte hain. Beginners ke liye aasan hain lekin limited precision dete hain.

Code Example:

```
h1 { color: tomato; } div {
background-color: lightblue; }
```

Output / Result: Heading 'tomato' (orange-red) color mein aur div ka background halka blue hoga.

Real-World Example: Quick prototyping mein named colors fast hain, lekin production mein exact brand colors ke liye HEX/RGB behtar hai.

HEX Colors

HEX colors #RRGGBB format mein hote hain, har pair (Red, Green, Blue) 00 se FF tak hoti hai. Ye sabse commonly used format hai design tools (Figma, Photoshop) mein.

Code Example:

```
h1 { color: #FF5733; }
.box { background-color: #1A1A2E; }
```

Output / Result: Heading orange-red shade mein, box dark navy background mein dikhega.

Real-World Example: Designers Figma se HEX code copy karke seedha CSS mein paste karte hain.

RGB Colors

RGB(red, green, blue) format mein har value 0-255 ke darmiyan hoti hai, jo teenon colors ka mix batati hai.

Code Example:

```
h1 { color: rgb(255, 87, 51); }  
.box { background-color: rgb(26, 26, 46); }
```

Output / Result: Same colors jo HEX mein thay, lekin RGB numeric format mein.

Real-World Example: JavaScript se dynamically colors generate karte waqt RGB format zyada convenient hota hai.

RGBA Colors (with Transparency)

RGBA RGB ka extension hai jisme 4th value 'Alpha' (transparency) hoti hai, 0 (fully transparent) se 1 (fully opaque) tak.

Code Example:

```
.overlay { background-color: rgba(0,  
0, 0, 0.5);  
}
```

Output / Result: Ek 50% transparent black background dikhega, jiske peeche ka content halka dikhega.

Real-World Example: Modal popups aur image overlays banane ke liye RGBA transparency bohot use hoti hai.

HSL Colors

HSL (Hue, Saturation, Lightness) color ko human-friendly tarike se define karta hai: Hue (0-360 color wheel angle), Saturation (0-100% intensity), Lightness (0-100% dark/light).

Code Example:

```
h1 { color: hsl(9, 100%, 60%); }  
.box { background-color: hsl(240, 27%, 14%); }
```

Output / Result: Same orange-red heading, dark navy box - lekin HSL format se define kiya gaya.

Real-World Example: HSL se same color ki lighter/darker shades banana bohot aasan hota hai, sirf Lightness value change karke.

Units (px, %, rem, em, vw, vh)

CSS units do types ke hote hain: Absolute (px - fixed pixels) aur Relative (% - parent ke relative, em - parent font-size ke relative, rem - root/html font-size ke relative, vw/vh - viewport width/height ka 1%). Responsive design ke liye rem, %, vw, vh best hain kyunke ye screen size ke mutabiq scale hote hain.

Code Example:

```
.box { width: 50%; padding: 2rem; font-size: 1.2em;  
height: 100vh;  
}
```

Output / Result: Box apne parent ki 50% width lega, पूरी screen height (100vh) tak fail-a jayega.

Real-World Example: Responsive websites mein px ki jagah rem/% use hota hai taake mobile aur desktop dono par sahi scale ho.

Module 3: Backgrounds

background-color

Ye property element ka solid background color set karti hai, koi bhi color format (named, hex, rgb, hsl) use ho sakta hai.

Code Example:

```
.card { background-color:
#f0f0f0;
}
```

Output / Result: Card element ka background halka gray hoga.

Real-World Example: Website ke sections ko visually separate karne ke liye alag background colors use hote hain.

background-image

Ye property element ke background mein image lagati hai. url() function se image path specify karte hain.

Code Example:

```
.hero { background-image:
url('banner.jpg');
}
```

Output / Result: Hero section ka background specified image se bhar jayega (default repeat ke sath).

Real-World Example: Website ke hero/banner sections mein full-width background images is tarah lagayi jaati hain.

background-repeat

Ye control karta hai background image repeat (tile) hogi ya nahi. Values: repeat (default, dono directions), repeat-x, repeat-y, no-repeat.

Code Example:

```
.pattern {
background-image: url('texture.png'); background-repeat: no-repeat;
}
```

Output / Result: Image sirf ek baar dikhegi, repeat nahi hogi.

Real-World Example: Logo ya single decorative image ke liye hamesha no-repeat use hota hai.

background-position

Ye batata hai background image container ke andar kahan position hogi - keywords (center, top, left) ya values (px, %) use kar sakte hain.

Code Example:

```
.hero {
background-image: url('banner.jpg');
background-position: center center;
}
```

Output / Result: Image container ke bilkul center mein position hogi.

Real-World Example: Hero images ko hamesha center karke important content visible rakha jata hai.

background-size

Ye image ka size control karta hai. 'cover' image ko poore container ko cover karne ke liye scale karta hai (crop ho sakti hai), 'contain' पूरी image fit karta hai (blank space reh sakti hai).

Code Example:

```
.hero {  
  background-image: url('banner.jpg');  background-size: cover;  
  background-position: center;  
}
```

Output / Result: Image poore section ko without distortion cover karegi.

Real-World Example: Ye combination (cover + center) almost har modern website ke hero section mein use hoti hai.

background-attachment

Ye control karta hai background image scroll ke sath move kare ya fixed rahe. 'fixed' value se parallax-jaisa effect milta hai.

Code Example:

```
.parallax {  
  background-image: url('mountains.jpg');  background-attachment: fixed;  
  background-size: cover;  
}
```

Output / Result: Scroll karte waqt background image apni jagah 'fixed' rahega, content uske upar se scroll hoga - parallax effect.

Real-World Example: Storytelling websites aur landing pages mein parallax scroll effect ke liye ye use hota hai.

Module 4: Borders

Border

Border property element ke charon taraf line banati hai. Iske 3 parts hote hain: width, style (solid, dashed, dotted, double), aur color.

Code Example:

```
.box {  border: 2px  
solid #333;  
}
```

Output / Result: Box ke charon taraf 2px ki solid dark border dikhegi.

Real-World Example: Cards, buttons, input fields - sab jagah border se elements ko define kiya jata hai.

Border Radius

border-radius corners ko round karta hai. Value jitni zyada, corner utna zyada round hoga. 50% value se perfect circle/oval ban jata hai.

Code Example:

```
.card {  border-radius: 12px;  
}  
.avatar {  
  border-radius: 50%;  width: 80px;  
  height: 80px;  
}
```

Output / Result: Card ke corners halke round honge, avatar image perfect circle ban jayegi.

Real-World Example: Profile pictures (Facebook, WhatsApp) hamesha border-radius: 50% se circular banayi jaati hain.

Outline

Outline border jaisi hi line hai lekin element ke box-model se bahar honi hai (space nahi leti) aur usually accessibility (focus state) ke liye use hoti hai jab user Tab se element select kare.

Code Example:

```
button:focus { outline: 3px
solid dodgerblue;
}
```

Output / Result: Jab button par keyboard se focus aayega, uske around ek blue outline dikhegi.

Real-World Example: Accessibility ke liye outline zaroori hai taake keyboard users dekh saken kaunsa element currently selected hai.

Module 5: Box Model

Box Model Overview

Har HTML element ek 'box' hota hai jiske 4 layers hain (andar se bahar): Content (actual text/image), Padding (content ke around khali space, background color tak extend hota hai), Border (padding ke around line), aur Margin (border ke bahar khali space, transparent hota hai, dusre elements se distance banata hai). Box model samajhna CSS layout ki foundation hai.

Code Example:

```
.box { width: 200px;
padding: 20px; border:
5px solid black; margin:
15px;
}
```

Output / Result: Box ki actual total width hogi: $200 + (20 * 2 \text{ padding}) + (5 * 2 \text{ border}) + (15 * 2 \text{ margin ka space}) = \text{visual layout mein zyada jagah legi.}$

Real-World Example: Layout bugs (elements overlap ya misaligned hona) aksar box model samajh na hone ki wajah se hote hain.

Margin

Margin element ke bahar ka space hai, jo usay dusre elements se door rakhta hai. Individual sides set kar sakte hain: margin-top, margin-right, margin-bottom, margin-left, ya shorthand.

Code Example:

```
.section {
margin: 20px;
}
.center-box { margin: 0 auto; width: 300px;
}
```

Output / Result: Section ke charon taraf 20px space hoga; center-box horizontally page ke center mein aa jayega (margin: 0 auto trick).

Real-World Example: Elements ko horizontally center karne ke liye 'margin: 0 auto' ek bohot common CSS trick hai.

Padding

Padding content aur border ke darmiyan ka space hai, jo background color ke andar hota hai. Element ko 'breathing space' deta hai.

Code Example:

```
button {  
  padding: 12px 24px;  background-color: dodgerblue;  color: white;  
}
```

Output / Result: Button ke text ke around top/bottom 12px aur left/right 24px space hoga, jisse button bara aur clickable dikhega.

Real-World Example: Buttons aur cards mein padding se hi woh 'comfortable' spacious feel aati hai.

Content

Content box ki innermost part hai jahan actual text, image, ya child elements hote hain. width aur height properties directly is content area ko control karte hain (box-sizing ke depend karte hue).

Code Example:

```
.box {  width: 300px;  
        height: 150px;  
}
```

Output / Result: Content area 300x150px hoga (default box-sizing ke sath, isme padding/border شامل nahi).

Real-World Example: Fixed-size cards ya containers banane ke liye content width/height set ki jaati hai.

Box-sizing

box-sizing: content-box (default) mein width/height sirf content area ko refer karta hai, padding/border extra add hoti hai. box-sizing: border-box mein width/height mein padding aur border already included hoti hai, jo calculations ko bohot aasan banata hai. Best practice hai * { box-sizing: border-box; } sabse upar likhna.

Code Example:

```
* {  box-sizing: border-  
box;  
}  
.box {  width:  
300px;  padding:  
20px;  
        border: 5px solid black;  
}
```

Output / Result: Box-sizing border-box ki wajah se poora box (content+padding+border) exactly 300px width ka rahega, extra nahi badhega.

Real-World Example: Almost har modern CSS framework (Bootstrap, Tailwind) border-box ko default use karta hai.

Module 6: Typography

Fonts (font-family)

font-family element ka font specify karta hai. Multiple fonts comma se dete hain (fallback ke liye) - agar pehla font available na ho to browser agla try karta hai, aakhir mein generic family (serif/sans-serif) hoti hai.

Code Example:

```
body { font-family: 'Segoe UI', Arial,
sans-serif;
}
```

Output / Result: Text 'Segoe UI' font mein dikhega, agar woh na mile to Arial, warna koi bhi default sans-serif font.

Real-World Example: Professional websites hamesha fallback fonts list karti hain taake sab devices par text sahi dikhe.

Google Fonts

Google Fonts free custom fonts provide karta hai jo link tag se import karke use kar sakte hain, bina font file khud download kiye.

Code Example:

```
<!-- HTML head mein -->
<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;700&display=swap"
rel="stylesheet">

<!-- CSS mein -->
<style>
  body { font-family: 'Poppins', sans-serif; } </style>
```

Output / Result: Poori website ka font Poppins ho jayega, jo default system fonts se zyada attractive lagta hai.

Real-World Example: Modern websites (startups, portfolios) apna unique typography Google Fonts se hi hasil karti hain.

Font Weight

font-weight text ki boldness control karta hai: normal (400), bold (700), ya numeric values 100-900 (agar font family support kare).

Code Example:

```
h1 { font-weight: 700; } p
{ font-weight: 300; }
```

Output / Result: Heading bold dikhegi, paragraph halka thin (light) dikhega.

Real-World Example: Design hierarchy banane ke liye headings heavy weight aur body text light/normal weight rakha jata hai.

Line Height

line-height lines ke darmiyan vertical spacing control karta hai. Behtar readability ke liye usually 1.4 se 1.8 ke darmiyan rakha jata hai.

Code Example:

```
p { line-height:
1.6;
}
```

Output / Result: Paragraph ki lines ke darmiyan comfortable spacing hogi, text parhna aasan hoga.

Real-World Example: Blog articles mein line-height 1.6-1.8 use hota hai taake lamba text parhne mein aankhein na thakein.

Letter Spacing

letter-spacing characters ke darmiyan horizontal space control karta hai. Positive value spacing barhati hai, negative kam karti hai.

Code Example:

```
h1 {  
  letter-spacing: 2px;  
  text-transform: uppercase;  
}
```

Output / Result: Heading ke letters ke darmiyan thoda extra space hoga - ye style aksar uppercase headings ke sath use hoti hai.

Real-World Example: Logo text aur bare headings mein letter-spacing se ek premium, spacious look milta hai.

Text Align text-align text ko horizontally align karta hai: left (default), right, center, justify (dono taraf edges align, newspaper style).

Code Example:

```
.title {  text-align:  
center;  
}  
.article {  text-  
align: justify;  
}
```

Output / Result: Title center mein aayega, article ka text dono taraf se seedha align hoga.

Real-World Example: Headings aur call-to-action sections aksar center-aligned hote hain.

Text Transform text-transform text ka case change karta hai bina actual HTML text change kiye: uppercase, lowercase, capitalize.

Code Example:

```
button {  text-transform:  
uppercase;  
}
```

Output / Result: Button ka text automatically CAPITAL LETTERS mein dikhega, chahe HTML mein lowercase likha ho.

Real-World Example: Buttons aur navigation menus mein consistent uppercase style ke liye ye use hota hai.

Text Decoration

text-decoration text ke around lines control karta hai: underline, line-through, none (links se default underline hatane ke liye sabse zyada use hota hai).

Code Example:

```
a {      text-decoration:  
none;  
}  
.strikethrough {  text-  
decoration: line-through;  
}
```

Output / Result: Link ka default underline hat jayega (custom hover effect ke liye common practice), dusra text strikethrough dikhega.

Real-World Example: Modern websites mein links ka default browser underline hata kar custom hover styles use karte hain.

Module 7: Display

display: block

Block elements apni poori available width lete hain aur hamesha nayi line se start hote hain (jaise div, p, h1). Inpar width, height, margin, padding sab apply hoti hain.

Code Example:

```
div {  
  display: block; width: 200px;  
  background: lightblue;  
}
```

Output / Result: Div poori line legi (default), width set karne par 200px tak limit hogi, agla element neeché jayega.

Real-World Example: Sections, containers, cards - sab default block elements hain.

display: inline

Inline elements sirf apne content jitni width lete hain, next line par nahi jate (jaise span, a, strong). width aur height properties inpar effect nahi karti.

Code Example:

```
span {  
  display: inline;  
  color: red;  
}
```

Output / Result: Span apni text ki jitni width legi aur same line mein baaki content ke sath rahegi.

Real-World Example: Text ke andar single word highlight karne ke liye span (inline) use hota hai.

display: inline-block

Inline-block dono ki khoobiyen combine karta hai - element line mein rehta hai (inline ki tarah) lekin width, height, margin, padding set kar sakte hain (block ki tarah).

Code Example:

```
button {  
  display: inline-block; width: 120px; height: 40px;  
}
```

Output / Result: Buttons ek dusre ke sath same line mein rahenge lekin apni fixed width/height rakhenge.

Real-World Example: Navigation menu items aur buttons row mein dikhane ke liye inline-block classic tareeka hai (ab flexbox zyada use hota hai).

display: none

display: none element ko completely hide kar deta hai - na woh dikhta hai, na page mein space leta hai, jaise woh exist hi nahi karta.

Code Example:

```
.hidden-message {  
  display: none;  
}
```

Output / Result: Is class wala element bilkul invisible hoga aur uski jagah bhi khali nahi rahegi.

Real-World Example: JavaScript ke sath milkar dropdown menus, modals, tabs show/hide karne ke liye display: none use hota hai.

Module 8: Position

position: static

Ye default position hai - element normal document flow mein rehta hai, top/left/right/bottom properties koi effect nahi karti.

Code Example:

```
div { position:
static;
}
```

Output / Result: Element apni normal jagah par rahega, jaisa HTML order mein likha gaya hai.

Real-World Example: Zyada tar elements default static hi rehte hain jab tak special positioning ki zaroorat na ho.

position: relative

Relative element apni normal position ke relative move hota hai using top/left/right/bottom, lekin uski original jagah document flow mein reserved rehti hai (space khali nahi hoti).

Code Example:

```
.box {
position: relative; top: 10px;
left: 20px;
}
```

Output / Result: Box apni original position se 10px neeche aur 20px right shift hoga, lekin uski asal jagah abhi bhi reserved hai.

Real-World Example: Relative positioning aksar absolute children ke liye 'reference point' banane ke liye use hoti hai.

position: absolute

Absolute element document flow se nikal jata hai (uski jagah khali nahi rehti) aur apne nearest 'positioned' (relative/absolute/fixed) parent ke relative position hota hai. Agar koi positioned parent na ho to poore page (body) ke relative hota hai.

Code Example:

```
.parent { position: relative;
}
.badge {
position: absolute; top: 0;
right: 0;
}
```

Output / Result: Badge parent ke top-right corner mein exactly fix ho jayega.

Real-World Example: Notification badges, tooltips, dropdown menus absolute positioning se banaye jate hain.

position: fixed

Fixed element viewport (browser window) ke relative position hota hai aur scroll karne par bhi apni jagah par 'fixed' rehta hai.

Code Example:

```
.navbar { position: fixed; top: 0;
width: 100%;
}
```

Output / Result: Navbar hamesha screen ke top par chipka rahega, chahe user kitna bhi scroll kare. **Real-World**

Example: Sticky navigation bars aur 'back to top' buttons fixed positioning se banaye jate hain.

position: sticky

Sticky element normal flow mein rehta hai jab tak user scroll na kare, phir ek specified point (jaise top: 0) tak pahunchne par 'fixed' ho jata hai.

Code Example:

```
h2.section-title { position: sticky; top: 0;
background: white;
}
```

Output / Result: Jab tak section screen mein hai, heading normal rahegi; jab scroll ho kar top tak pahunchegi, wahan chipak jayegi.

Real-World Example: Table headers aur category headings mein sticky positioning bohot common hai (jaise Amazon ki category filters).

z-index

z-index elements ki stacking order (kaunsa upar, kaunsa neeche) control karta hai jab woh overlap karte hain. Sirf positioned elements (relative/absolute/fixed/sticky) par kaam karta hai. Bara number upar dikhega.

Code Example:

```
.modal { position: fixed;
z-index: 1000;
}
.overlay { position: fixed;
z-index: 999;
}
```

Output / Result: Modal (z-index 1000) overlay (z-index 999) ke upar dikhega.

Real-World Example: Popups, dropdowns, aur modals mein z-index se ensure kiya jata hai ke woh baaki content ke upar dikhein.

Module 9: Flexbox

Flex Container

display: flex se parent element ek 'flex container' ban jata hai, uske direct children automatically 'flex items' ban jate hain jo by default ek row mein side-by-side aa jate hain.

Code Example:

```
.container {
display: flex;
}
```

Output / Result: Container ke andar sab children ek horizontal row mein side-by-side aa jayenge (default direction).

Real-World Example: Flexbox modern websites mein layout banane ka sabse common tareeka hai.

Flex Items & flex-direction

flex-direction batata hai items kis direction mein arrange hongy: row (default, horizontal), column (vertical), row-reverse, column-reverse.

Code Example:

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

Output / Result: Items ab horizontal ki bajaye vertically ek dusre ke neeche stack hongy.

Real-World Example: Mobile layouts mein aksar column direction use hoti hai, desktop mein row.

justify-content

justify-content items ko main axis (default: horizontal) par align karta hai: flex-start, flex-end, center, space-between, space-around, space-evenly.

Code Example:

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
}
```

Output / Result: Navbar ke items - pehla item bilkul left aur akhri item bilkul right, darmiyan mein equal space.

Real-World Example: Navigation bars mein logo left aur menu items right dikhane ke liye space-between bohot use hota hai.

align-items align-items items ko cross axis (default: vertical) par align karta hai: flex-start, flex-end, center, stretch (default).

Code Example:

```
.container {  
  display: flex;  
  align-items: center;  
  height: 300px;  
}
```

Output / Result: Items container ki height ke vertically center mein align ho jayengi.

Real-World Example: Kisi bhi element ko perfectly vertically center karne ke liye align-items: center sabse aasan tareeka hai.

gap

gap property flex (ya grid) items ke darmiyan spacing deta hai, bina margin use kiye - ye cleaner aur zyada predictable hai.

Code Example:

```
.container {  
  display: flex;  
  gap: 20px;  
}
```

Output / Result: Har item ke darmiyan 20px ka consistent gap hoga.

Real-World Example: Card grids aur button groups mein gap property se equal spacing di jati hai.

flex-wrap

Default mein flex items ek hi line mein fit hone ki koshish karte hain (shrink ho jate hain). flex-wrap: wrap unhe multiple lines mein wrap hone deta hai jab space kam ho.

Code Example:

```
.gallery { display: flex; flex-wrap: wrap; gap: 10px; }
```

Output / Result: Agar items ek line mein fit na hon to woh automatically agli line par chale jayenge.

Real-World Example: Image galleries aur product grids mein flex-wrap responsive layout banane ke liye essential hai.

Module 10: CSS Grid

Grid Container

display: grid se parent element grid container ban jata hai, jisme hum rows aur columns dono ek sath define kar sakte hain - flexbox se zyada powerful 2D layout system.

Code Example:

```
.container { display: grid; }
```

Output / Result: Container grid layout ke liye ready ho jayega, lekin columns/rows define karna zaroori hai.

Real-World Example: Complex page layouts (dashboard, magazine-style) CSS Grid se banaye jate hain.

Grid Items

Grid container ke direct children automatically 'grid items' ban jate hain jo defined grid cells mein place hote hain.

Code Example:

```
.container { display: grid; grid-template-columns: 1fr 1fr 1fr; }  
<!-- HTML mein 3 divs honge -->
```

Output / Result: Teeno child divs equal width ke 3 columns mein arrange ho jayenge.

Real-World Example: 'fr' unit (fraction) grid mein available space ko proportionally divide karta hai.

Grid Template Columns/Rows grid-template-columns aur grid-template-rows batate hain kitne columns/rows honge aur unki size kya hogi.

Code Example:

```
.container { display: grid; grid-template-columns: 200px 1fr 1fr; grid-template-rows: 100px auto; }
```

Output / Result: Pehla column fixed 200px, baaki do equal flexible width lenge; pehli row 100px, dusri content ke mutabiq.

Real-World Example: Sidebar layouts (fixed sidebar + flexible content) is tarah banaye jate hain.

Grid Area

grid-template-areas se hum named regions define kar sakte hain jaise ek visual map banate hain, phir har item ko gridarea se assign karte hain - layout samajhna bohot aasan ho jata hai.

Code Example:

```
.container {  
  display: grid;  grid-  
  template-areas:  
  "header header"  
    "sidebar content"  
    "footer footer";  
}  
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }
```

Output / Result: Ek classic website layout ban jayega: full-width header, sidebar+content, full-width footer.

Real-World Example: Ye technique dashboard aur admin panel layouts ke liye bohot popular hai kyunke visually samajhna aasan hota hai.

Gap (Grid)

Grid mein bhi gap property row aur column gaps ke liye use hoti hai, row-gap aur column-gap alag se bhi set kar sakte hain.

Code Example:

```
.gallery {  display: grid;  grid-  
  template-columns: repeat(3, 1fr);  gap:  
  15px;  
}
```

Output / Result: 3 columns ka grid banega, har item ke darmiyan 15px ka gap hoga.

Real-World Example: repeat(3, 1fr) syntax se equal-width columns quickly banaye jate hain, bar-bar likhne ki zaroorat nahi.

Module 11: Responsive Design

Media Queries

Media queries CSS ko conditionally apply karte hain based on screen size (ya device features). @media rule ke andar likhi jati hain.

Code Example:

```
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

Output / Result: Jab screen width 768px ya usse chota ho (tablets/mobiles), container ki items column mein stack ho jayengi.

Real-World Example: Har responsive website (jo mobile par bhi sahi dikhti hai) media queries use karti hai.

Mobile First Approach

Mobile-first ka matlab hai pehle mobile ke liye base CSS likhna, phir min-width media queries se larger screens ke liye enhance karna. Ye approach performance aur maintainability ke liye better maani jati hai.

Code Example:

```
/* Base styles - mobile ke liye */
.container { display: block; }

/* Larger screens ke liye enhance */
@media (min-width: 768px) {
  .container { display: flex; }
}
```

Output / Result: Mobile par default block layout, tablet/desktop (768px+) par flex layout activate hoga.

Real-World Example: Google aur modern frameworks (Tailwind, Bootstrap 4+) mobile-first approach recommend karte hain.

Breakpoints

Breakpoints woh specific screen widths hain jahan layout change hoti hai. Common breakpoints: 576px (mobile), 768px (tablet), 992px (laptop), 1200px (desktop) - ye exact values project ke mutabiq vary karti hain.

Code Example:

```
@media (min-width: 576px) { /* Mobile Large */ }
@media (min-width: 768px) { /* Tablet */ }
@media (min-width: 992px) { /* Desktop */ }
@media (min-width: 1200px) { /* Large Desktop */ }
```

Output / Result: Har breakpoint par layout apne mutabiq adjust hoga taake har device par perfect dikhe.

Real-World Example: E-commerce websites (Daraz, Amazon) multiple breakpoints use karti hain taake mobile, tablet, desktop sab par UI optimized ho.

Module 12: Transforms

translate `translate()` element ko uski original position se move karta hai (X aur Y axis par) bina document flow ko affect kiye.

Code Example:

```
.box:hover { transform:
  translateY(-10px);
}
```

Output / Result: Hover karne par box 10px upar move hoga - ek subtle 'lift' effect. **Real-World**

Example: Cards par hover effect (upar uthna) is technique se banaya jata hai.

rotate `rotate()` element ko specified degrees mein rotate karta hai (clockwise positive, counter-clockwise negative).

Code Example:

```
.icon:hover { transform:
  rotate(180deg);
}
```

Output / Result: Hover karne par icon 180 degree ghoom jayega (ulta ho jayega).

Real-World Example: Dropdown arrow icons expand hone par rotate hote hain (jaise accordion menus mein).

scale scale() element ka size barhata/ghatata hai. scale(1) normal size, scale(1.1) 10% bara, scale(0.9) 10% chota.

Code Example:

```
.product-image:hover {  
  transform: scale(1.1);  
}
```

Output / Result: Hover karne par image 10% zoom-in ho jayegi.

Real-World Example: E-commerce product images par ye zoom-on-hover effect bohot common hai.

skew skew() element ko tirschha (slant) kar deta hai X ya Y axis par, ek dynamic/creative visual effect ke liye.

Code Example:

```
.banner { transform:  
skewY(-3deg);  
}
```

Output / Result: Banner section thoda tirschha (slanted) dikhega, ek modern dynamic design effect.

Real-World Example:

Creative agency websites mein sections ko skew karke unique, non-boring layouts banaye jate hain.

Module 13: Transitions

CSS Transitions

Transitions kisi property ke value change hone par smooth animation create karti hain (jaise color, size, position change), sudden jump ki bajaye. Syntax: transition: property duration timing-function delay.

Code Example:

```
button {  
  background-color: dodgerblue;  
  transition: background-color 0.3s ease;  
}  
button:hover { background-  
color: darkblue;  
}
```

Output / Result: Hover karne par button ka color achanak change hone ki bajaye 0.3 second mein smoothly darkblue ho jayega.

Real-World Example: Har modern website ke buttons, links, aur cards par hover effects transitions se smooth banaye jate hain.

Multiple Property Transitions

Ek sath multiple properties par transition apply kar sakte hain, comma se separate karke, ya 'all' keyword se sab properties cover kar sakte hain.

Code Example:

```
.card { transition: transform 0.3s ease, box-shadow 0.3s  
ease;  
}  
.card:hover {  
  transform: translateY(-5px); box-  
shadow: 0 10px 20px rgba(0,0,0,0.2);  
}
```

Output / Result: Hover karne par card smoothly upar uthega aur uski shadow bhi smoothly deep hogi, dono ek sath animate honge.

Real-World Example: Product cards aur portfolio items mein ye combined hover effect professional touch deta hai.

Module 14: Animations

CSS Animations (@keyframes)

Animations transitions se zyada powerful hain kyunke multiple steps (keyframes) define kar sakte hain aur automatically (bina hover/trigger ke) chala sakte hain, repeat bhi kar sakte hain. @keyframes se stages define karte hain, phir animation property se element par apply karte hain.

Code Example:

```
@keyframes bounce {
  0% { transform: translateY(0); }
  50% { transform: translateY(-20px); }
  100% { transform: translateY(0); }
}
.icon { animation: bounce 1s infinite;
}
```

Output / Result: Icon continuously upar-neeche bounce karega, 1 second mein ek cycle, hamesha ke liye repeat hoga.

Real-World Example: Loading spinners, attention-grabbing icons, aur notification badges is tarah animate hote hain.

Animation Properties

animation shorthand mein multiple properties hoti hain: name, duration, timing-function, delay, iteration-count (kitni baar chale, ya infinite), direction (normal/reverse/alternate).

Code Example:

```
.loader {
  animation-name: spin; animation-duration: 2s; animation-iteration-count: infinite;
  animation-timing-function: linear;
}
@keyframes spin {
  from { transform: rotate(0deg); } to { transform: rotate(360deg); }
}
```

Output / Result: Loader continuously 360 degree ghoomta rahega, consistent speed ke sath (linear).

Real-World Example: Loading spinners (jab website data fetch kar rahi ho) is exact pattern se banaye jate hain.

Module 15: Pseudo Classes

What are Pseudo Classes

Pseudo classes element ki special state ko target karte hain jo normal HTML mein nazar nahi aati, jaise hover hona, focus hona, ya list mein position. Syntax: selector:pseudo-class { }.

Code Example:

```
a:hover {
  color: red;
}
```

Output / Result: Jab mouse link par jayega, uska color red ho jayega.

Real-World Example: Interactive UI (hover effects, focus states) pseudo classes ke bina possible nahi.

:hover, :focus, :active

:hover jab mouse element par ho, :focus jab element select/active ho (jaise input field), :active jab element click ho raha ho (mouse press hone ke waqt).

Code Example:

```
button:hover { background: darkblue; }
input:focus { border-color: blue; }
button:active { transform: scale(0.95); }
```

Real-World Example:

Output / Result: Button hover par dark hoga, input focus par blue border lega, click ke waqt button thoda chota (press effect) dikhega.

Ye teeno states har interactive website element (buttons, inputs, links) mein use hoti hain.

:nth-child()

`nth-child()` specific position wale children ko select karta hai - number, 'even', 'odd', ya formula $(2n+1)$ use kar sakte hain.

Code Example:

```
tr:nth-child(even) { background-color: #f2f2f2; }
```

Output / Result: Table ki even rows (2nd, 4th, 6th...) ka background light gray hoga - zebra striping effect.

Real-World Example: Tables mein alternate row colors is exact technique se banaye jate hain, readability ke liye.

:first-child & :last-child first-child parent ke pehle child element ko target karta hai, last-child aakhri child ko.

Code Example:

```
li:first-child { font-weight: bold; }
li:last-child { border-bottom: none; }
```

Output / Result: List ka pehla item bold hoga, aakhri item ki bottom border hataa di jayegi.

Real-World Example: Lists aur menus mein pehle/aakhri item ka special styling (jaise border remove karna) is se hoti hai.

:not() `not()` selector kisi condition ke ilawa baaki sab elements select karta hai - negation/exclusion ke liye.

Code Example:

```
button:not(.disabled) { cursor: pointer; }
```

Output / Result: Jo buttons 'disabled' class nahi rakhte, unpar cursor pointer (hand icon) dikhega.

Real-World Example: Disabled states ko exclude karke baaki interactive elements par proper styling apply karne ke liye ye useful hai.

Module 16: Pseudo Elements

::before and ::after

::before aur ::after element ke andar content ke pehle/baad ek 'virtual' element insert karte hain, bina HTML change kiye. content property zaroori hai (khali bhi ho sakti hai).

Code Example:

```
.required::after { content: " *";  
  color: red;  
}
```

Output / Result: Jis bhi element par 'required' class hai, uske text ke baad ek red asterisk (*) automatically add ho jayega.

Real-World Example: Form fields mein required (*) marks aur decorative icons is technique se add kiye jate hain, bina extra HTML likhe.

::first-letter and ::first-line

::first-letter pehle character ko target karta hai (drop-cap effect ke liye), ::first-line pehli line ko.

Code Example:

```
p::first-letter { font-size: 3em; font-weight: bold;  
  float: left;  
}
```

Output / Result: Paragraph ka pehla letter bara aur bold dikhega, magazine-style 'drop cap' effect.

Real-World Example: Magazine aur blog article designs mein ye classic 'drop cap' style use hoti hai.

::selection

::selection user jab text ko mouse se select (highlight) kare us waqt ka color style karta hai.

Code Example:

```
::selection { background: gold;  
  color: black;  
}
```

Output / Result: Jab user page ka text select karega, default blue highlight ki bajaye gold color dikhega.

Real-World Example: Brand-consistent websites apna selection color bhi customize karti hain.

Module 17: CSS Variables

Defining & Using CSS Variables

CSS Variables (Custom Properties) reusable values store karte hain, --variable-name se define hote hain (usually :root selector mein, jo global scope deta hai) aur var() function se use hote hain. Ye theme colors, spacing, fonts consistent rakhne ke liye bohot powerful hain.

Code Example:

Real-World Example:

```
:root {
  --primary-color: #4a90e2;
  --spacing: 16px;
}

button {
  background-color: var(--primary-color);
  padding: var(--spacing);
} h1 { color: var(--primary-color);
}
```

Output / Result: Button aur heading dono same blue color use karenge, jo ek hi jagah (root) se control hota hai.

Dark mode / Light mode toggle CSS variables se hi implement hota hai - sirf root variables change karke poori site ka theme badal jata hai.

CSS Variables for Theming (Dark Mode Example)

CSS variables ki value context (jaise class) ke mutabiq override ki ja sakti hai, jo dynamic theming enable karta hai.

Code Example:

```
:root {
  --bg-color: white;
  --text-color: black;
}
.dark-mode {
  --bg-color: #1a1a1a;
  --text-color: white;
} body
{
  background: var(--bg-color);
  color: var(--text-color);
}
```

Output / Result: Normal mein white background/black text, lekin body par 'dark-mode' class add karte hi poori site dark theme mein switch ho jayegi.

Real-World Example: Modern apps (Twitter, YouTube) ka dark mode toggle isi concept par based hota hai.

Module 18: Shadows

box-shadow

box-shadow element ke around shadow banata hai. Syntax: box-shadow: x-offset y-offset blur-radius spread-radius color.

Code Example:

```
.card { box-shadow: 0 4px 12px
  rgba(0,0,0,0.15);
}
```

Output / Result: Card ke neeche ek soft, subtle shadow dikhegi jo usay page se 'uthaa hua' (elevated) feel deti hai.

Real-World Example: Material Design (Google) ki elevation ka poora system box-shadow par based hai - cards, buttons, modals sab.

text-shadow text-shadow text ke around shadow deta hai, syntax box-shadow jaisa hi hai lekin text ke liye.

Code Example:

```
h1 { text-shadow: 2px 2px 4px  
      rgba(0,0,0,0.5);  
}
```

Output / Result: Heading text ke neeche-right thodi si shadow dikhegi, jo text ko depth deti hai.

Real-World Example: Hero banners mein image ke upar likhe text ko readable banane ke liye text-shadow use hoti hai.

Module 19: Gradients

Linear Gradient

linear-gradient() do ya zyada colors ke darmiyan smooth transition banata hai ek straight direction mein. Direction (deg ya keyword jaise 'to right') aur colors specify karte hain.

Code Example:

```
.hero { background: linear-gradient(to right, #4a90e2,  
#9013fe);  
}
```

Output / Result: Background left se right blue se purple mein smoothly transition karega.

Real-World Example: Modern SaaS websites aur app UIs mein gradient backgrounds bohot trendy hain (buttons, hero sections).

Radial Gradient radial-gradient() colors ko center se bahar ki taraf circular/elliptical pattern mein transition karta hai.

Code Example:

```
.spotlight { background: radial-gradient(circle,  
#ffffff, #333333);  
}
```

Output / Result: Center mein white color hoga jo dheere dheere edges ki taraf dark gray mein badlega - 'spotlight' jaisa effect.

Real-World Example: Spotlight effects aur decorative background patterns radial gradients se banaye jate hain.

Gradient as Text (Creative Use)

background-clip: text aur transparent color ke combination se gradient text effect banaya ja sakta hai - ek popular modern design trend.

Code Example:

```
h1 { background: linear-gradient(to right, #f857a6, #ff5858);  
      -webkit-background-clip: text;  
      -webkit-text-fill-color: transparent;  
}
```

Real-World Example:

Output / Result: Heading ka text solid color ki bajaye pink-to-red gradient mein dikhega.

Real-World Example: Startup landing pages mein colorful gradient headings bohot popular hain (jaise Stripe, Linear ki websites).

Module 20: Best Practices

Consistent Naming Convention (BEM)

BEM (Block Element Modifier) ek popular CSS naming convention hai: `.block__element--modifier`. Ye classes ko organized, reusable, aur naming conflicts se paak rakhta hai large projects mein.

Code Example:

```
<div class="card">
  <h2 class="card__title">Title</h2>
```

```
<button class="card__button card__button--primary">Buy</button> </div>

.card__title { font-size: 20px; }
.card__button--primary { background: blue; }
```

Output / Result: Class names se hi pata chal jata hai konsa element kis block ka hissa hai aur uska variant kya hai.

Real-World Example: Large scale projects (enterprise apps) mein BEM se code maintain karna bohot aasan ho jata hai.

Avoid !important

!important kisi bhi CSS rule ki normal priority (specificity) ko override kar deta hai. Ye 'quick fix' lagta hai lekin future mein debugging bohot mushkil bana deta hai kyunke normal rules kaam nahi karte. Best practice: sahi selectors aur specificity use karke isse avoid karein.

Code Example:

```
/* Avoid this */
.button { color: red !important; }

/* Better: use more specific selector */
.card .button { color: red; }
```

Output / Result: Dono se same result milega, lekin dusra approach maintainable aur predictable hai.

Real-World Example: Bare codebases mein !important ka overuse ek common 'CSS smell' hai jo refactoring mushkil banata hai.

Organize CSS with Comments & Sections

Bare CSS files ko logical sections mein divide karein (Reset, Variables, Typography, Layout, Components) aur comments se separate karein - taake file navigate karna aasan ho.

Code Example:

```
/* ===== Variables ===== */
:root { --primary: #4a90e2; }

/* ===== Typography ===== */ body { font-family: sans-serif; }

/* ===== Buttons ===== */ button { padding: 10px 20px; }
```

Output / Result: File clearly organized sections mein dikhegi jo team collaboration ke liye behtar hai.

Real-World Example: Large CSS files (1000+ lines) mein ye organization dhoondne mein bohot time bachati hai.

Use Shorthand Properties

Shorthand properties multiple related properties ko ek line mein combine karti hain, code chota aur readable banate hain.

Code Example:

```
/* Instead of 4 lines */
margin-top: 10px; margin-right:
15px; margin-bottom: 10px;
margin-left: 15px;

/* Use shorthand */ margin:
10px 15px;
```

Output / Result: Dono same result dete hain, lekin shorthand code ko chota aur maintain karna aasan banata hai.

Real-World Example: Professional CSS codebases hamesha shorthand (margin, padding, font, background) use karti hain jahan possible ho.

Module 21: CSS Projects - Responsive Landing Page

Ab hum CSS ke saare concepts combine karke ek real project banayenge - ek complete 'Responsive Landing Page' jisme Flexbox, Grid, Gradients, Shadows, Transitions, Media Queries, aur CSS Variables sab use honge. Ye pattern samajhne ke baad aap khud Portfolio, Dashboard, ya E-commerce page jaisi cheezein bana sakte hain.

Project Steps:

- Step 1: HTML structure banayein - navbar, hero section, services (3 cards), footer.
- Step 2: CSS variables (:root) mein primary color aur spacing define karein taake reuse ho sake.
- Step 3: Navbar ko Flexbox se style karein (justify-content: space-between) aur sticky positioning dein.
- Step 4: Hero section mein linear-gradient background aur centered text lagayein.
- Step 5: Services section ko CSS Grid se 3 equal columns mein arrange karein.
- Step 6: Cards par box-shadow aur hover par transform+shadow transition add karein.
- Step 7: Media query add karein taake mobile par grid 1 column ho jaye aur navbar column mein stack ho.
- Step 8: Browser mein resize karke test karein - desktop aur mobile dono par sahi dikhna chahiye.

Complete Project Code — [index.html](#):

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Landing Page</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="navbar">
```

```
<h2 class="logo">DevStudio</h2>
<nav>
  <a href="#">Home</a>
  <a href="#services">Services</a>
  <a href="#pricing">Pricing</a>
</nav>
</header>

<section class="hero">
  <h1>We Build Beautiful Websites</h1>
  <p>Frontend development services jo aapke business ko next level tak le jayein.</p>
  <button class="btn-primary">Get Started</button>
</section>

<section id="services" class="services">
  <div class="card">
    <h3>Web Design</h3>
    <p>Modern, responsive, aur attractive designs.</p>
  </div>
  <div class="card">
    <h3>Development</h3>
    <p>Clean, fast, aur scalable code.</p>
  </div>
  <div class="card">
    <h3>SEO</h3>
    <p>Google par top rank hasil karein.</p>
  </div>
</section>
<footer>
  <p>&copy; 2026 DevStudio. All rights reserved.</p>
</footer>
</body>
</html>
```

Complete Project Code — [style.css](#):

```
:root {
  --primary-color: #4a90e2;
  --dark-color: #1a1a2e;
  --spacing: 20px;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body
{
  font-family: 'Segoe UI', sans-serif;    line-
height: 1.6;
  color: #333;
}

/* ===== Navbar ===== */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: var(--spacing) 40px;
  background: white;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  position: sticky;
  top: 0;
  z-index: 100;
}
```

```
.navbar nav a {    margin-  
left: 20px;    text-  
decoration: none;    color:  
var(--dark-color);  
    transition: color 0.3s ease;  
}
```

```
.navbar nav a:hover {    color:
var(--primary-color);
}

/* ===== Hero Section ===== */
.hero {
    text-align: center;    padding: 100px 20px;    background: linear-
gradient(to right, var(--primary-color), #9013fe);    color: white;
}

.hero h1 {    font-
size: 48px;
    margin-bottom: 15px;
}

.btn-primary {    margin-top: 20px;
padding: 14px 32px;    background:
white;    color: var(--primary-
color);    border: none;    border-
radius: 8px;    font-weight: bold;
cursor: pointer;    transition:
transform 0.3s ease;
}

.btn-primary:hover {
    transform: translateY(-4px);
}

/* ===== Services (CSS Grid) ===== */
.services {    display: grid;    grid-
template-columns: repeat(3, 1fr);    gap:
var(--spacing);
    padding: 60px 40px;
}

.card {
    padding: 30px;    border-
radius: 12px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);    transition:
transform 0.3s ease, box-shadow 0.3s ease;
}
```

```
.card:hover {  
  transform: translateY(-8px);    box-  
shadow: 0 12px 24px rgba(0,0,0,0.15);  
}
```

```
footer {  
  text-align: center;  
padding: 20px;    background:  
var(--dark-color);    color:  
white;  
}  
  
/* ===== Responsive: Mobile ===== */  
@media (max-width: 768px) {  
  .services {  
    grid-template-columns: 1fr;  
  }  
  .navbar {  
    flex-direction: column;  
gap: 10px;  
  }  
  .hero h1 {  
    font-size: 32px;  
  }  
}
```

Practice Karne Ke Liye Baaki Project Ideas:

- Pricing Table: 3 pricing cards banayein Flexbox se, middle card ko 'featured' style (bara, shadow) dein.
- Image Gallery: CSS Grid se responsive photo gallery banayein jisme hover par image zoom ho.
- Testimonials Slider: Cards ka horizontal scroll section banayein gradient background ke sath.
- Dark Mode Toggle: CSS Variables use karke light/dark theme switch karne wala button banayein.
- Animated Loading Spinner: @keyframes se ek custom loading spinner banayein.
- Login Form UI: Centered login card banayein box-shadow, border-radius, aur focus states ke sath.

— End of part 2:CSS---

Chapter # 3 JAVASCRIPT

(ES6+)

Is section mein JavaScript ke tamam 13 modules deeply, beginner-level se explain kiye gaye hain — har topic ke sath code example, uska output, aur real-world use case diya gaya hai. Aakhir mein ek complete Todo List App project bhi banaya gaya hai jisme ye saare concepts combine ho kar use hote hain.

Module 1: Introduction & Basics

Introduction to JavaScript

JavaScript woh programming language hai jo websites ko 'interactive' aur 'dynamic' banati hai. Agar HTML skeleton hai aur CSS makeup hai, to JavaScript brain/behavior hai - ye button clicks handle karti hai, data validate karti hai, animations control karti hai, aur server se data fetch karti hai. JavaScript browser mein directly chalti hai (koi installation ki zaroorat nahi) aur ab server-side (Node.js) mein bhi use hoti hai.

Code Example:

```
<script>
  alert('Hello    from    JavaScript!');
  console.log('Ye  console  mein  dikhega');
</script>
```

Output / Result: Browser mein ek popup 'Hello from JavaScript!' dikhega, aur console (F12 se open karein) mein message print hoga.

Real-World Example: Facebook ka like button, Instagram ki stories, Google Maps ki dragging - sab JavaScript se possible hain.

Variables (var, let, const)

Variables data store karne ke containers hain. 3 tareeqon se declare hote hain: var (purana, function-scoped, avoid karein), let (modern, block-scoped, value change ho sakti hai), const (block-scoped, value change nahi ho sakti - constant). Best practice: hamesha const use karein, agar value change karni ho to let.

Code Example:

```
let age = 25; const
name = "Ali";
age = 26; // ye chal jayega
// name = "Sara"; // ye error dega

console.log(age, name);
```

Output / Result: Console mein '26 Ali' print hoga.

Real-World Example: User ka naam, age, ya form data store karne ke liye variables har website mein use hote hain.

Data Types

JavaScript mein primitive data types hain: String (text), Number (numbers), Boolean (true/false), Undefined (value assign nahi hui), Null (intentionally empty), aur complex types: Object, Array, Function. typeof operator se kisi variable ka type check kar sakte hain.

Code Example:

```
let name = "Ali";      // String let  
age = 25;              // Number
```

```
let isStudent = true;    // Boolean let
city;                   // Undefined let
score = null;           // Null

console.log(typeof name, typeof age, typeof isStudent);
```

Output / Result: Console mein 'string number boolean' print hoga.

Real-World Example: Form validation mein data types check karna zaroori hota hai (jaise age number honi chahiye, text nahi).

Operators

Operators values par operations perform karte hain: Arithmetic (+, -, *, /, %), Comparison (==, ===, !=, >, <), Logical (&&, ||, !), Assignment (=, +=, -=). === (strict equality) type bhi check karta hai, == sirf value (isliye === use karna best practice hai).

Code Example:

```
let a = 10, b = 3; console.log(a
+ b);    // 13
console.log(a % b);    // 1 (remainder)
console.log(a > b);    // true console.log(5 ===
"5"); // false (different types) console.log(5 ==
"5"); // true (loose comparison)
```

Output / Result: Har console.log apna respective result print karega jaisa comment mein likha hai.

Real-World Example: Discount calculation, age verification, login checks - sab operators use karte hain.

Type Conversion

Type conversion ek data type ko dusre mein badalna hai. Implicit conversion JavaScript khud kar deti hai (coercion), explicit conversion hum Number(), String(), Boolean() functions se karte hain.

Code Example:

```
let str = "123";
let num = Number(str); // 123 (number) console.log(num +
1);    // 124

let val = String(456); // "456" (string) console.log(typeof
val); // string
```

Output / Result: Console mein '124' aur 'string' print hoga.

Real-World Example: Form inputs hamesha text/string mein aate hain, calculation ke liye unhe Number() se convert karna padta hai.

Module 2: Strings & Numbers

Strings & String Methods

Strings text data hain, single, double, ya backticks se likhi ja sakti hain. Common methods: .length (lambai), .toUpperCase(), .toLowerCase(), .trim() (extra spaces hatana), .includes(), .slice(), .substring(), .split() (array mein todna), .replace().

Code Example:

```
let text = " Hello World ";
console.log(text.trim());    // "Hello World"
```

```
console.log(text.toUpperCase()); // "HELLO WORLD"
console.log(text.includes("World")); // true
console.log(text.trim().split(" ")); // ["Hello", "World"]
```

Output / Result: Har method apna specific transformation perform karega jaisa comments mein dikhaya gaya.

Real-World Example: Search functionality (includes), username formatting (trim, toUpperCase) - roz use hote hain.

Numbers & Number Methods

Number data type integers aur decimals dono handle karta hai. Useful methods: .toFixed() (decimal places fix karna), Number.isInteger(), parseInt()/parseFloat() (string se number nikalna).

Code Example:

```
let price = 99.9999;
console.log(price.toFixed(2)); // "100.00"

let str = "42px"; console.log(parseInt(str)); // 42
```

Output / Result: Console mein '100.00' aur '42' print hoga.

Real-World Example: E-commerce websites price ko hamesha .toFixed(2) se 2 decimal places tak round karti hain.

Math Object

Math built-in object hai jo mathematical operations provide karta hai: Math.round(), Math.floor(), Math.ceil(), Math.random() (0 se 1 tak random number), Math.max(), Math.min(), Math.pow().

Code Example:

```
console.log(Math.round(4.6)); // 5
console.log(Math.floor(4.9)); // 4 console.log(Math.random());
// e.g. 0.732...
console.log(Math.max(10, 25, 3)); // 25
```

Output / Result: Har Math function apna calculation result print karega.

Real-World Example: Random number generation (OTP codes, lottery, games) mein Math.random() bohot use hota hai.

Template Literals

Template literals backticks (`) se likhe jate hain aur \${} ke andar variables/expressions directly embed karne dete hain - string concatenation (+) se zyada readable hai. Multi-line strings bhi easily ban sakti hain.

Code Example:

```
let name = "Sara"; let
age = 22;
console.log(`Mera naam ${name} hai aur mein ${age} saal ki hun.`); console.log(`Agla
saal age hogi: ${age + 1}`);
```

Output / Result: Console mein 'Mera naam Sara hai aur mein 22 saal ki hun.' aur 'Agla saal age hogi: 23' print hoga.

Real-World Example: Dynamic messages (jaise 'Welcome back, Ali!') template literals se hi banaye jate hain.

Module 3: Control Flow

Conditionals (if, else if, else)

Conditionals decision-making enable karte hain - code sirf tab chalta hai jab condition true ho. if, else if (multiple conditions), else (fallback).

Code Example:

```
let marks = 75; if (marks
>= 90) {
  console.log("Grade A+"); }
else if (marks >= 70) {
  console.log("Grade A");
} else {
  console.log("Needs Improvement");
}
```

Output / Result: Console mein 'Grade A' print hoga kyunke marks 70 se zyada hain lekin 90 se kam.

Real-World Example: Login validation ('agar password sahi hai to dashboard, warna error dikhao') conditionals se hoti hai.

Switch Statement

Switch multiple specific values check karne ka clean tareeka hai, jab bohot saari else-if chains likhni pare to iski jagah switch behtar readable hota hai.

Code Example:

```
let day = 3; switch (day) {
  case 1:
    console.log("Monday");
    break; case 2:
    console.log("Tuesday");
    break; case 3:
    console.log("Wednesday");
    break; default:
    console.log("Invalid day");
}
```

Output / Result: Console mein 'Wednesday' print hoga.

Real-World Example: Menu navigation ya multi-option forms mein switch statement use hoti hai.

Loops (for, while)

Loops code ko repeatedly chalate hain. for loop jab iterations ki count pata ho, while loop jab condition-based repeat karna ho. for...of arrays ke liye, for...in objects ke liye use hota hai.

Code Example:

```
for (let i = 1; i <= 5; i++) {
  console.log("Count: " + i);
}

let i = 0; while
(i < 3) {
  console.log("While loop: " + i);
  i++;
}
```

Output / Result: For loop 1 se 5 tak count print karega, while loop 0,1,2 print karega.

Real-World Example: Product lists render karna, table rows generate karna - sab loops se hota hai.

break & continue break loop ko turant rok deta hai, continue current iteration skip karke agli par chala jata hai.

Code Example:

```
for (let i = 1; i <= 10; i++) {  
  if (i === 5) break;   if (i % 2  
  === 0) continue;  
  console.log(i);  
}
```

Output / Result: Console mein 1, 3 print honge (even numbers skip honge, aur 5 par loop ruk jayega).

Real-World Example: Search algorithms mein jab match mil jaye to break se loop rok dete hain, unnecessary iterations bachane ke liye.

Module 4: Functions & Scope

Functions

Functions reusable code blocks hain jo specific task perform karte hain. Function Declaration, Function Expression, aur Arrow Function - teen tareeqon se define ho sakte hain. Parameters input lete hain, return value output dete hain.

Code Example:

```
function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
const add = (a, b) => a + b;  
  
console.log(greet("Ali")); console.log(add(5, 3));
```

Output / Result: Console mein 'Hello, Ali!' aur '8' print hoga.

Real-World Example: Reusable calculations (jaise tax calculate karna, discount lagana) functions mein wrap kiye jate hain taake bar-bar likhna na pare.

Default & Rest Parameters

Default parameters function call mein value na di jaye to fallback value use karte hain. Rest parameter (...) unlimited arguments ko ek array mein collect karta hai.

Code Example:

```
function greet(name = "Guest") {  
  console.log(`Welcome, ${name}`);  
}  
greet(); // koi argument nahi diya  
  
function sum(...numbers) { return numbers.reduce((a,b) => a+b, 0);  
}  
console.log(sum(1,2,3,4));
```

Output / Result: Console mein 'Welcome, Guest' aur '10' print hoga.

Real-World Example: Flexible functions (jaise koi bhi count of items sum karna) rest parameters se banaye jate hain.

Scope (Global, Function, Block)

Scope decide karta hai variable kahan accessible hai. Global scope - पूरी file mein accessible. Function scope - sirf us function ke andar. Block scope (let/const) - sirf {} ke andar jahan define hua.

Code Example:

```
let globalVar = "Mein sab jagah accessible hun";

function testScope() {
  let localVar = "Mein sirf function ke andar hun";
  console.log(globalVar); // ye chalega
  console.log(localVar); // ye bhi chalega
}
testScope();
// console.log(localVar); // ye error dega, bahar accessible nahi
```

Output / Result: Function ke andar dono variables print honge, lekin function ke bahar localVar access karne par error aayega.

Real-World Example: Scope samajhna bugs se bachata hai - accidental variable overwriting large codebases mein scope ki wajah se hoti hai.

Hoisting

Hoisting JavaScript ka wo behavior hai jisme variable aur function declarations code chalne se pehle 'upar utha' li jaati hain memory mein. var hoisted hoti hai (undefined ke sath), let/const bhi hoisted hoti hain lekin 'temporal dead zone' mein hoti hain (use karne se pehle error dega). Function declarations पूरी तरह hoisted hoti hain (call kar sakte hain define hone se pehle bhi).

Code Example:

```
console.log(x); // undefined (hoisted lekin value nahi)
var x = 5;

sayHello(); // ye kaam karega function
sayHello() {
  console.log("Hello!");
}
```

Output / Result: Pehla console.log 'undefined' print karega, dusra function call successfully 'Hello!' print karega.

Real-World Example: Hoisting samajhna zaroori hai taake unexpected 'undefined' errors debug kar sakein.

Closures

Closure ek function hai jo apne 'outer function' ke variables ko yaad rakhta hai chahe outer function khatam ho chuki ho. Ye data privacy aur state management ke liye bohot powerful concept hai.

Code Example:

```
function counter() {  
  let count = 0;  
  return function() {  
    count++;    return  
    count;  
  };  
}  
  
const increment = counter(); console.log(increment());  
// 1 console.log(increment()); // 2  
console.log(increment()); // 3
```

Output / Result: Har call par count barhata rahega: 1, 2, 3 - kyunke inner function ne 'count' variable yaad rakha hai.

Real-World Example: React ke useState hook internally closures use karta hai state maintain karne ke liye.

'this' Keyword

'this' current execution context ko refer karta hai. Regular function mein 'this' us object ko refer karta hai jisne function call kiya. Arrow functions apna 'this' nahi rakhte, apne surrounding (lexical) scope se 'this' lete hain.

Code Example:

```
const person = {  name: "Ali",  greet:  
function() {    console.log(`Hi, I am  
${this.name}`);  
  } };  
person.greet();
```

Output / Result: Console mein 'Hi, I am Ali' print hoga kyunke 'this' yahan 'person' object ko refer kar raha hai.

Real-World Example: Object methods mein 'this' se hum us object ki apni properties access karte hain.

Module 5: Arrays, Objects & Dates

Arrays

Array ordered list of values store karta hai (square brackets [] mein). Har item ka index hota hai (0 se start). Common methods: .push() (end mein add), .pop() (end se remove), .length.

Code Example:

```
let fruits = ["Apple", "Banana", "Mango"]; fruits.push("Orange");  
console.log(fruits);          // ["Apple", "Banana", "Mango", "Orange"]  
console.log(fruits[1]);       // "Banana" console.log(fruits.length);  
// 4
```

Output / Result: Array mein 4 items honge, index 1 par 'Banana' hoga.

Real-World Example: Product lists, todo items, user lists - sab arrays mein store hote hain.

Array Methods: map, filter, reduce

Ye 3 methods modern JavaScript ka dil hain. map() har item ko transform karke naya array banata hai, filter() condition ke mutabiq items chunta hai, reduce() poore array ko ek single value mein compress karta hai (jaise sum).

Code Example:

```
let numbers = [1, 2, 3, 4, 5];
```

```
let doubled = numbers.map(n => n * 2); console.log(doubled); // [2, 4, 6, 8, 10]

let evens = numbers.filter(n => n % 2 === 0); console.log(evens); // [2, 4]

let total = numbers.reduce((sum, n) => sum + n, 0); console.log(total); // 15
```

Output / Result: Teeno console.log apne respective transformed results dikhayenge.

Real-World Example: React mein list rendering (map), search filters (filter), aur cart total (reduce) is tarah hi kaam karte hain.

Array Methods: forEach, find, sort

forEach() har item par ek function chalata hai (koi naya array nahi banta), find() pehla matching item return karta hai, sort() array ko arrange karta hai (default string sort, numbers ke liye comparator function chahiye).

Code Example:

```
let nums = [5, 2, 8, 1];
nums.forEach(n => console.log(n * 10));

let found = nums.find(n => n > 4); console.log(found); // 5

nums.sort((a, b) => a - b); console.log(nums); // [1, 2, 5, 8]
```

Output / Result: forEach har number ko 10 se multiply karke print karega, find pehla number 4 se bara return karega, sort ascending order mein array arrange karega.

Real-World Example: Leaderboards aur price sorting (low to high) is sort() method se hoti hai.

Objects

Object key-value pairs store karta hai ({} mein), real-world entities represent karne ke liye best hai. Dot notation (obj.key) ya bracket notation (obj["key"]) se access karte hain.

Code Example:

```
let user = {
  name: "Sara",
  age: 24,
  isAdmin: false
};

console.log(user.name); // "Sara"
console.log(user["age"]); // 24
user.city = "Karachi"; // naya property add karna console.log(user);
```

Output / Result: user.name 'Sara' print karega, phir object mein 'city' property bhi add ho jayegi.

Real-World Example: API se aane wala data (user profile, product details) usually objects ki form mein hota hai.

Object Methods (keys, values, entries)

Object.keys() sab keys ka array deta hai, Object.values() sab values ka, Object.entries() [key, value] pairs ka array deta hai - loops ke sath milkar objects iterate karne ke liye use hote hain.

Code Example:

```
let user = { name: "Ali", age: 25, city: "Lahore" };

console.log(Object.keys(user));    // ["name", "age", "city"] console.log(Object.values(user));
// ["Ali", 25, "Lahore"]

Object.entries(user).forEach(([key, value]) => {
  console.log(`${key}: ${value}`); });
```

Output / Result: Har property key: value format mein print hogi.

Real-World Example: Dynamic forms banane ke liye jab object ki saari properties ko loop karke display karna ho, ye methods use hote hain.

Dates

Date object date aur time handle karta hai. new Date() current date/time deta hai. Methods: .getFullYear(), .getMonth(), .getDate(), .getDay(), .toLocaleDateString().

Code Example:

```
let now = new Date();
console.log(now.getFullYear()); // e.g. 2026 console.log(now.toLocaleDateString());
// e.g. 7/10/2026

let specificDate = new Date("2026-12-25"); console.log(specificDate.getDay()); //
day of week (0=Sunday)
```

Output / Result: Current year aur formatted date print hogi.

Real-World Example: Blog posts ki publish date, order tracking timestamps - sab Date object se handle hote hain.

Module 6: DOM & Events

What is the DOM?

DOM (Document Object Model) HTML page ka JavaScript-accessible tree representation hai. Har HTML element DOM mein ek 'node' hai jise JavaScript select, modify, add, ya remove kar sakti hai - isi se websites 'dynamic' banti hain.

Code Example:

```
// HTML: <h1 id="title">Hello</h1> console.log(document.title);
// page ka title console.log(document.body);    // poora body
element
```

Output / Result: Console mein document ki properties print hongii.

Real-World Example: Jab aap kisi website par like button click karte hain aur count badalta hai, ye DOM manipulation se hota hai.

Selecting Elements

getElementById() ek element uski id se select karta hai. querySelector() CSS selector syntax use karta hai (pehla matching element). querySelectorAll() sab matching elements ka list deta hai.

Code Example:

```
// HTML: <p class="msg">Hello</p> const byId = document.getElementById("title"); const byQuery = document.querySelector(".msg"); const allMsgs = document.querySelectorAll(".msg");  
  
console.log(byQuery.textContent); // "Hello"
```

Output / Result: Console mein 'Hello' print hoga (paragraph ka text).

Real-World Example: Form validation ke liye pehle input fields ko select karna padta hai, phir unki value check hoti hai.

Modifying Elements

Elements select karne ke baad unhe modify kar sakte hain: `.textContent` (text change), `.innerHTML` (HTML content change), `.style` (CSS change), `.setAttribute()` (attribute change), `classList.add()/remove()/toggle()` (class manage).

Code Example:

```
const title = document.querySelector("h1");  
title.textContent = "Naya Title"; title.style.color  
= "blue"; title.classList.add("highlight");
```

Output / Result: Heading ka text change hoga, color blue ho jayega, aur 'highlight' class add ho jayegi.

Real-World Example: Dark mode toggle button `classList.toggle()` se hi kaam karta hai - class add/remove karke theme badalta hai.

Creating & Removing Elements

`document.createElement()` naya element banata hai, `appendChild()/append()` usay page mein add karta hai, `remove()` element ko delete karta hai.

Code Example:

```
const newLi = document.createElement("li"); newLi.textContent = "Naya Item";  
document.querySelector("ul").appendChild(newLi);  
  
// Remove karna  
// newLi.remove();
```

Output / Result: List mein ek naya item 'Naya Item' add ho jayega.

Real-World Example: Todo apps mein naya task add karna is exact technique se hota hai.

Events (addEventListener)

Events user interactions hain (click, hover, keypress, submit). `addEventListener()` se function attach karte hain jo event hone par chalega.

Code Example:

```
const btn = document.querySelector("button");  
btn.addEventListener("click", function() {  
  alert("Button clicked!");  
});  
  
document.addEventListener("keydown", (e) => {  
  console.log(`Key pressed: ${e.key}`); });
```

Output / Result: Button click par alert popup aayega, koi bhi key press karne par console mein us key ka naam print hoga.

Real-World Example: Har interactive feature (form submit, dropdown open, image slider) events se hi trigger hoti hai.

Event Delegation

Event delegation ek technique hai jisme parent element par ek hi event listener lagate hain (child elements ke bajaye), aur event.target se check karte hain kaunsa child clicked hua. Ye performance ke liye behtar hai especially dynamic content ke liye.

Code Example:

```
document.querySelector("ul").addEventListener("click", function(e) { if (e.target.tagName === "LI") { console.log("Clicked item: " + e.target.textContent); } });
```

Output / Result: List ke kisi bhi li item par click karne par uska text console mein print hoga - chahe woh item baad mein dynamically add hua ho.

Real-World Example: Todo apps mein jahan items dynamically add/delete hote hain, event delegation se har naye item par alag se listener lagane ki zaroorat nahi padti.

Module 7: Classes, OOP & Modules

Classes

Class object banane ka blueprint/template hai. constructor() naya object banate waqt initial values set karta hai. Methods class ke andar define hoti hain.

Code Example:

```
class Person {
  constructor(name, age) {
    this.name = name;    this.age
    = age;
  } greet() { console.log(`Hi, I'm ${this.name},
  ${this.age} years old.`); } }

const p1 = new Person("Ali", 25); p1.greet();
```

Output / Result: Console mein 'Hi, I'm Ali, 25 years old.' print hoga.

Real-World Example: React class components (purane style) aur backend models (Node.js) classes se hi banaye jate hain.

Inheritance

Inheritance ek class ko dusri class ki properties/methods 'extend' (inherit) karne deta hai, code reusability barhata hai. extends keyword use hota hai, super() parent constructor call karta hai.

Code Example:

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  makeSound() { console.log(`${this.name}
  makes a sound.`); } }

class Dog extends Animal {
  makeSound() {
    console.log(`${this.name} barks.`); }
}

const d = new Dog("Rex"); d.makeSound();
```

Output / Result: Console mein 'Rex barks.' print hoga (Dog class ne Animal ka method override kiya).

Real-World Example: UI component libraries mein base components banakar unhe extend karke specialized versions banayi jaati hain.

OOP Principles (Encapsulation, Abstraction)

OOP ke 4 core principles hain: Encapsulation (data aur methods ko ek unit mein bandhna, private fields se protect karna), Abstraction (complex logic ko simple interface ke peeche chupana), Inheritance (upar cover ki gayi), Polymorphism (same method naam, alag behavior different classes mein).

Code Example:

```
class BankAccount {
  #balance = 0; // private field
  deposit(amount) {
    this.#balance += amount;
  }
  getBalance() {
    return this.#balance;
  }
}

const acc = new BankAccount(); acc.deposit(500);

console.log(acc.getBalance()); // 500
// console.log(acc.#balance); // Error - private hai
```

Output / Result: Balance 500 print hoga, lekin #balance ko directly access karne ki koshish error degi.

Real-World Example: Banking aur payment systems mein sensitive data (balance) hamesha private rakha jata hai, sirf controlled methods se access hota hai.

Modules (import/export)

Modules code ko alag files mein organize karne dete hain. export se kisi file se cheez bahar bhejte hain, import se dusri file mein use karte hain. Ye code ko maintainable aur reusable banata hai.

Code Example:

```
// math.js file
export function add(a, b) {
  return a + b;
}
export const PI = 3.14159;

// main.js file
import { add, PI } from './math.js';
console.log(add(2, 3)); // 5 console.log(PI);
// 3.14159
```

Output / Result: main.js mein import ki gayi functions/values use ho sakengi.

Real-World Example: React components hamesha modules ki tarah export/import hote hain - har component apni file mein.

Module 8: ES6+ Features

let & const (ES6 Basics)

ES6 (2015) mein var ki jagah let aur const introduce hui, jo block-scoping deti hain aur bohot saare bugs (jaise loop variable leakage) fix karti hain. Ye modern JavaScript ka foundation hai.

Code Example:

```
for (let i = 0; i < 3; i++) {  setTimeout(()
=> console.log(i), 100);
}
// Output: 0, 1, 2 (let ki wajah se, var se 3,3,3 aata)
```

Output / Result: 0, 1, 2 print honge - har iteration ka apna 'i' hota hai let ke sath.

Real-World Example: Ye classic interview question hai: 'var vs let in loops' - modern code hamesha let/const use karta hai.

Destructuring

Destructuring arrays/objects se values easily nikalne deta hai, individual variables mein - ek line mein multiple assignments.

Code Example:

```
const user = { name: "Sara", age: 24, city: "Lahore" };
const { name, age } = user; console.log(name, age); //
"Sara" 24

const numbers = [10, 20, 30]; const
[first, second] = numbers;
console.log(first, second); // 10 20
```

Output / Result: Object aur array dono se values seedha variables mein nikal aayengi.

Real-World Example: React mein props aur state destructuring se hi access kiye jate hain: const { name, age } = props.

Spread Operator

Spread (...) ek array/object ke elements ko 'phaila' (spread) deta hai. Arrays/objects copy karne, merge karne, ya function arguments pass karne ke liye use hota hai.

Code Example:

```
const arr1 = [1, 2, 3]; const arr2 =  
[...arr1, 4, 5]; console.log(arr2); //  
[1, 2, 3, 4, 5]  
  
const obj1 = { name: "Ali" }; const  
obj2 = { ...obj1, age: 25 };  
console.log(obj2); // { name: "Ali", age: 25 }
```

Output / Result: Naye array aur object banenge jinme purani values ke sath nayi values bhi merge hongy.

Real-World Example: React mein state update karte waqt spread se purana state copy karke naya data merge kiya jata hai (immutability).

Rest Parameter

Rest (...) function parameters mein multiple arguments ko ek array mein collect karta hai (spread ka opposite context mein use).

Code Example:

```
function sum(...nums) { return nums.reduce((total, n) => total + n, 0);  
}  
console.log(sum(1, 2, 3, 4, 5)); // 15
```

Output / Result: Console mein '15' print hoga, chahe kitne bhi arguments diye jayen.

Real-World Example: Flexible utility functions (jaise Math.max jaisa custom function) rest parameters se banaye jate hain.

Module 9: Asynchronous JavaScript

Callbacks

Callback ek function hai jo dusre function ko argument ki tarah pass kiya jata hai, aur baad mein call hota hai (aksar kisi task ke complete hone par). Ye asynchronous programming ka pehla tareeqa tha.

Code Example:

```
function fetchData(callback) { setTimeout(() => {  
    callback("Data mil gaya!");  
}, 1000);  
}  
  
fetchData((result) => {  
    console.log(result);  
});
```

Output / Result: 1 second ke baad console mein 'Data mil gaya!' print hoga.

Real-World Example: Purane AJAX-based websites callbacks se hi server data handle karti thin.

Promises

Promise ek object hai jo future mein complete hone wali operation (jaise API call) ko represent karta hai. 3 states: pending, fulfilled (resolve), rejected (reject). .then() success handle karta hai, .catch() error handle karta hai. Callbacks se zyada readable hai ('callback hell' se bachata hai).

Code Example:

```
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    let success = true;
    if (success) resolve("Data mil gaya!");
    else reject("Error hua!");
  }, 1000);
});

fetchData
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

Output / Result: 1 second baad console mein 'Data mil gaya!' print hoga (kyunke success true hai).

Real-World Example: Har API call (fetch, axios) internally Promises return karti hai.

Async/Await

async/await Promises ko likhne ka modern, cleaner tareeqa hai - synchronous code jaisa dikhta hai lekin asynchronous kaam karta hai. async keyword function ko Promise return karne wala banata hai, await Promise resolve hone ka wait karta hai.

Code Example:

```
async function getData() {
  try {
    let response = await
    fetch("https://api.example.com/data");
    let data = await
    response.json();
    console.log(data);
  } catch (error) {
    console.log("Error: ", error);
  }
}

getData();
```

Output / Result: API se data fetch hoga aur console mein print hoga, ya error catch block mein handle hoga.

Real-World Example: Modern React apps mein data fetching (useEffect ke andar) almost hamesha async/await se ki jaati hai.

Fetch API

fetch() built-in function hai jo server se data get/send karta hai (HTTP requests). Ye Promise return karta hai. Default GET request hoti hai, options object se method/headers/body customize kar sakte hain.

Code Example:

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then(response => response.json())
  .then(data => console.log(data.name))
  .catch(error => console.log("Error:", error));
```

Output / Result: API se user data fetch ho kar uska naam console mein print hoga.

Real-World Example: Weather apps, news feeds, social media - sab APIs se data fetch() (ya axios) se hi lete hain.

JSON

JSON (JavaScript Object Notation) data exchange ka standard format hai - APIs isi format mein data bhejti hain. JSON.stringify() JS object ko JSON string mein badalta hai, JSON.parse() JSON string ko wapis JS object mein badalta hai.

Code Example:

```
const user = { name: "Ali", age: 25 }; const jsonString = JSON.stringify(user);  
console.log(jsonString); // '{"name":"Ali","age":25}'  
  
const backToObject = JSON.parse(jsonString); console.log(backToObject.name);  
// "Ali"
```

Output / Result: Object JSON string mein convert hoga, phir wapis object mein parse hoga.

Real-World Example: localStorage mein complex data (arrays/objects) store karne se pehle hamesha JSON.stringify() karna padta hai.

Module 10: Browser Storage

Local Storage

localStorage browser mein data permanently store karta hai (tab/browser band karne ke baad bhi rehta hai jab tak manually clear na kiya jaye). Sirf strings store kar sakta hai, isliye objects ke liye JSON.stringify/parse use karte hain.

Code Example:

```
localStorage.setItem("username", "Ali"); console.log(localStorage.getItem("username")); // "Ali"  
  
localStorage.setItem("user", JSON.stringify({ name: "Sara", age: 24 })); const user =  
JSON.parse(localStorage.getItem("user")); console.log(user.name); // "Sara"  
  
localStorage.removeItem("username");
```

Output / Result: Data store, retrieve, aur remove hoga - browser refresh karne ke baad bhi 'user' data mojood rahega.

Real-World Example: Dark mode preference, login tokens, aur cart items localStorage mein save kiye jate hain.

Session Storage

sessionStorage bhi localStorage jaisa hai, lekin data sirf us tab/session tak rehta hai - tab band karte hi delete ho jata hai. Syntax same hai (setItem, getItem, removeItem).

Code Example:

```
sessionStorage.setItem("tempData", "Ye sirf is session ke liye hai");  
console.log(sessionStorage.getItem("tempData"));  
  
// Tab band karne par ye automatically delete ho jayega
```

Output / Result: Data current tab session mein available rahega, lekin tab close karte hi khatam ho jayega.

Real-World Example: Multi-step forms (jaise checkout process) ka temporary data sessionStorage mein rakha jata hai.

Module 11: Error Handling & Regular Expressions

Error Handling (try, catch, finally)

try/catch code ko safely run karne deta hai - agar error aaye to program crash nahi hota, catch block usay handle karta hai. finally block hamesha chalta hai (error aaye ya nahi).

Code Example:

```
try {
  let result = riskyOperation();
  console.log(result); } catch
(error) {
  console.log("Kuch ghalat hua: " + error.message);
} finally {
  console.log("Ye hamesha chalega");
}

function riskyOperation() {
  throw new Error("Custom error!");
}
```

Output / Result: Console mein 'Kuch ghalat hua: Custom error!' phir 'Ye hamesha chalega' print hoga.

Real-World Example: API calls fail hone par (internet issue, server down) try/catch se graceful error messages dikhaye jate hain, app crash nahi hoti.

Regular Expressions (Regex)

Regex patterns hain jo text mein specific formats match/search/validate karte hain - email validation, phone number format, password strength check ke liye bohot use hote hain. test() method match hone par true/false return karta hai.

Code Example:

```
const emailPattern = /^[^@.\s]+\.[^@.\s]+\.[^@.\s]+$/;
console.log(emailPattern.test("test@example.com")); // true console.log(emailPattern.test("invalid-
email")); // false

const phonePattern = /^03\d{9}$/; console.log(phonePattern.test("03001234567")); // true
```

Output / Result: Valid formats ke liye true, invalid ke liye false return hoga.

Real-World Example: Signup forms mein email/phone/password validation regex se hi hoti hai.

Module 12: Advanced Concepts

Debounce & Throttle

Debounce ek function ko tab tak call hone se rokta hai jab tak user rukta nahi (jaise typing khatam hone ke baad search chalana). Throttle function ko fixed interval mein sirf ek baar chalne deta hai (jaise scroll event har 200ms mein sirf ek baar). Dono performance optimize karte hain.

Code Example:

```
function debounce(func, delay) { let timer;
return function(...args) { clearTimeout(timer);
timer = setTimeout(() => func(...args), delay);
};
}

const search = debounce((query) => {
console.log("Searching for: " + query); },
500);

// User type karte waqt search() bar bar call hoga
// lekin actual search sirf typing rukne ke 500ms baad chalegi
```

Output / Result: User typing khatam karne ke 500ms baad hi 'Searching for: ...' console mein print hoga, har keystroke par nahi.

Real-World Example: Google search bar, live search filters - sab debounce use karte hain taake har letter par API call na ho.

Execution Context & Call Stack

Execution context woh 'environment' hai jahan JavaScript code run hota hai (global context, function context). Call stack track karta hai kaunse functions currently execute ho rahe hain (LIFO - Last In First Out order mein).

Code Example:

```
function first() { second();
console.log("First function");
}
function second() {
console.log("Second function");
}
first();
// Output order: "Second function" phir "First function"
```

Output / Result: 'Second function' pehle print hoga, phir 'First function' - kyunke second() first() ke andar call hoti hai aur pehle complete hoti hai.

Real-World Example: Stack overflow errors (infinite recursion) call stack ki wajah se hi hote hain - interview mein commonly poocha jata hai.

Event Loop

Event Loop JavaScript ka mechanism hai jo asynchronous operations (setTimeout, fetch, promises) ko handle karta hai. JavaScript single-threaded hai (ek waqt mein ek kaam), lekin event loop background tasks complete hone par unhe queue se uthakar main thread par execute karta hai, jab call stack khali ho.

Code Example:

```
console.log("1");
setTimeout(() => console.log("2"), 0); console.log("3");
// Output: 1, 3, 2 (setTimeout hamesha baad mein chalta hai, chahe delay 0 ho)
```

Output / Result: Console mein order hoga: 1, 3, 2 - setTimeout ka callback event loop se hi chalta hai, synchronous code ke baad.

Real-World Example: Ye JavaScript interviews ka bohot common concept hai - samajhna zaroori hai ke async code kab actually chalta hai.

Web APIs (Browser APIs)

Web APIs browser ke built-in features hain jo JavaScript se access hote hain, core JS language ka hissa nahi hain. Examples: DOM API, Fetch API, Geolocation API, LocalStorage API, Notification API.

Code Example:

```
// Geolocation API example
navigator.geolocation.getCurrentPosition((position) => {
  console.log(position.coords.latitude, position.coords.longitude);
});

// Notification API example if
(Notification.permission === "granted") {
  new Notification("Hello from Web API!");
}
```

Output / Result: User ki location coordinates print hogi (permission dene ke baad), aur ek browser notification show hogi.

Real-World Example: Ride-hailing apps (Careem, Uber) user ki location Geolocation API se hi lete hain.

Module 13: Mini Projects - Todo List App

Ab hum JavaScript ke saare concepts combine karke ek real project banayenge - ek complete 'Todo List App' jisme DOM manipulation, events, event delegation, arrays, localStorage, aur template literals sab use honge. Ye ek classic beginner project hai jo interview aur portfolio dono ke liye bohot important hai.

Project Steps:

- Step 1: HTML structure banayein - input field, add button, aur empty ul (task list).
- Step 2: localStorage se purane tasks load karein (agar mojud hon) using JSON.parse.
- Step 3: renderTasks() function banayein jo tasks array ko DOM mein loop kar ke dikhaye (map/forEach + createElement ya innerHTML).
- Step 4: addTask() function banayein jo input value le kar tasks array mein push kare, phir localStorage update kare.
- Step 5: Add button aur Enter key dono se addTask() call hone ka event listener lagayein.
- Step 6: Event Delegation use karke parent 'taskList' par ek listener lagayein jo Done/Delete buttons handle kare.
- Step 7: Har change ke baad saveTasks() (localStorage) aur renderTasks() call karein taake UI aur data sync rahen.
- Step 8: Browser mein test karein - task add karein, done mark karein, delete karein, aur page refresh kar ke check karein data persist ho raha hai ya nahi.

Complete Project Code — index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Todo App</title>
<link rel="stylesheet" href="style.css">
```

```
</head>
<body>
  <div class="todo-app">
    <h1>My Todo List</h1>
    <div class="input-group">
      <input type="text" id="taskInput" placeholder="Naya task likhein...">
    <button id="addBtn">Add</button>
    </div>
    <ul id="taskList"></ul>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

Complete Project Code — style.css:

```
body {
  font-family: 'Segoe UI', sans-serif;
  background: #f4f4f9; display: flex;
  justify-content: center;
  padding-top: 60px;
}

.todo-app {
  background: white;
  padding: 30px; border-radius: 12px;
  box-shadow: 0 4px 15px rgba(0,0,0,0.1);
  width: 400px;
}

.input-group {
  display: flex; gap:
10px;
  margin: 20px 0;
}

#taskInput {
  flex: 1;
  padding: 10px; border:
1px solid #ccc;
  border-radius: 6px;
}

#addBtn {
```

```

padding: 10px 16px;    background: #4a90e2;    color: white;    border: none;
border-radius: 6px;
cursor: pointer;
}

#taskList li {    display: flex;    justify-content: space-between;    align-items: center;
padding: 10px;    border-bottom: 1px solid #eee;
}

#taskList li.completed .task-text {    text-decoration: line-through;    color: #999;
}

```

Complete Project Code — script.js:

```

// ===== Todo App Logic =====

// LocalStorage se purane tasks load karein (agar hon) let
tasks = JSON.parse(localStorage.getItem("tasks")) || [];

const taskInput = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn"); const
taskList = document.getElementById("taskList");

// Tasks ko screen par render karne wala function
function renderTasks() {    taskList.innerHTML = "";
// pehle list khali karein

    tasks.forEach((task, index) => {
        const li = document.createElement("li");
li.className = task.completed ? "completed" : "";
li.innerHTML = `
            <span class="task-text">${task.text}</span>
            <div class="task-actions">
                <button class="done-btn" data-index="${index}">Done</button>
                <button class="delete-btn" data-index="${index}">Delete</button>
            </div>
        `;
        taskList.appendChild(li);
    });
}

// LocalStorage mein tasks save karne wala function
function saveTasks() {    localStorage.setItem("tasks",
JSON.stringify(tasks));
}

```

```
// Naya task add karna function
addTask() {
  const text = taskInput.value.trim();    if (text === "")
  return; // khali input ignore karein

  tasks.push({ text: text, completed: false });
  taskInput.value = "";    saveTasks();
  renderTasks();
}

addBtn.addEventListener("click", addTask);

// Enter key se bhi task add ho sake
taskInput.addEventListener("keydown", (e) => {
  if (e.key === "Enter") addTask(); });

// Event Delegation: Done aur Delete buttons ke liye
// (parent 'taskList' par ek hi listener - performance ke liye best practice)
taskList.addEventListener("click", (e) => {    const index =
e.target.dataset.index;    if (index === undefined) return;

  if (e.target.classList.contains("done-btn")) {
    tasks[index].completed = !tasks[index].completed;
  }
  if (e.target.classList.contains("delete-btn")) {
    tasks.splice(index, 1);
  }
  saveTasks();
  renderTasks();
});
// Page load hote hi purane tasks dikhayein renderTasks();
```

Practice Karne Ke Liye Baaki Project Ideas:

- Calculator: Buttons ke sath ek basic calculator banayein using event listeners aur eval-free math logic.
- Weather App: Fetch API se kisi free weather API se data la kar current weather dikhayein.
- Notes App: Todo App jaisa hi, lekin notes ke liye textarea aur edit functionality ke sath.
- Quiz App: Array of questions/answers bana kar ek interactive quiz banayein with score tracking.
- Movie Search App: Fetch API se movie database (jaise OMDb API) se search kar ke movies dikhayein.
- Digital Clock: setInterval() use karke real-time clock banayein jo har second update ho.

— End of Part 3: JavaScript —

Chapter # 4 Git Version Control

Git ek distributed version control system hai jo code changes ko track karta hai, taake developers apna code history save kar sakein, purane versions par wapis ja sakein, aur team ke sath mil kar kaam kar sakein bina ek dusre ka code overwrite kiye.

Module 1: Getting Started with Git

Git Installation

Git ko official website (git-scm.com) se download karke install kiya jata hai. Windows, Mac, Linux teenon par install hota hai, aur install hone ke baad terminal/command prompt mein 'git' command available ho jati hai.

Code Example:

```
git --version
```

Output / Result: Terminal par installed Git ka version number show hoga, jaise 'git version 2.43.0'.

Real-World Example: Har developer ke laptop par sabse pehla setup step Git install karna hota hai, chahe woh kisi bhi company mein kaam kare.

Git Config

Git config se apna naam aur email set karte hain jo har commit ke sath attach hota hai, taake pata chale konsa change kisne kiya.

Code Example:

```
git config --global user.name "Ali Khan" git  
config --global user.email "ali@example.com"
```

Output / Result: Ab har commit is naam aur email ke sath record hoga.

Real-World Example: Company projects mein commit history se pata chalta hai kis developer ne konsa feature ya bug fix kiya - ye config isi ke liye zaroori hai.

git init git init kisi folder ko Git repository bana deta hai - ek chupi hui .git folder create hoti hai jo poori history store karti hai.

Code Example:

```
mkdir my-project  
cd my-project  
git init
```

Output / Result: 'Initialized empty Git repository' message aayega aur .git folder ban jayega.

Real-World Example: Naya project shuru karte waqt sabse pehla step git init hota hai taake day-1 se hi version control mile.

Module 2: Basic Git Commands

git status

Ye batata hai kaunse files modify hui hain, kaunsi staged hain, aur kaunsi untracked hain - working directory ki current state dikhata hai.

Code Example:

```
git status
```

Output / Result: Modified, staged, aur untracked files ki list color-coded (red/green) format mein show hogi.

Real-World Example: Commit karne se pehle developers hamesha git status check karte hain taake pata chale kya changes ja rahe hain.

git add

Ye changes ko 'staging area' mein le jata hai - matlab agla commit mein kaunsi files شامل hongi ye decide karta hai.

Code Example:

```
git add index.html git add . # sab files ek  
sath add karne ke liye
```

Output / Result: Files staged ho jati hain, git status mein green color mein dikhengi.

Real-World Example: Bade projects mein sirf relevant files add ki jati hain taake unrelated changes accidentally commit na hon.

git commit

Commit ek 'save point' banata hai staged changes ka, ek message ke sath jo batata hai kya change hua.

Code Example:

```
git commit -m "Add homepage layout"
```

Output / Result: Commit create ho jayega ek unique hash ke sath, jaise (a1b2c3d).

Real-World Example: Har feature ya bug fix apne alag commit mein hota hai, taake history clear aur readable rahe.

git log

Ye repository ki poori commit history dikhata hai - kisne, kab, aur kya commit kiya.

Code Example:

```
git log git log --oneline #  
short format
```

Output / Result: Commits ki list unke hash, author, date, aur message ke sath show hogi.

Real-World Example: Bug dhoondte waqt developers git log se dekhte hain konsa recent commit issue laya hoga.

git diff

Ye batata hai files mein exactly kya lines change hui hain - old aur new content ka comparison.

Code Example:

```
git diff
```

Output / Result: Removed lines red (-) mein aur added lines green (+) mein dikhengi.

Real-World Example: Code review se pehle git diff se apne changes double-check kiye jate hain.

Module 3: Undoing Changes

git restore

Ye kisi file ke changes ko discard karke usay last committed state mein wapis le aata hai.

Code Example:

```
git restore index.html
```

Output / Result: File apni pichli committed state mein wapis chali jayegi, unsaved changes remove ho jayenge.

Real-World Example: Galti se file mein kuch break ho jaye to git restore se turant purani working state par wapis aa sakte hain.

git reset

Ye commits ko undo karta hai - staging area ya commit history ko peeche le jata hai, teen modes: --soft, --mixed, -hard.

Code Example:

```
git reset --soft HEAD~1 # last commit undo, changes staged rehte hain
git reset --hard HEAD~1 # sab kuch permanently discard
```

Output / Result: HEAD pointer aur history specified commit tak wapis chali jayegi.

Real-World Example: Galat commit push hone se pehle git reset se usay remove kiya ja sakta hai.

Module 4: Branching & Merging

Branches

Branch ek independent line of development hai - main code (usually 'main' branch) ko affect kiye bina naye features par kaam karne deta hai.

Code Example:

```
git branch feature-login git
checkout feature-login # ya
ek sath: git checkout -b
feature-login
```

Output / Result: Naya branch ban jayega aur aap us par switch ho jayenge.

Real-World Example: Team mein har developer apne feature ke liye alag branch banata hai taake ek dusre ka kaam disturb na ho.

Merge

Merge do branches ke changes ko combine karta hai - jaise feature branch ka kaam main branch mein le aana.

Code Example:

```
git checkout main git
merge feature-login
```

Output / Result: Feature branch ke commits main branch mein add ho jayenge.

Real-World Example: Feature complete hone ke baad usay main branch mein merge karke live product mein shamil kiya jata hai.

Rebase (Basic)

Rebase branch ki history ko dusre branch ke upar 'replay' karta hai, jisse ek clean, linear history milti hai (merge commits ke bina).

Code Example:

```
git checkout feature-login git
rebase main
```

Output / Result: feature-login ke commits main ke latest commits ke baad reapply ho jayenge.

Real-World Example: Kai teams pull request se pehle rebase karti hain taake commit history saaf aur linear rahe.

Module 5: Stash & Tags

Stash

Stash incomplete changes ko temporarily 'side mein rakh' deta hai bina commit kiye, taake aap kisi aur kaam par switch kar saken.

Code Example:

```
git stash
# baad mein wapis lane ke liye:
git stash pop
```

Output / Result: Working directory clean ho jayegi, changes stash list mein save ho jayenge.

Real-World Example: Achanak urgent bug fix karna ho aur current kaam incomplete ho to stash se turant switch kiya ja sakta hai.

Tags

Tags kisi specific commit ko permanent label dete hain, usually version releases mark karne ke liye (jaise v1.0.0).

Code Example:

```
git tag v1.0.0 git
push origin v1.0.0
```

Output / Result: Us commit par ek permanent 'v1.0.0' tag lag jayega.

Real-World Example: Software releases (v1.0, v2.0) ko track karne ke liye tags industry standard hain.

Chapter # 5 : GitHub

GitHub ek cloud platform hai jahan Git repositories host hoti hain, jisse code online store hota hai, team collaboration aasan hoti hai, aur open-source projects duniya bhar ke developers ke sath share kiye ja sakte hain.

Module 1: GitHub Basics

GitHub Introduction

GitHub par developers apni Git repositories online host karte hain, jahan code safe rehta hai aur kahin se bhi access ho sakta hai.

Code Example:

```
# GitHub par account banane ke baad #  
repository ka URL kuch is tarah hota hai:  
https://github.com/username/repo-name
```

Output / Result: Aapki repository ka ek public/private online page ban jata hai.

Real-World Example: Almost har company apna source code GitHub (ya GitLab/Bitbucket) par store karti hai.

Create Repository

GitHub par 'New Repository' button se naya repo banaya jata hai, jisme naam, description, aur visibility (public/private) set hoti hai.

Code Example:

```
# GitHub UI se 'New' click karke:  
# Repository name: my-portfolio  
# Public / Private select karein  
# Create Repository
```

Output / Result: Ek empty ya README ke sath repository ban jayegi apne GitHub account par.

Real-World Example: Naye project ka pehla step usually GitHub par repository create karna hota hai taake code turant online store ho sake.

Clone

Clone GitHub par mojud repository ki poori copy apne computer par download karta hai, poori history ke sath.

Code Example:

```
git clone https://github.com/username/repo-name.git
```

Output / Result: Repository ki poori copy ek naye folder mein download ho jayegi.

Real-World Example: Naye developer team mein join karte hi sabse pehle project repository clone karta hai.

Module 2: Syncing Code

Push

Push local commits ko GitHub (remote repository) par upload karta hai.

Code Example:

```
git push origin main
```

Output / Result: Local commits GitHub par visible ho jayenge.

Real-World Example: Din ke end mein developers apna kaam push karte hain taake progress cloud par safe rahe.

Pull

Pull GitHub se latest changes download karke local repository ko update karta hai (fetch + merge).

Code Example:

```
git pull origin main
```

Output / Result: Local code GitHub ke latest version ke barabar ho jayega.

Real-World Example: Kaam shuru karne se pehle hamesha pull karke latest team changes le liye jate hain, taake conflicts kam hon.

Module 3: Collaboration

Fork

Fork kisi dusre ke repository ki apni khud ki copy GitHub account mein bana deta hai, jisse aap uska code apne tareeke se edit kar sakte hain.

Code Example:

```
# GitHub UI par 'Fork' button click karein #  
Fir apni copy clone karein:  
git clone https://github.com/your-username/forked-repo.git
```

Output / Result: Aapke account mein us project ki independent copy ban jayegi.

Real-World Example: Open-source contribution ka pehla step usually project ko fork karna hota hai.

Pull Request

Pull Request (PR) ek request hai ke aapke branch/fork ke changes original repository mein merge kiye jayein, review ke baad.

Code Example:

```
# Apna branch push karke:  
git push origin feature-login  
# Fir GitHub par 'Compare & Pull Request' click karein
```

Output / Result: Ek PR page banega jahan team code review karke approve/reject kar sakti hai.

Real-World Example: Company projects mein bina PR approval ke koi bhi code main branch mein merge nahi hota.

Issues

Issues bugs, feature requests, ya tasks track karne ka tareeka hai - team members discuss kar sakte hain aur progress track kar sakte hain.

Code Example:

```
# GitHub repo mein 'Issues' tab -> 'New Issue'  
# Title: "Login button not working on mobile" #  
Description likh kar submit karein
```

Output / Result: Ek trackable issue ban jayega jisme comments aur status (open/closed) hota hai.

Real-World Example: Bug tracking aur project management ke liye teams GitHub Issues use karti hain.

Module 4: Documentation & Hosting

README

README.md ek markdown file hai jo repository ke main page par dikhti hai - project ka intro, setup instructions, aur usage batati hai.

Code Example:

```
# My Project

## Description
Ye ek todo app hai React se bani hui.

## Installation
`npm install`
`npm run dev`
`npm run dev`
```

Output / Result: Repository ke homepage par ye formatted documentation show hogi.

Real-World Example: Achi README file se new developer 5 minute mein samajh jata hai project kaise run karein.

GitHub Pages

GitHub Pages free static website hosting deta hai directly repository se - HTML/CSS/JS ya React build ko live website bana sakte hain.

Code Example:

```
# Settings -> Pages -> Source: main branch #
URL milega: https://username.github.io/repo-name/
```

Output / Result: Aapka project bina extra hosting cost ke live website ban jayega.

Real-World Example: Portfolio websites aur small projects free mein GitHub Pages par host kiye jate hain.

Chapter # 6 : npm & Node.js

npm (Node Package Manager) aur Node.js modern JavaScript development ki backbone hain - Node.js JavaScript ko browser ke bahar chalata hai, aur npm hazaron ready-made packages install karne deta hai taake pura code khud se na likhna pare.

Module 1: Node.js & npm Basics

Node.js

Node.js ek JavaScript runtime hai jo JS code ko browser ke bahar (server/terminal par) chalane deta hai. Ye backend development aur build tools (jaise Vite, Webpack) ki foundation hai.

Code Example:

```
node --version node script.js    # koi bhi JS file
run karne ke liye
```

Output / Result: Terminal par Node ka version show hoga, ya script.js run ho jayegi.

Real-World Example: React, Vite, aur almost saare modern frontend tools Node.js par hi chalte hain. [npm](#)

npm Node.js ke sath aata hai aur JavaScript packages install/manage karne ka standard tool hai - duniya ka sabse bara package registry.

Code Example:

```
npm --version npm
install react
```

Output / Result: npm ka version dikhega, aur react package project mein install ho jayega.

Real-World Example: Har React/Node project npm (ya yarn/pnpm) se apni dependencies manage karta hai.

Module 2: package.json Deep Dive

package.json

package.json project ka 'identity card' hai - project ka naam, version, dependencies, aur scripts sab isi file mein define hote hain.

Code Example:

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": { "dev": "vite" },
  "dependencies": { "react": "^18.2.0" }
}
```

Output / Result: Ye file project ka blueprint hoti hai jo bataye kya install karna hai aur kaise run karna hai.

Real-World Example: Naya project clone karne ke baad sabse pehle 'npm install' package.json parh kar sab dependencies install karta hai.

Dependencies

Dependencies woh packages hain jo app ko production mein chalne ke liye zaroori hain, jaise react, axios.

Code Example:

```
npm install axios --save
```

Output / Result: axios package.json ke 'dependencies' section mein add ho jayega.

Real-World Example: React, Redux, Axios jaisi libraries jo live app mein use hoti hain wo dependencies mein hoti hain.

Dev Dependencies

Dev Dependencies sirf development ke waqt zaroori hote hain (jaise testing tools, linters) - final production build mein shamil nahi hote.

Code Example:

```
npm install eslint --save-dev
```

Output / Result: eslint package.json ke 'devDependencies' section mein add ho jayega.

Real-World Example: Testing libraries (Jest) aur build tools sirf devDependencies mein rakhe jate hain taake final app ka size na barhe.

Scripts

Scripts custom commands hote hain jo package.json mein define ho kar 'npm run' se chalayate jate hain, jaise dev server start karna ya build banana.

Code Example:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build"  
}
```

```
# Terminal mein:  
npm run dev
```

Output / Result: Vite dev server start ho jayega jo browser mein live project dikhayega.

Real-World Example: Har React project mein 'npm run dev' aur 'npm run build' sabse zyada use hone wale commands hain.

Module 3: Managing Packages

Install/Uninstall Packages npm install se packages add hoti hain, npm uninstall se remove hoti hain - specific version bhi install ki ja sakti hai.

Code Example:

```
npm install lodash npm  
uninstall lodash  
npm install react@18.2.0 # specific version
```

Output / Result: lodash node_modules folder mein install/remove ho jayega.

Real-World Example: Project mein naya feature ke liye library chahiye ho to seedha npm install se add kar liya jata hai, code khud likhne ki zaroorat nahi.

Chapter # 7: Vite

Vite ek modern, super-fast build tool hai jo React/Vue projects setup karne aur develop karne ke liye use hota hai - purane Create React App se kaafi zyada tez hai.

Module 1: Getting Started with Vite

Installation

Vite se naya React project banane ke liye ek hi command chahiye - ye poora project structure, config, aur dependencies list khud generate kar deta hai.

Code Example:

```
npm create vite@latest my-app -- --template react
cd my-app npm install
```

Output / Result: Ek naya React project folder ban jayega tamam zaroori files ke sath.

Real-World Example: 2024 ke baad se almost har naya React project Vite se hi start hota hai, Create React App ki jagah.

Project Structure

Vite project mein important files: index.html (entry point), src/main.jsx (React entry), src/App.jsx (main component), vite.config.js (settings).

Code Example:

```
my-app/
index.html
vite.config.js
src/
main.jsx
App.jsx
assets/
```

Output / Result: Ye clean, minimal structure milta hai jo samajhna aasan hai.

Real-World Example: Vite ka simple structure beginners ke liye jaldi samajh mein aata hai.

Module 2: Development & Build

Dev Server

Vite ka dev server instant start hota hai aur 'Hot Module Replacement' deta hai - code save karte hi browser bina full reload ke turant update ho jata hai.

Code Example:

```
npm run dev
```

Output / Result: Terminal mein ek local URL milega, jaise http://localhost:5173

Real-World Example: Development ke waqt changes turant, bina delay ke browser mein dikhte hain, jo productivity barhata hai.

Build

Build command production ke liye optimized, minified files banata hai jo real server par deploy hoti hain.

Code Example:

```
npm run build
```

Output / Result: Ek 'dist' folder banega jisme optimized HTML, CSS, JS files hongi.

Real-World Example: Deployment se pehle har project ko build karna zaroori hota hai taake fast aur optimized version live jaye.

Environment Variables

Environment variables sensitive ya environment-specific values (jaise API keys) store karte hain, .env file mein, VITE_ prefix ke sath.

Code Example:

```
# .env file
VITE_API_URL=https://api.example.com

// code mein use:
console.log(import.meta.env.VITE_API_URL)
```

Output / Result: API URL code mein dynamically available ho jayega, hardcode kiye bina.

Real-World Example: Development aur production mein alag API URLs use karne ke liye environment variables zaroori hain.

Chapter # 8 : React.js

React.js ek JavaScript library hai jo Facebook (Meta) ne banayi, jo interactive, component-based User Interfaces banane ke liye use hoti hai. Ye is course ka sabse important aur bara chapter hai isliye har topic ko step-by-step, deeply explain kiya gaya hai.

Module 1: React Fundamentals

Introduction

React UI ko chote, reusable 'components' mein todta hai. Ye 'Virtual DOM' use karta hai jo real DOM ko efficiently update karta hai, isliye React apps fast hoti hain.

Code Example:

```
function App() {  
  return <h1>Hello React!</h1>;  
} export default App;
```

Output / Result: Browser mein 'Hello React!' heading show hogi.

Real-World Example: Instagram, Netflix, Facebook - sab ki UI React se banayi gayi hai.

JSX

JSX (JavaScript XML) syntax extension hai jo HTML jaisa code JavaScript ke andar likhne deta hai. Browser samajh nahi sakta, isliye Babel/Vite isay normal JS mein convert karta hai.

Code Example:

```
const name = "Ali";  
const element = <h1>Hello, {name}!</h1>;  
// curly braces {} mein JS expressions likhte hain
```

Output / Result: 'Hello, Ali!' render hoga - {name} ki value automatically insert hoti hai.

Real-World Example: JSX se HTML aur JS ek hi jagah likhna dynamic UI banana bohot aasan bana deta hai.

Components

Components React apps ke building blocks hain - independent, reusable pieces of UI (functions jo JSX return karte hain). Function components aaj kal standard hain (class components purani technique hai).

Code Example:

```
function Welcome() {  
  return <h2>Welcome to the app</h2>;  
}  
function App() {  
  return (  
    <div>  
      <Welcome />  
      <Welcome />  
    </div>  
  );  
}
```

```
}
```

Output / Result: 'Welcome to the app' heading do baar dikhegi - ek hi component reuse ho raha hai.

Real-World Example: Button, Navbar, Card jaise components ek baar bana kar poori site mein reuse kiye jate hain.

Module 2: Props & State

Props

Props (properties) parent component se child component ko data bhejne ka tareeka hain - ye read-only hain, child unhe change nahi kar sakta.

Code Example:

```
function Greeting({ name }) {  
  return <h2>Hello, {name}!</h2>;  
}  
function App() {  
  return <Greeting name="Sara" />;  
}
```

Output / Result: 'Hello, Sara!' show hoga - 'name' prop parent se child ko pass hua.

Real-World Example: Product card component ko har product ka naam, price, image props ke zariye bheja jata hai, taake ek hi component sab products dikha sake.

State

State component ka internal, changeable data hai. Jab state update hoti hai, React automatically component ko rerender kar deta hai. useState hook se state banayi jati hai.

Code Example:

```
import { useState } from 'react';  
  
function Counter() {  
  const [count, setCount] = useState(0);  return (  
    <button onClick={() => setCount(count + 1)}>  
      Count: {count}  
    </button>  
  );  
}
```

Output / Result: Har click par button ka number 1 se barhta jayega (0, 1, 2...).

Real-World Example: Shopping cart mein items ka count, form ke input values - sab state se manage hote hain.

Module 3: Handling Events & Rendering

Event Handling

React mein events (click, change, submit) camelCase mein likhe jate hain (onClick, onChange), aur function pass ki jati hai jo event hone par chalti hai.

Code Example:

```
function Button() {
  function handleClick() {
    alert('Button clicked!');
  }
  return <button onClick={handleClick}>Click Me</button>;
}
```

Output / Result: Button click karne par ek alert box 'Button clicked!' show hoga.

Real-World Example: Form submit, button click, input typing - sab kuch event handling se hi manage hota hai.

Conditional Rendering

React mein JavaScript conditions (if, ternary operator, &&) use karke decide karte hain kya render karna hai based on kisi condition par.

Code Example:

```
function Status({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <p>Welcome back!</p> : <p>Please login.</p>}
    </div>
  );
}
```

Output / Result: Agar isLoggedIn true hai to 'Welcome back!' warna 'Please login.' dikhega.

Real-World Example: Login/logout states, loading spinners, error messages - sab conditional rendering se control hote hain.

Lists & Keys

Arrays ko .map() se loop karke multiple elements render kiye jate hain. Har item ko unique 'key' prop dena zaroori hai taake React efficiently track kar sake kaunsa item change hua.

Code Example:

```
const fruits = ['Apple', 'Banana', 'Mango'];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit, i) => (
        <li key={i}>{fruit}</li>
      ))}
    </ul>
  );
}
```

Output / Result: Ek list dikhegi: Apple, Banana, Mango - har ek alag mein.

Real-World Example: Product listings, todo items, comments sections - sab data arrays se .map() ke zariye render hote hain.

Module 4: Forms in React

Forms

React mein forms user input lete hain - lekin normal HTML form ki tarah page reload nahi karni chahiye, isliye onSubmit par preventDefault() call karte hain.

Code Example:

```
function ContactForm() {
  function handleSubmit(e) {
    e.preventDefault();
    alert('Form submitted!');
  }
  return (
    <form onSubmit={handleSubmit}>
      <input type="text" placeholder="Your name" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Output / Result: Submit karne par page reload nahi hoga, sirf alert show hoga.

Real-World Example: Login forms, signup forms, contact forms - sab is pattern par bante hain.

Controlled Components

Controlled component woh input hai jiski value React state se control hoti hai - input ki har keystroke state update karti hai, aur state hi input ki value decide karti hai.

Code Example:

```
function NameInput() {
  const [name, setName] = useState('');
  return (
    <input
      value={name}
      onChange={e => setName(e.target.value)}
    />
  );
}
```

Output / Result: Jo bhi type karenge woh state mein save hoga aur input mein reflect hoga - React input ko fully control karta hai.

Real-World Example: Real-time validation (jaise password strength checker) sirf controlled components se possible hai.

Module 5: React Hooks

useState

useState kisi component mein state variable banane ka basic hook hai. Ye ek array return karta hai: current value aur usay update karne wala function.

Code Example:

```
const [count, setCount] = useState(0); setCount(count + 1);
```

Output / Result: count value update hoti hai aur component re-render hota hai naye value ke sath.

Real-World Example: Almost har interactive React component (counters, toggles, forms) useState use karta hai.

useEffect

useEffect 'side effects' handle karta hai - jaise API call karna, subscriptions, ya DOM manipulation - jo render ke baad chalna chahiye. Dependency array batati hai ye kab dobara chale.

Code Example:

```
useEffect(() => {  
  console.log('Component mounted or updated'); return () =>  
  console.log('Cleanup');  
}, [count]); // sirf count change hone par chalega
```

Output / Result: Har baar jab count change hoga, effect dobara chalega aur cleanup function pehle call hoga.

Real-World Example: Page load hote hi API se data fetch karna (jaise product list load karna) useEffect se hota hai.

useRef

useRef ek mutable value store karta hai jo re-render nahi karati, aur DOM elements ko directly access karne ke liye bhi use hoti hai (jaise input par focus karna).

Code Example:

```
const inputRef = useRef(null);  
function focusInput() {  
  inputRef.current.focus();  
}  
  
<input ref={inputRef} />
```

Output / Result: Button click karne par input field automatically focus ho jayegi.

Real-World Example: Video players, form focus management, aur previous value track karne ke liye useRef use hota hai.

useMemo

useMemo kisi expensive calculation ka result 'memoize' (cache) karta hai, taake har render par dobara calculate na ho jab tak dependencies change na hon.

Code Example:

```
const total = useMemo(() => {  
  return items.reduce((sum, item) => sum + item.price, 0); },  
[items]);
```

Output / Result: 'total' sirf tab dobara calculate hoga jab 'items' change ho, warna cached value use hogi.

Real-World Example: Large lists ki filtering/sorting jaise heavy calculations useMemo se optimize ki jati hain.

useCallback

useCallback kisi function ki 'memoized' version return karta hai - taake wo function har render par naya na bane, jo child components ki unnecessary re-renders rok sakta hai.

Code Example:

```
const handleClick = useCallback(() => {  
  console.log('Clicked'); }, []);
```

Output / Result: handleClick function reference stable rahega across renders (dependencies change na hon to).

Real-World Example: Large lists mein har item ko function pass karte waqt performance ke liye useCallback use hota hai.

useContext

useContext kisi Context ki value ko directly access karne deta hai bina props ko har level se manually pass kiye ('prop drilling' avoid karta hai).

Code Example:

```
const theme = useContext(ThemeContext); return <div style={{ background: theme.bg }}>Hello</div>;
```

Output / Result: Component ko global context ki value directly mil jayegi, kisi prop drilling ke bina.

Real-World Example: Theme (dark/light mode) aur logged-in user ki info poore app mein useContext se share ki jati hai.

useReducer

useReducer complex state logic manage karne ka hook hai - multiple related state updates ek 'reducer function' ke zariye handle hoti hain, Redux jaisa pattern.

Code Example:

```
function reducer(state, action) {
  if (action.type === 'increment') return { count: state.count + 1 };
  return state;
}
const [state, dispatch] = useReducer(reducer, { count: 0 }); dispatch({
  type: 'increment' });
```

Output / Result: state.count 1 se barh jayega jab dispatch call hoga.

Real-World Example: Shopping cart jaisi complex state (add, remove, update quantity) useReducer se cleanly manage hoti hai.

Custom Hooks

Custom Hooks apne khud ke reusable hooks hain jo built-in hooks (useState, useEffect) ko combine karke banaye jate hain - naam hamesha 'use' se start hota hai.

Code Example:

```
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => {
    const handler = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handler);
    return () => window.removeEventListener('resize', handler);
  }, []);
  return width;
}
```

Output / Result: Ye hook current window width return karega jo resize hone par automatically update hoga.

Real-World Example: useFetch, useAuth, useLocalStorage jaise custom hooks har bare project mein reusable logic ke liye banaye jate hain.

Module 6: Component Patterns

Component Lifecycle

Function components mein lifecycle (mount, update, unmount) useEffect se manage hoti hai - empty array [] se sirf mount par, dependency array se update par, aur cleanup return se unmount par chalta hai.

Code Example:

```
useEffect(() => {  
  console.log('Mounted');  
  return () => console.log('Unmounted');  
}, []);
```

Output / Result: Component screen par aane par 'Mounted' aur remove hone par 'Unmounted' print hoga.

Real-World Example: Event listeners ya timers ko component unmount hone par clean karna zaroori hai warna memory leaks hoti hain.

Lifting State Up

Jab do sibling components ko same data share karna ho, to state ko unke common parent mein 'upar utha' dete hain, aur props ke zariye neeche pass karte hain.

Code Example:

```
function Parent() {  
  const [value, setValue] = useState('');  return (  
    <>  
      <Input value={value} setValue={setValue} />  
      <Display value={value} />  
    </>  
  );  
}
```

Output / Result: Input mein type karne par Display component turant update value dikhayega.

Real-World Example: Filter aur result list jaise related components ke beech state lifting se sync rakha jata hai.

Composition

Composition components ko 'children' prop ke zariye combine karne ka tareeka hai - inheritance ki bajaye React composition prefer karta hai.

Code Example:

```
function Card({ children }) {  
  return <div className="card">{children}</div>;  
}  
  
<Card>  
  <h2>Title</h2>  
  <p>Some content</p> </Card>
```

Output / Result: Card component apne andar diya gaya koi bhi content (h2, p) wrap kar ke dikhayega.

Real-World Example: Modal, Card, Layout components ismein 'children' composition pattern se flexible banaye jate hain.

Portals

Portals kisi component ko DOM tree ke bahar, kisi alag DOM node mein render karne dete hain - lekin React tree mein wo wahin rehta hai (events waghera).

Code Example:

```
import { createPortal } from 'react-dom';

function Modal({ children }) {
  return createPortal(
    <div className="modal">{children}</div>,
    document.getElementById('modal-root')
  );
}
```

Output / Result: Modal content 'modal-root' div mein render hoga, chahe component tree mein kahin bhi ho.

Real-World Example: Modals, tooltips, aur dropdowns ko z-index issues se bachane ke liye portals use hote hain.

Error Boundaries

Error Boundaries class components hain jo apne child components mein aane wale JavaScript errors ko catch karte hain aur poori app crash hone se bachate hain, fallback UI dikha kar.

Code Example:

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static
  getDerivedStateFromError() {
    return {
      hasError: true
    };
  }
  render() {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
```

Output / Result: Agar child component mein error aaye to poori app crash hone ki bajaye 'Something went wrong.' dikhega.

Real-World Example: Production apps mein critical sections ko error boundaries se wrap karke user experience protect ki jati hai.

Module 7: Performance & Code Splitting

Lazy Loading

React.lazy() se components ko sirf tab load karte hain jab unki zaroorat ho, initial bundle size chota rakhne ke liye.

Code Example:

```
const Dashboard = React.lazy(() => import('./Dashboard'));
```

Output / Result: Dashboard component ka code sirf tab download hoga jab wo actually render ho.

Real-World Example: Bade dashboards ya admin panels ka code lazy load karke app ka initial loading time kam kiya jata hai.

Suspense

Suspense lazy-loaded components ke load hone tak ek fallback UI (jaise loading spinner) dikhata hai.

Code Example:

```
<Suspense fallback={<p>Loading...</p>}>
  <Dashboard />
</Suspense>
```

Output / Result: Jab tak Dashboard load ho raha hai, 'Loading...' text dikhega.

Real-World Example: Lazy loading ke sath Suspense hamesha use hota hai taake user ko blank screen na dikhe.

Code Splitting

Code Splitting bare app ke JavaScript bundle ko chote chote pieces mein tor deta hai, jo zaroorat ke hisab se load hote hain, initial load time kam karta hai.

Code Example:

```
// Route-based code splitting example:
const Home = React.lazy(() => import('./pages/Home')); const
About = React.lazy(() => import('./pages/About'));
```

Output / Result: Har page ka code alag chunk mein hoga, sirf visited page ka code load hoga.

Real-World Example: Bare e-commerce sites route-based code splitting se page load speed behtar rakhte hain.

Performance Optimization

React performance behtar banane ke techniques: React.memo se unnecessary re-renders rokna, keys sahi use karna, aur heavy calculations useMemo/useCallback se optimize karna.

Code Example:

```
const ProductCard = React.memo(function ProductCard({ product }) {
  return <div>{product.name}</div>; });
```

Output / Result: ProductCard sirf tab re-render hoga jab uska 'product' prop actually change ho.

Real-World Example: Large product listing pages mein React.memo se hundreds of cards ka unnecessary re-render bacha jata hai.

Module 8: Advanced React Patterns & Modern Hooks

Fragments

Fragments (<>...</> ya <React.Fragment>) multiple elements ko group karte hain bina extra DOM node (jaise ek faltu <div>) add kiye - HTML structure clean rehta hai.

Code Example:

```
function List() {  return (
  <>
    <li>Item 1</li>
    <li>Item 2</li>
  </>
);
}
```

Output / Result: Dono `<li>` render hongy bina kisi extra wrapping `<div>` ke DOM mein.

Real-World Example: Table rows (`<tr>/<td>`) jaise jagah jahan extra div HTML structure break kar deta, wahan Fragments zaroori hain.

forwardRef & useImperativeHandle forwardRef parent ko child component ke andar kisi DOM element (ya function) tak ref access dene deta hai. useImperativeHandle iske sath mil kar decide karta hai parent ko exactly kya expose karna hai.

Code Example:

```
const CustomInput = forwardRef((props, ref) => {
  const inputRef = useRef();
  useImperativeHandle(ref, () => ({      focus: ()
=> inputRef.current.focus()
  }));
  return <input ref={inputRef} {...props} />;
});

// Parent: const ref =
useRef(); <CustomInput
ref={ref} />;
ref.current.focus();
```

Output / Result: Parent component child ke internal input ko 'focus()' method ke zariye control kar sakega.

Real-World Example: Reusable component libraries (jaise custom Modal ya Input) mein parent ko limited control dene ke liye ye pattern use hota hai.

useLayoutEffect

useLayoutEffect, useEffect jaisa hi hai lekin browser paint karne se PEHLE synchronously chalta hai - jab DOM measurements (jaise element ki height) turant chahiye hon, tab use hota hai taake flicker na ho.

Code Example:

```
useLayoutEffect(() => {
  const height = divRef.current.getBoundingClientRect().height;
  setBoxHeight(height); }, []);
```

Output / Result: Element ki height paint hone se pehle hi calculate ho kar state mein set ho jayegi - koi visual flicker nahi hoga.

Real-World Example: Tooltips ya dropdowns ko screen ke andar sahi position par rakhne ke liye (auto-flip logic) useLayoutEffect use hota hai.

useTransition & useDeferredValue

Ye React 18 ke naye hooks hain jo UI ko responsive rakhte hain jab heavy updates ho rahi hon. useTransition kisi update ko 'low priority' mark karta hai, useDeferredValue kisi value ka 'delayed' version deta hai taake typing jaisi urgent cheezein lag na khayen.

Code Example:

```
const [isPending, startTransition] = useTransition();

function handleChange(e) {
  setQuery(e.target.value); // urgent
  startTransition(() => {
    setResults(filterBigList(e.target.value)); // less urgent
  });
}
```

Output / Result: Typing turant smooth rahegi, bhale hi bara filtering result thoda der se update ho.

Real-World Example: Search-as-you-type jahan hazaron items filter ho rahe hon (jaise large product catalog), wahan typing lag rokne ke liye useTransition use hota hai.

Higher-Order Components (HOC)

HOC ek function hai jo ek component leta hai aur usay extra functionality ke sath wrap kar ke naya component return karta hai - logic reuse karne ka purana lekin still-used pattern.

Code Example:

```
function withLoading(Component) {
  return function WrappedComponent({ isLoading, ...props }) {
    if (isLoading) return <p>Loading...</p>;    return
    <Component {...props} />;
  };
}
const ProductListWithLoading = withLoading(ProductList);
```

Output / Result: ProductListWithLoading automatically loading state handle karega bina ProductList ke andar logic dohraye.

Real-World Example: Authentication check (withAuth) ya loading states multiple components mein wrap karne ke liye HOC pattern use hota hai.

Render Props

Render Props pattern mein ek component apni state/logic ek function (jo prop ke through pass hoti hai) ko deta hai, jo decide karta hai kya render karna hai - logic aur UI ko separate karta hai.

Code Example:

```
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return (
    <div onMouseMove={(e) => setPos({ x: e.clientX, y: e.clientY })}>
      {render(pos)}
    </div>
  );
}
<MouseTracker render={(pos) => <p>X: {pos.x}, Y: {pos.y}</p>} />
```

Output / Result: Mouse move karne par x,y coordinates live update hote dikhenge.

Real-World Example: Aaj kal iski jagah zyada tar Custom Hooks use hote hain, lekin kai libraries (jaise Formik) is pattern par based hain, isliye samajhna zaroori hai.

React.StrictMode

StrictMode development mein extra checks aur warnings deta hai (jaise unsafe lifecycle methods, deprecated APIs) - production build mein iska koi effect nahi hota, sirf development mein components ko intentionally double-render karta hai bugs pakadne ke liye.

Code Example:

```
<React.StrictMode>
  <App />
</React.StrictMode>
```

Output / Result: Development mein components thodi extra checks ke sath render honge, console mein potential issues ke warnings aa sakte hain.

Real-World Example: Har naya Vite/CRA project by default StrictMode ke sath aata hai - best practice hai isay hata na jaye.

PropTypes

PropTypes runtime mein check karta hai ke component ko sahi type ke props mil rahe hain ya nahi (jaise string ki jagah number to nahi bheja gaya) - chota, lightweight type-checking tool.

Code Example:

```
import PropTypes from 'prop-types';

function Greeting({ name, age }) {
  return <p>{name} is {age} years old</p>;
}

Greeting.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number };

```

Output / Result: Agar galat type ka prop pass ho (jaise age='abc' string) to console mein development warning aayegi.

Real-World Example: TypeScript na use karne wale projects mein PropTypes basic type-safety ke liye use hoti hai.

TypeScript with React (Introduction)

TypeScript, JavaScript ka superset hai jo static typing deta hai - components, props, aur state ki types define karke bugs compile-time par hi pakre jate hain, runtime tak pahunchne se pehle.

Code Example:

```
type GreetingProps = {
  name: string;   age?:
  number; // optional };
function Greeting({ name,
  age }: GreetingProps) {
  return <p>{name} is {age ??
  'unknown'} years old</p>;
}
```

Output / Result: Agar koi galat type ka prop pass kare (jaise number ki jagah object) to code compile hi nahi hoga, error turant editor mein dikh jayega.

Real-World Example: Almost har professional/enterprise React project (Meta, Airbnb, Stripe) aaj kal TypeScript use karta hai - job market mein ye ab expected skill hai.

Chapter # 9 : React Router

React Router library hai jo React apps mein multiple pages (routes) banane deti hai bina page reload kiye - is se 'Single Page Application' (SPA) banti hai jo fast aur smooth feel deti hai.

Module 1: Setting Up React Router

Installation

React Router ek separate npm package hai jo project mein install karna padta hai.

Code Example:

```
npm install react-router-dom
```

Output / Result: react-router-dom package install ho jayega, routing components use karne ke liye ready.

Real-World Example: Multi-page React app (Home, About, Contact) banane ke liye ye sabse pehla step hai.

BrowserRouter

BrowserRouter poore app ko wrap karta hai aur browser ki history API use karke routing enable karta hai.

Code Example:

```
import { BrowserRouter } from 'react-router-dom';

<BrowserRouter>
  <App />
</BrowserRouter>
```

Output / Result: Ab poori app ke andar routing features (Routes, Link waghera) use ho sakti hain.

Real-World Example: main.jsx file mein App ko hamesha BrowserRouter se wrap kiya jata hai.

Routes

Routes ek container hai jiske andar sab individual Route definitions rehti hain - ye decide karta hai konsa Route match karke render hoga.

Code Example:

```
import { Routes, Route } from 'react-router-dom';

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} /> </Routes>
```

Output / Result: URL ke mutabiq sahi component render hoga - '/' par Home, '/about' par About.

Real-World Example: Har React website mein pages define karne ka standard tareeka Routes hai.

Route

Route ek single path aur uska corresponding component define karta hai using 'path' aur 'element' props.

Code Example:

```
<Route path="/contact" element={<Contact />} />
```

Output / Result: Jab user '/contact' par jaye, Contact component render hoga.

Real-World Example: Har page (Home, Products, Contact) ka apna alag Route hota hai.

Module 2: Navigation

Link

Link component normal `<a>` tag ki jagah use hota hai - page reload kiye bina navigation karta hai, isliye app fast rehta hai.

Code Example:

```
import { Link } from 'react-router-dom';

<Link to="/about">About Us</Link>
```

Output / Result: Click karne par bina page reload URL `/about` par change hoga.

Real-World Example: Navbar ke sab menu items Link se banaye jate hain, `<a href>` se nahi.

NavLink

NavLink, Link jaisa hi hai lekin active route ko automatically special class/style de deta hai, jo current page highlight karne ke liye useful hai.

Code Example:

```
<NavLink to="/about" className={({ isActive }) => isActive ? 'active-link' : ''}> About
</NavLink>
```

Output / Result: Jab user About page par ho, 'About' link ko 'active-link' class mil jayegi.

Real-World Example: Navbar mein current active page ko highlight karne ke liye NavLink standard hai.

useNavigate useNavigate hook programmatically (code se) navigate karne deta hai, jaise form submit hone ke baad redirect karna.

Code Example:

```
import { useNavigate } from 'react-router-dom';
const navigate =
useNavigate(); function
handleLogin() {
  navigate('/dashboard');
}
```

Output / Result: Login ke baad user automatically `/dashboard` route par redirect ho jayega.

Real-World Example: Login/signup ke baad user ko dashboard par bhejna is hook se hota hai.

useParams useParams URL mein मौजूद dynamic values (jaise product id) ko access karne deta hai.

Code Example:

```
// Route: <Route path="/product/:id" element={<Product />} />

import { useParams } from 'react-router-dom'; const
{ id } = useParams();
```

Output / Result: Agar URL `/product/5` hai to 'id' ki value '5' milegi.

Real-World Example: Product detail page, user profile page - sab dynamic URL params se specific data fetch karte hain.

Module 3: Advanced Routing

Nested Routes

Nested Routes se ek route ke andar sub-routes define kar sakte hain, jaise dashboard ke andar profile aur settings pages.

Code Example:

```
<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} /> </Route>
```

Output / Result: '/dashboard/profile' par Dashboard ke andar Profile component render hoga.

Real-World Example: Admin panels mein sidebar ke saath nested pages is pattern se banaye jate hain.

Layout Routes

Layout Route ek common structure (jaise Navbar + Footer) provide karta hai jisme <Outlet /> se child routes render hote hain.

Code Example:

```
function Layout() {  return (
  <>
    <Navbar />
    <Outlet />
    <Footer />
  </>
);
}
```

Output / Result: Har page par Navbar aur Footer consistent rahenge, sirf beech ka content change hoga.

Real-World Example: Almost har website mein common layout (header/footer) sab pages mein reuse hota hai.

Protected Routes

Protected Routes woh routes hain jo sirf logged-in users access kar sakein - agar user login nahi hai to login page par redirect ho jata hai.

Code Example:

```
function ProtectedRoute({ children }) {
  const isLoggedIn = Boolean(localStorage.getItem('token'));
  return isLoggedIn ? children : <Navigate to="/login" />;
}
```

Output / Result: Bina login token ke user Dashboard route access karne ki koshish kare to Login page par bhej diya jayega.

Real-World Example: Dashboard, profile, checkout jaisi pages hamesha protected routes hoti hain.

404 Page

404 Page tab dikhta hai jab user kisi aisi URL par jaye jo define nahi ki gayi - path='*' se wildcard route banate hain.

Code Example:

```
<Route path="*" element={<NotFound />} />
```

Output / Result: Koi bhi undefined URL par jane se 'Page Not Found' dikhega.

Real-World Example: Har production website mein galat/purani URL ke liye friendly 404 page hota hai.

Real apps sirf static data nahi dikhati - unhe server se live data lena hota hai. Ye chapter sikhata hai React app ko backend APIs se kaise connect karte hain.

Chapter # 10 : API Integration

Module 1: REST API

REST API ek standard tareeka hai jisse frontend aur backend HTTP ke zariye data exchange karte hain - resources (users, products) URLs ke through access hote hain.

Code Example:

```
GET    /api/products      -> saare products lao
GET    /api/products/5    -> ek specific product lao
POST   /api/products         -> naya product banao
```

Output / Result: Har request server se JSON format mein response deti hai.

Real-World Example: Almost har modern web app (Instagram, Amazon) apne backend se REST APIs ke zariye baat karti hai.

HTTP Methods

HTTP methods batate hain kya action karna hai: GET (data lena), POST (naya data banana), PUT/PATCH (update karna), DELETE (remove karna).

Code Example:

```
GET    -> Read
POST   -> Create
PUT    -> Update (poora)
PATCH -> Update (partial) DELETE -> Delete
```

Output / Result: Har method server par alag operation trigger karta hai.

Real-World Example: Todo app mein: naya todo POST se, complete mark karna PATCH se, delete karna DELETE se hota hai.

Module 2: Making Requests

Fetch API

Fetch API browser ka built-in function hai jo server se data mangwane ke liye use hota hai, koi extra library install karne ki zaroorat nahi.

Code Example:

```
fetch('https://api.example.com/products')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Output / Result: Products ka data JavaScript object/array ki shakal mein console mein aayega.

Real-World Example: Choti projects ya simple API calls ke liye Fetch API kaafi hoti hai.

Axios

Axios ek popular third-party library hai jo Fetch se zyada features deti hai - automatic JSON parsing, better error handling, request/response interceptors.

Code Example:

```
import axios from 'axios';

axios.get('https://api.example.com/products')
  .then(res => console.log(res.data));
```

Output / Result: Response data directly 'res.data' mein milta hai, .json() call karne ki zaroorat nahi.

Real-World Example: Bare production projects mein Axios zyada use hoti hai kyunke ye zyada developer-friendly hai.

Module 3: Handling Data & States

CRUD Operations

CRUD ka matlab Create, Read, Update, Delete hai - ye chaar operations har data-driven app ki foundation hain.

Code Example:

```
// Create
axios.post('/api/todos', { title: 'Learn React' });
// Read
axios.get('/api/todos');
// Update
axios.put('/api/todos/1', { done: true });
// Delete
axios.delete('/api/todos/1');
```

Output / Result: Har operation server par corresponding change karega aur response return karega.

Real-World Example: Todo app, blog, e-commerce - sab CRUD operations par based hain.

Error Handling

API calls fail bhi ho sakti hain (network issue, server error), isliye try-catch ya .catch() se errors handle karna zaroori hai taake app crash na ho.

Code Example:

```
try { const res = await axios.get('/api/products');
  setProducts(res.data); } catch (error) {
  setError('Failed to load products');
}
```

Output / Result: Agar request fail ho to app crash hone ki bajaye friendly error message dikhega.

Real-World Example: Internet slow hone par ya server down hone par user ko clear error message dikhana professional apps ki nishani hai.

Loading State

Loading state batata hai ke data abhi fetch ho raha hai - user ko spinner ya 'Loading...' text dikhaya jata hai jab tak data na aaye.

Code Example:

```
const [loading, setLoading] = useState(true);

useEffect(() => {
  axios.get('/api/products').then(res => {
    setProducts(res.data);    setLoading(false);
  });
}, []);
if (loading) return <p>Loading...</p>;
```

Output / Result: Data aane tak 'Loading...' dikhega, phir products list show hogi.

Real-World Example: Instagram feed load hote waqt jo skeleton/spinner dikhta hai wo isi loading state concept se hai.

Chapter 11 : Tailwind CSS

Tailwind CSS ek utility-first CSS framework hai jisme pehle se bani chotti chotti classes (jaise p-4, text-center) use karke fast, custom designs banaye jate hain bina alag CSS file likhe.

Module 1: Getting Started with Tailwind

Installation

Tailwind Vite/React project mein npm se install hoti hai aur config files mein setup ki jati hai.

Code Example:

```
npm install -D tailwindcss postcss autoprefixer npx  
tailwindcss init -p
```

Output / Result: tailwind.config.js aur postcss.config.js files ban jayengi.

Real-World Example: Modern startups aur agencies design speed ke liye Tailwind ko custom CSS par prefer karti hain.

Utility Classes

Utility classes chote, single-purpose classes hain (jaise 'flex', 'p-4', 'text-red-500') jo directly HTML/JCX mein use hoti hain, CSS file likhne ki zaroorat khatam.

Code Example:

```
<button className="bg-blue-500 text-white px-4 py-2 rounded">  
  Click Me  
</button>
```

Output / Result: Blue background, white text, padding, aur rounded corners wala button milega - bina custom CSS likhe.

Real-World Example: Tailwind se prototyping aur design bohot fast ho jati hai kyunke class name se hi style pata chal jati hai.

Module 2: Styling Basics

Spacing

Spacing (margin/padding) classes ek consistent numeric scale follow karti hain: p- (padding), m- (margin), jaise p-4 = 1rem padding.

Code Example:

```
<div className="m-4 p-6">Spaced Box</div>
```

Output / Result: Div ke around 1rem margin aur andar 1.5rem padding hoga.

Real-World Example: Consistent spacing scale se poori website mein uniform look milta hai.

Colors

Tailwind ka pehle se defined color palette hai (red, blue, gray waghera), har color ki 50-900 tak shades hoti hain.

Code Example:

```
<p className="text-gray-700 bg-gray-100">Text with background</p>
```

Output / Result: Text dark gray color mein, background light gray shade mein dikhega.

Real-World Example: Design consistency ke liye teams apna khud ka color palette bhi Tailwind config mein define karti hain.

Typography

Font size, weight, aur alignment classes se text style ki jati hai: text-xl, font-bold, text-center waghera.

Code Example:

```
<h1 className="text-3xl font-bold text-center">  
  Welcome  
</h1>
```

Output / Result: Bara, bold, center-aligned heading dikhega.

Real-World Example: Blog aur marketing pages mein typography classes se quickly hierarchy banayi jati hai.

Module 3: Layout with Tailwind

Flexbox

Tailwind mein flex, justify-between, items-center jaisi classes se Flexbox layouts bina custom CSS ke banayi jati hain.

Code Example:

```
<div className="flex justify-between items-center">
  <h2>Logo</h2>
  <nav>Menu</nav>
</div>
```

Output / Result: Logo left aur Menu right mein, vertically centered dikhenge.

Real-World Example: Navbars aur card layouts Tailwind mein aksar Flexbox utility classes se banaye jate hain.

Grid

grid, grid-cols-3 jaisi classes se CSS Grid layouts banayi jati hain, columns aur gaps directly class names se control hote hain.

Code Example:

```
<div className="grid grid-cols-3 gap-4">
  <div>Card 1</div>
  <div>Card 2</div>
  <div>Card 3</div> </div>
```

Output / Result: Teen equal-width columns ka grid ban jayega, har card ke darmiyan gap ke sath.

Real-World Example: Product grids aur image galleries Tailwind grid classes se seconds mein ban jati hain.

Responsive Design

Tailwind mobile-first hai - breakpoint prefixes (sm:, md:, lg:) se different screen sizes ke liye alag styles diye jate hain.

Code Example:

```
<div className="grid grid-cols-1 md:grid-cols-3">
  ...
</div>
```

Output / Result: Mobile par 1 column, medium screens (768px+) par 3 columns dikhenge.

Real-World Example: Ek hi class list se poori site responsive ban jati hai, alag media query likhne ki zaroorat nahi.

Module 4: Advanced Tailwind

Dark Mode

Tailwind ka 'dark:' prefix dark mode ke liye alag styles define karne deta hai, jo class ya system preference se toggle hoti hai.

Code Example:

```
<div className="bg-white dark:bg-gray-900 text-black dark:text-white">
  Content </div>
```

Output / Result: Normal mode mein white background, dark mode enable hone par dark background aur white text.

Real-World Example: Modern apps (Twitter, YouTube) ka dark mode toggle isi tarah implement kiya jata hai.

Custom Theme

tailwind.config.js file mein apne custom colors, fonts, spacing values define kar sakte hain jo brand identity ke mutabiq hon.

Code Example:

```
// tailwind.config.js
module.exports = {
  theme: {    extend:
  {
    colors: { brand: '#4a90e2' }
  }
}
}

// use: className="bg-brand"
```

Output / Result: 'bg-brand' class ab hamesha aapke custom blue color use karegi.

Real-World Example: Company projects apna brand color palette Tailwind config mein define karke consistent design maintain karti hain.

Chapter # 12 : Context API

Context API React ka built-in tareeka hai global data (jaise logged-in user, theme) ko poori app mein share karne ka, bina har component ko manually props pass kiye ('prop drilling' se bachne ke liye).

Module 1: Understanding Context API

Context

Context ek 'container' hai jo koi data hold karta hai jo poore component tree mein available ho sakta hai, createContext() se banaya jata hai.

Code Example:

```
import { createContext } from 'react';
export const ThemeContext =
createContext();
```

Output / Result: Ek ThemeContext object ban jayega jo Provider/Consumer ke liye use hoga.

Real-World Example: Auth info, theme, language settings - ye sab typically Context mein rakhi jati hain.

Provider

Provider component apne andar sab children ko Context ki value provide karta hai - jo bhi is Provider ke andar hoga wo value access kar sakta hai.

Code Example:

```
<ThemeContext.Provider value={{ mode: 'dark' }}>
  <App />
</ThemeContext.Provider>
```

Output / Result: App ke andar koi bhi component ab { mode: 'dark' } value access kar sakta hai.

Real-World Example: App.jsx ko usually AuthProvider aur ThemeProvider se wrap kiya jata hai.

Consumer

Consumer purana tareeka hai Context value ko read karne ka (JSX render-prop pattern se) - aaj kal iski jagah useContext hook use hota hai.

Code Example:

```
<ThemeContext.Consumer>
  {(theme) => <p>Mode: {theme.mode}</p>}
</ThemeContext.Consumer>
```

Output / Result: 'Mode: dark' text render hoga.

Real-World Example: Purane React codebases mein Consumer milta hai, naye projects hooks use karte hain.

Module 2: Using Context in Practice

[useContext](#) useContext hook Context ki value ko directly, simple tareeke se access karne deta hai bina Consumer wrapper ke.

Code Example:

```
import { useContext } from 'react';

function Header() {
  const theme = useContext(ThemeContext);
  return <h1>Mode: {theme.mode}</h1>;
}
```

Output / Result: 'Mode: dark' heading directly render hoga.

Real-World Example: Modern React apps mein useContext hi standard tareeka hai global data access karne ka.

Global State

Context ko useState ke sath combine karke poori app ke liye global, shared state banayi jati hai jo koi bhi component read/update kar sake.

Code Example:

```
function AuthProvider({ children }) {  
  const [user, setUser] = useState(null);  
  return (  
    <AuthContext.Provider value={{ user, setUser }}>  
      {children}  
    </AuthContext.Provider>  
  );  
}
```

Output / Result: Poori app mein 'user' state kahin se bhi read/update ki ja sakti hai.

Real-World Example: Logged-in user ki info (naam, email) is pattern se poore app mein share hoti hai - Navbar se lekar Profile page tak.

Chapter # 13: Redux Toolkit

Redux Toolkit (RTK) Redux ka modern, official tareeka hai state management ka - bare apps mein jab Context API kaafi na ho to Redux predictable, centralized state manage karta hai.

Module 1: Redux Toolkit Basics

Store

Store poori app ki state ka single, central container hai - saara global data yahan store hota hai.

Code Example:

```
import { configureStore } from '@reduxjs/toolkit';
export const store =
configureStore({  reducer: { cart:
cartReducer } });
```

Output / Result: Ek central store banega jo poori app ki state hold karega.

Real-World Example: Bare e-commerce apps ka cart, user, aur products data ek hi Redux store mein manage hota hai.

Slice

Slice state ke ek particular hisse (jaise 'cart') ke liye reducers aur actions ek sath define karta hai - createSlice() Redux Toolkit ka core function hai.

Code Example:

```
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: {
    addItem: (state, action) => {
      state.items.push(action.payload);
    }
  }
}); export const { addItem } =
cartSlice.actions; export default
cartSlice.reducer;
```

Output / Result: Ek 'cart' slice ban jayega jisme addItem action available hoga.

Real-World Example: Har feature (cart, auth, products) apni alag slice mein organize kiya jata hai.

Reducers

Reducers pure functions hain jo current state aur action lekar naya state return karte hain - Redux Toolkit ke andar Immer library use hoti hai jisse 'mutating' style code likh sakte hain safely.

Code Example:

```
reducers: {
  increment: (state) => { state.count += 1; },  decrement: (state) => {
state.count -= 1; }
}
```

Output / Result: increment/decrement action dispatch hone par state.count update hoga.

Real-World Example: Har state change (add to cart, remove item, update quantity) ek reducer function ke through hoti hai.

Actions

Actions events hain jo describe karte hain 'kya hua' - jaise 'addItem' ya 'removeItem'. createSlice automatically actions generate karta hai reducers se.

Code Example:

```
dispatch(addItem({ id: 1, name: 'Shoes' }));
```

Output / Result: Ye action cartSlice ke 'addItem' reducer ko trigger karega.

Real-World Example: User jab 'Add to Cart' click karta hai to background mein ek action dispatch hoti hai.

Module 2: Connecting Redux to React

Dispatch

Dispatch action ko store tak bhejne ka tareeka hai - jab dispatch hota hai to Redux corresponding reducer chala kar state update karta hai.

Code Example:

```
import { useDispatch } from 'react-redux';  
const dispatch =  
useDispatch();  
dispatch(addItem(product));
```

Output / Result: Product cart slice ki state mein add ho jayega.

Real-World Example: 'Add to Cart' button ka onClick handler dispatch call karta hai.

useSelector

useSelector store se koi specific state value read karne ke liye use hota hai - component ko us value ke change hone par automatically re-render karta hai.

Code Example:

```
import { useSelector } from 'react-redux';  
const items = useSelector((state) =>  
state.cart.items);
```

Output / Result: 'items' hamesha current cart state ke sath sync rahega.

Real-World Example: Cart icon ke upar item count dikhane ke liye useSelector se cart items ki length li jati hai.

useDispatch useDispatch hook component ko dispatch function dete hai jisse woh Redux actions trigger kar sake.

Code Example:

```
const dispatch = useDispatch();  
function handleRemove(id)  
{  
  dispatch(removeItem(id));  
}
```

Output / Result: Button click karne par specified item cart se remove ho jayega.

Real-World Example: Delete, update, add - sab user interactions Redux mein useDispatch se hoti hain.

Module 3: Async Data & RTK Query

Async Thunk

`createAsyncThunk` async operations (jaise API calls) ko Redux ke sath integrate karne deta hai, aur automatically pending/fulfilled/rejected states handle karta hai.

Code Example:

```
import { createAsyncThunk } from '@reduxjs/toolkit';

export const fetchProducts = createAsyncThunk('products/fetch', async () => {
  const res = await fetch('/api/products');
  return res.json();
});
```

Output / Result: Dispatch hone par ye API call karega aur result automatically store mein aa jayega.

Real-World Example: Product list, user profile jaisa server data fetch karne ke liye async thunks use hoti hain Redux projects mein.

RTK Query (Introduction)

RTK Query Redux Toolkit ka built-in data fetching aur caching solution hai - manual loading/error state likhne ki zaroorat khatam kar deta hai.

Code Example:

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';

export const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  endpoints: (builder) => ({
    getProducts: builder.query({ query: () => 'products' })
  })
});

export const { useGetProductsQuery } = api;
```

Output / Result: `useGetProductsQuery()` hook se directly data, loading, aur error mil jayenge, automatically cached.

Real-World Example: Naye Redux projects mein manual Axios calls ki jagah RTK Query zyada use ho rahi hai kyunke caching built-in hai.

Chapter # 14: Authentication

Authentication woh process hai jisse app confirm karti hai user kaun hai (login) aur decide karti hai wo kya access kar sakta hai. Ye chapter React apps mein secure login system banana sikhata hai.

Module 1: Authentication Basics

JWT

JWT (JSON Web Token) ek secure token hai jo login ke baad server deta hai, user ki identity prove karne ke liye - har protected request ke sath ye token bheja jata hai.

Code Example:

```
// Login ke baad server se milta hai:
```

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6I.."} }
```

Output / Result: Ye token client-side store hota hai aur future requests mein use hota hai.

Real-World Example: Almost har modern app (bina session cookies ke) JWT based authentication use karti hai, especially APIs ke liye.

Login

Login form user ka email/password leke server ko bhejta hai, server verify karke JWT token return karta hai agar credentials sahi hon.

Code Example:

```
async function handleLogin(email, password) {  
  const res = await axios.post('/api/login', { email, password });  
  localStorage.setItem('token', res.data.token);  
}
```

Output / Result: Successful login par token save ho jayega aur user authenticated ho jayega.

Real-World Example: Har app ka pehla screen usually login/signup hota hai.

Signup

Signup naya user account server par create karta hai - naam, email, password lekar backend mein save karta hai (password hashed hokar).

Code Example:

```
async function handleSignup(data) {  
  await axios.post('/api/signup', data);  
}
```

Output / Result: Naya user database mein create ho jayega, phir usually login page par redirect hota hai.

Real-World Example: E-commerce, social apps - sab pehle signup form se new users register karte hain.

Module 2: Securing the App

Protected Routes

Protected Routes sirf authenticated users ko specific pages (dashboard, profile) access karne dete hain - JWT token ki presence check karke.

Code Example:

```
function ProtectedRoute({ children }) {  
  const token = localStorage.getItem('token');  
  return token ?  
    children : <Navigate to="/login" />;  
}
```

Output / Result: Bina token ke user Dashboard access nahi kar sakega, Login page par redirect ho jayega.

Real-World Example: Banking apps, dashboards - sab sensitive pages protected routes ke peeche hoti hain.

Token Storage

JWT token ko localStorage, sessionStorage, ya secure httpOnly cookie mein store kiya ja sakta hai - har method ka apna trade-off hai (security vs simplicity).

Code Example:

```
localStorage.setItem('token', jwtToken); //  
read karne ke liye:  
const token =  
localStorage.getItem('token');
```

Output / Result: Token browser mein save ho jayega aur page refresh hone par bhi maujood rahega.

Real-World Example: httpOnly cookies zyada secure hoti hain (XSS se safe), lekin localStorage implement karna aasan hai isliye beginners projects mein zyada milta hai.

Refresh Token (Basic)

Refresh Token ek longer-lived token hai jo expired access token ko dobara naya token lene ke liye use hota hai, bina user ko dobara login kiye.

Code Example:

```
async function refreshAccessToken() {      const res = await  
  axios.post('/api/refresh', {              refreshToken:  
    localStorage.getItem('refreshToken')  
  });  
  localStorage.setItem('token', res.data.token);  
}
```

Output / Result: Naya access token mil jayega, user ka session bina interruption ke continue rahega.

Real-World Example: Long login sessions (jaise 'remember me') refresh tokens ki wajah se possible hote hain.

Chapter # 15 : React Query (TanStack Query)

React Query server data fetch, cache, aur sync karne ka powerful library hai - loading/error states, caching, aur refetching automatically handle karta hai, jo manual useEffect+Axios se kaafi behtar hai.

Module 1: React Query Basics

Query Client

QueryClient poore React Query system ka central manager hai jo saari queries ka cache store karta hai - app ko QueryClientProvider se wrap karna zaroori hai.

Code Example:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient();

<QueryClientProvider client={queryClient}>
  <App />
</QueryClientProvider>
```

Output / Result: Ab poori app ke andar useQuery/useMutation hooks use ho sakte hain.

Real-World Example: React Query use karne wale har project mein ye setup App ke root par hota hai.

useQuery

useQuery data fetch karne ka main hook hai - automatically loading, error, aur data states manage karta hai, aur result cache karta hai.

Code Example:

```
const { data, isLoading, error } = useQuery({
  queryKey: ['products'],
  queryFn: () => axios.get('/api/products').then(res => res.data)
});
```

Output / Result: 'data' mein products aayenge, 'isLoading' true/false ho kar loading state batayega.

Real-World Example: Product listing, user profile jaisa GET data fetch karne ke liye useQuery standard ban chuka hai.

Query Keys

Query Keys unique identifiers hain jo React Query use karta hai cache ko track karne ke liye - same key ka data automatically share/reuse hota hai.

Code Example:

```
useQuery({ queryKey: ['product', productId], queryFn: fetchProduct });
```

Output / Result: Har alag productId ke liye separate cache entry banegi.

Real-World Example: Detail pages (jaise product/5, product/6) apni alag cache rakhti hain query keys ki wajah se.

Module 2: Mutations & Cache

useMutation

useMutation data change karne wale operations (POST, PUT, DELETE) ke liye hota hai - jaise naya item create karna ya delete karna.

Code Example:

```
const mutation = useMutation({
  mutationFn: (newTodo) => axios.post('/api/todos', newTodo)
}); mutation.mutate({ title: 'Learn React Query' });
```

Output / Result: Naya todo server par create ho jayega, mutation.isSuccess true ho jayega.

Real-World Example: Add to cart, delete comment, update profile - sab mutations se hote hain.

Caching

React Query fetched data ko automatically cache karta hai, taake same data dobara maango to instantly (bina naya request kiye) mil jaye.

Code Example:

```
// Same queryKey dubara use karne par cached data turant milega useQuery({
queryKey: ['products'], queryFn: fetchProducts });
```

Output / Result: Second time page visit karne par products turant (cache se) dikhenge, background mein fresh data bhi fetch hoga.

Real-World Example: Instagram jaise apps jab back navigate karte hain to purana data turant dikhta hai - yehi caching hai.

Invalidation

Invalidation cache ko 'stale' mark karta hai taake React Query naya data fetch kare - usually mutation ke baad related data refresh karne ke liye use hota hai.

Code Example:

```
const queryClient = useQueryClient();

mutation.mutate(newTodo, {
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['todos'] });
  }
});
```

Output / Result: Naya todo add hone ke baad todos list automatically refresh ho jayegi.

Real-World Example: Add/delete operation ke baad list ko turant update dikhane ke liye invalidation use hota hai.

Module 3: Advanced Data Fetching

Pagination

Pagination bare data ko chote chote 'pages' mein todta hai, taake ek sath sara data load na ho aur app fast rahe.

Code Example:

```
const { data } = useQuery({
  queryKey: ['products', page],
  queryFn: () => fetchProducts(page)
});
```

Output / Result: Har page number ke liye alag data fetch aur cache hoga.

Real-World Example: E-commerce product listings (page 1, 2, 3...) pagination se load hoti hain.

Infinite Queries

Infinite Queries 'load more' ya infinite scroll pattern ke liye hain - jab user scroll kare to automatically agla data batch fetch ho.

Code Example:

```
const { data, fetchNextPage } = useInfiniteQuery({
  queryKey: ['posts'],
  queryFn: ({ pageParam = 1 }) => fetchPosts(pageParam),
  getNextPageParam: (lastPage) => lastPage.nextPage });
```

Output / Result: fetchNextPage() call karne par agla batch data mein add ho jayega.

Real-World Example: Instagram, Twitter ka infinite scroll feed isi pattern se implement hota hai.

Chapter 16: Testing

Testing se pata chalta hai code sahi kaam kar raha hai ya nahi, bina manually browser mein click-click kar ke check kiye. Ye chapter React apps test karna sikhata hai.

Module 1: Testing Tools

[Jest](#)

Jest ek JavaScript testing framework hai jo tests run karta hai aur pass/fail results deta hai - assertions, mocking, sab kuch built-in hai.

Code Example:

```
npm install --save-dev jest

// package.json
"scripts": { "test": "jest" }
```

Output / Result: npm test chalane par saari test files run hongy.

Real-World Example: Almost har JavaScript/React project mein Jest (ya Vitest) testing ke liye use hota hai.

React Testing Library

React Testing Library components ko 'user ki tarah' test karne par focus karti hai - implementation details ki bajaye actual UI behavior test hota hai.

Code Example:

```
npm install --save-dev @testing-library/react
```

Output / Result: Ab render(), screen jaise utilities available hongy components test karne ke liye.

Real-World Example: Jest ke sath React Testing Library combo React testing ka industry standard hai.

Module 2: Writing Tests

Unit Testing

Unit Testing chote, individual functions ya pieces of logic ko isolately test karta hai - jaise ek calculation function sahi result de raha hai ya nahi.

Code Example:

```
function add(a, b) { return a + b; }

test('adds 2 + 3 to equal 5', () => { expect(add(2, 3)).toBe(5);
});
```

Output / Result: Test pass hoga agar add(2,3) actually 5 return kare.

Real-World Example: Price calculation, discount logic jaisi functions unit tests se verify ki jati hain.

Component Testing

Component Testing React components ko render karke check karta hai ke woh sahi UI dikha rahe hain aur user interactions (click, type) par sahi react kar rahe hain.

Code Example:

```
import { render, screen, fireEvent } from '@testing-library/react'; import
Counter from './Counter';

test('increments count on click', () => {
  render(<Counter />);
  fireEvent.click(screen.getByText('Increment'));
  expect(screen.getByText('Count: 1')).toBeInTheDocument(); });
```

Output / Result: Test pass hoga agar button click karne par count actually 1 ho jaye.

Real-World Example: Login form, cart button jaisi important UI features par tests likh kar bugs production jaane se pehle pakre jate hain.

Chapter # 17 : Deployment

Deployment ka matlab hai apna project internet par live karna, taake koi bhi duniya mein kahin se bhi usay access kar sake. Ye chapter React app ko free platforms par deploy karna sikhati hai.

Module 1: Preparing for Deployment

Build Project

Deploy karne se pehle project ko 'build' karna zaroori hai - ye development code ko optimized, minified, productionready files mein convert karta hai.

Code Example:

```
npm run build
```

Output / Result: Ek 'dist' folder banega jisme final optimized HTML/CSS/JS files hongi.

Real-World Example: Koi bhi platform (Vercel, Netlify) deploy karne se pehle ye build step automatically ya manually chalata hai.

Environment Variables

Production deployment ke waqt environment variables (API keys, URLs) ko hosting platform ki settings mein securely set kiya jata hai, code mein hardcode nahi kiya jata.

Code Example:

```
# Vercel/Netlify dashboard mein:  
VITE_API_URL = https://api.myapp.com
```

Output / Result: App production mein sahi API URL use karegi, bina code change kiye.

Real-World Example: Development aur production ke alag-alag backend URLs is tarah manage kiye jate hain.

Module 2: Hosting Your App

Deploy on Vercel

Vercel React/Vite apps deploy karne ke liye sabse popular free platform hai - GitHub repository connect karke automatic deployment set ho jati hai.

Code Example:

```
# 1. vercel.com par GitHub se login karein  
# 2. 'New Project' -> apna repo select karein  
# 3. Deploy click karein
```

Output / Result: Kuch seconds mein app ek live URL (jaise myapp.vercel.app) par accessible ho jayegi.

Real-World Example: Har naya commit GitHub par push karne se Vercel automatically naya version deploy kar deta hai.

Deploy on Netlify

Netlify bhi Vercel jaisa hi free static hosting platform hai, GitHub integration ke sath automatic builds aur deployments deta hai.

Code Example:

```
# 1. netlify.com par login  
# 2. 'Add new site' -> GitHub repo select  
# 3. Build command: npm run build, Publish directory: dist
```

Output / Result: App live ho jayegi ek URL par jaise myapp.netlify.app

Real-World Example: Kai freelancers aur startups apni landing pages Netlify par free host karte hain.

Custom Domain (Basic)

Custom Domain se apni website ko apna khud ka naam (jaise myapp.com) diya ja sakta hai, hosting platform ki settings mein DNS configure karke.

Code Example:

```
# Vercel/Netlify Dashboard -> Domains -> Add Domain  
# Apne domain provider (GoDaddy waghera) mein DNS records add karein
```

Output / Result: Kuch der baad website 'myapp.vercel.app' ki jagah 'myapp.com' par accessible ho jayegi.

Real-World Example: Professional businesses hamesha apna custom domain use karte hain, free subdomain ki bajaye.

Chapter # 18 : Advanced Projects

Ab tak seekhe gaye concepts (React, Router, Tailwind, Redux, API integration) ko real projects mein combine kar ke practice karne ka waqt hai. Har project ke liye overview, key features, aur use hone wale tech stack diya gaya hai.

Module 1: Beginner-Friendly Projects

Portfolio Website

Apna personal introduction, skills, projects, aur contact info dikhane wali single-page website. Ye har developer ka pehla real project hona chahiye.

Code Example:

```
Sections: Hero, About, Skills, Projects, Contact  
Tech: React + Tailwind CSS + Framer Motion (optional animations)
```

Output / Result: Ek professional, responsive personal website jo job/freelance applications mein share ki ja sake.

Real-World Example: Job interviews mein recruiters sabse pehle candidate ka portfolio dekhte hain.

Todo App

Tasks add, complete mark, aur delete karne wali classic beginner app - React state aur CRUD logic practice karne ke liye perfect.

Code Example:

```
Features: Add task, Mark complete, Delete task, Filter (All/Active/Completed) Tech:  
React (useState) + LocalStorage
```

Output / Result: Ek fully functional task manager jisme data browser refresh ke baad bhi save rahe (localStorage se).

Real-World Example: Todo app har React course ka 'hello world' project hai kyunke ismein state management ki basics cover hoti hain.

Weather App

User kisi city ka naam type kare aur real-time weather data external API se fetch ho kar dikhaya jaye.

Code Example:

```
Features: City search, Temperature, Weather icon, Forecast Tech:  
React + Axios + OpenWeatherMap API
```

Output / Result: Live weather data dikhne wali app jo real API integration sikhati hai.

Real-World Example: Ye project API integration aur async data handling practice karne ke liye best hai.

Notes App

Notes create, edit, aur delete karne wali app, categories/colors ke sath - CRUD aur local storage ka combined practice.

Code Example:

```
Features: Create note, Edit note, Delete note, Color labels Tech:  
React + Context API + LocalStorage
```

Output / Result: Google Keep jaisi simple lekin functional notes-taking app.

Real-World Example: Ye project Context API se global notes state manage karna sikhati hai.

Module 2: Advanced Projects

E-commerce Store

Products listing, cart, checkout, aur authentication ke sath poora online store - is course ke almost saare concepts isme combine hote hain.

Code Example:

Features: Product listing, Cart (Redux), Login/Signup (JWT), Checkout
Tech: React + Redux Toolkit + React Router + Tailwind + API

Output / Result: Ek full-featured e-commerce app jo real production project jaisi lage.

Real-World Example: Ye project portfolio ke liye sabse impressive hota hai kyunke isme frontend ka almost har concept use hota hai.

Admin Dashboard

Charts, tables, aur data management ke sath admin panel - nested routes aur layout patterns practice karne ke liye.

Code Example:

Features: Sidebar navigation, Data tables, Charts, User management Tech:
React + React Router (nested routes) + Tailwind + Recharts

Output / Result: Ek professional-looking dashboard jisme sidebar se different sections access hote hain.

Real-World Example: Almost har SaaS company (analytics tools, CRMs) ka internal admin panel isi structure par based hota hai.

Chat App

Real-time messaging app jisme users ek dusre ko messages bhej sakein - real-time updates ke liye WebSockets/Socket.io use hoti hai.

Code Example:

Features: Real-time messages, Online status, Message history Tech:
React + Socket.io + Node.js backend

Output / Result: WhatsApp jaisa basic real-time chat interface.

Real-World Example: Ye project real-time data handling (beyond simple REST APIs) sikhane ke liye advanced hai.

Blog CMS

Blog posts create, edit, publish karne wala Content Management System - rich text editor aur authentication ke sath.

Code Example:

Features: Create/Edit posts, Rich text editor, Categories, Author login
Tech: React + React Router + Authentication + API

Output / Result: Ek Medium-jaisi simplified blogging platform.

Real-World Example: Content-heavy websites (news, blogs) is tarah ke CMS par based hoti hain.

Movie App

Movies browse aur search karne wali app, external Movie API (jaise TMDb) se data fetch karke.

Code Example:

Features: Movie search, Details page, Trending movies, Favorites Tech:
React + React Router + TMDb API

Output / Result: Netflix-inspired movie browsing interface, real API data ke sath.

Real-World Example: Ye project dynamic routing (useParams) aur external API integration ka bohot achha combined practice hai.

Food Delivery App

Restaurant menu browse karke cart mein items add karna aur order place karna - e-commerce jaisa hi flow, foodspecific UI ke sath.

Code Example:

Features: Restaurant list, Menu, Cart, Order tracking (basic) Tech:
React + Redux Toolkit + Tailwind + API

Output / Result: Foodpanda/Uber Eats jaisi basic food ordering app.

Real-World Example: Ye project state management (cart across multiple restaurants) ka advanced practice deta hai.

Chapter # 19: Interview Preparation

Ye chapter job interviews ke liye taiyari karata hai - technical questions ke sample answers, aur HR/resume/portfolio tips, taake aap confidently interview de sakein.

Module 1: Technical Interview Questions

HTML Interview Questions

Sample Q&A:

Q: Semantic HTML kya hai?

A: Woh tags jo apne content ka meaning batate hain (header, article, footer), sirf div/span ki bajaye - SEO aur accessibility behtar karte hain.

Q: HTML5 ke naye features kya hain?

A: Semantic tags, audio/video support, canvas, local storage, form validation attributes.

Code Example:

```
<article>
  <header><h2>Blog Title</h2></header>
  <p>Content...</p>
</article>
```

Output / Result: Screen readers aur search engines is structure ko better samajhte hain.

Real-World Example: Frontend interviews mein semantic HTML almost hamesha pucha jata hai.

CSS Interview Questions

Sample Q&A:

Q: Box model kya hai?

A: Content, padding, border, margin ka layered structure jo har element ka size determine karta hai.

Q: Flexbox vs Grid mein kya farq hai?

A: Flexbox 1D layout (row ya column) ke liye hai, Grid 2D layout (rows aur columns ek sath) ke liye behtar hai.

Code Example:

```
/* Flexbox: 1D */
display: flex; /*
Grid: 2D */
display: grid;
```

Output / Result: Interview mein practical example ke sath explain karna zyada impressive lagta hai.

Real-World Example: CSS specificity aur positioning bhi common interview topics hain.

JavaScript Interview Questions Sample

Q&A:

Q: let, const, var mein farq?

A: var function-scoped hai aur hoist hota hai, let/const block-scoped hain; const reassign nahi ho sakta.

Q: Closures kya hain?

A: Ek function jo apne outer function ki variables ko yaad rakhta hai, chahe outer function khatam ho chuka ho.

Code Example:

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;    return count;  
  }; }  
const counter = outer(); counter();  
// 1
```

Output / Result: 'counter' function 'count' variable ko remember rakhta hai - ye closure hai.

Real-World Example: Closures, promises, aur event loop JavaScript interviews ke sabse common topics hain.

React Interview Questions

Sample Q&A:

Q: Virtual DOM kya hai?

A: Real DOM ki ek lightweight JavaScript copy jo React use karta hai efficiently updates calculate karne ke liye, phir sirf changed parts real DOM mein update karta hai.

Q: useState vs useReducer kab use karein?

A: Simple, independent state ke liye useState; complex, related state logic (multiple sub-values) ke liye useReducer.

Code Example:

```
// useState - simple  
const [count, setCount] = useState(0);  
  
// useReducer - complex  
const [state, dispatch] = useReducer(reducer,  
  initialState);
```

Output / Result: Dono hi state manage karte hain, lekin complexity ke hisab se sahi choice karna important hai.

Real-World Example: React hooks aur performance optimization (memo, useMemo) frontend interviews mein deeply pucha jata hai.

Redux Interview Questions

Sample Q&A:

Q: Redux kyun use karte hain agar Context API maujood hai?

A: Redux bare apps mein predictable state updates, dev tools (time-travel debugging), aur better performance deta hai jab state complex ho.

Q: Action aur Reducer mein farq?

A: Action batata hai 'kya hua' (data), Reducer batata hai 'state kaise change hogi' us action ke response mein.

Code Example:

```
// Action: describes what happened  
{ type: 'cart/addItem', payload: product }  
  
// Reducer: describes how state changes  
(state, action) => ({ ...state, items: [...state.items, action.payload] })
```

Output / Result: Ye separation Redux ki state predictable aur debug-friendly banati hai.

Real-World Example: Bare companies (jinke paas complex global state hai) Redux experience ko highly value karti hain.

Git & GitHub Questions Sample

Q&A:

Q: Merge aur Rebase mein farq?

A: Merge history mein ek naya 'merge commit' banata hai, Rebase commits ko replay karke linear history banata hai (bina merge commit ke).

Q: Merge conflict kya hai aur kaise resolve karte hain?

A: Jab do branches ne ek hi line/file mein different changes kiye hon, Git khud decide nahi kar pata - manually conflict markers dekh kar sahi code choose karna padta hai.

Code Example:

```
<<<<<< HEAD
your changes
=====
their changes
>>>>>>
branch-name
```

Output / Result: Conflict resolve karne ke baad file ko dobara git add aur commit karna hota hai.

Real-World Example: Team collaboration roles mein Git conflict resolution ek practical, hands-on interview question hoti hai.

Module 2: Non-Technical Preparation

HR Interview Questions

Common HR questions: 'Apne baare mein batao', 'Apni strengths/weaknesses kya hain', 'Ye job kyun chahiye', 'Apne aap ko 5 saal baad kahan dekhte hain'. Answers concise, honest, aur position se related hone chahiye.

Code Example:

```
Tip: STAR method use karein answers ke liye: Situation
- Task - Action - Result
```

Output / Result: Structured answers interviewer par zyada professional impression chorte hain.

Real-World Example: Technical skills ke sath HR round bhi utna hi important hota hai final selection ke liye.

Resume Building

Achi resume mein: clear contact info, concise summary, relevant skills (React, Redux, Git waghera), projects (links ke sath), aur education - ek page mein rakhna best hai.

Code Example:

```
Sections: Header, Summary, Skills, Projects, Experience, Education
```

Output / Result: Recruiter 10 second mein resume scan karke decide karta hai interview dena hai ya nahi - isliye clarity important hai.

Real-World Example: ATS (Applicant Tracking Systems) simple, clean-formatted resumes ko behtar parhte hain complex designs ke muqable.

Portfolio Building

Portfolio mein best 3-4 projects hone chahiye, live links + GitHub code ke sath, har project ki short description ke sath ke kya problem solve kiya aur kaunsi technologies use hui.

Code Example:

Har project card mein shamil karein:

- Live Demo link
- GitHub repo link
- Tech stack used
- 2-3 line description

Output / Result: Ek strong portfolio jo practically dikhaye ke aap kya bana sakte hain, sirf resume par likhne ki bajaye.

Real-World Example: Freelancing aur job dono mein portfolio hi sabse zyada trust build karta hai.

— End of Course — *Ab practice karein, projects banayein, aur interview ke liye taiyar ho jayein* —

-Frontend Development Notes-

Final Thoughts

Learning frontend development is not about memorizing syntax—it's about understanding concepts and applying them in real-world projects.

Keep practicing HTML, CSS, JavaScript, React, and modern frontend tools regularly. Build projects, solve problems, and stay updated with new technologies.

Remember:

- ☐ Learn Concepts
- ☐ Practice Daily
- ☐ Build Projects
- ☐ Fix Bugs
- ☐ Never Stop Learning

"Great developers are made by consistent practice, not by completing a course."

Best Wishes for Your Frontend Development Journey!

— End of Frontend Development Notes —

Prepared By: **Engr. Umar Naveed**

Frontend Developer

✉ **Email:** (enrumarnaveed@gmail.com)

🌐 **GitHub:** ([enrumarnaveed](#))

🔗 **LinkedIn:** ([0umarnaveed](#))

© 2026 All Rights Reserved.